

AWS Machine Learning specialty notes

For training and deploying sequence-to-sequence (seq2seq) models in AWS, selecting the right type of instance is crucial for efficient performance and cost management. Seq2seq models are commonly used for tasks like machine translation, text summarization, and conversational AI, which typically require significant computational resources, especially during training. Here's a basic guide to choosing AWS instance types for these purposes:

AWS Sagemaker File Mode and Pipe Mode

In AWS, particularly when using Amazon SageMaker, you often encounter two different ways to input data into your machine learning models during training: **File Mode** and **Pipe Mode**. These modes determine how data is transferred from storage (like Amazon S3) to the model for training. Understanding these modes can help optimize the performance and efficiency of your machine learning training processes.

File Mode

What It Is: In File Mode, your training data is first downloaded in entirety from the source (usually an S3 bucket) to the local storage of the training instance before the training job starts.

Advantages:

- **Simplicity:** Training algorithms can access the data locally, which can simplify code since the data is readily available as if it were on a local disk.
- **Reusability:** The data can be reused multiple times during training without needing to download it again.

Disadvantages:

- **Slower Start:** Training cannot start until all the data is downloaded, which can delay the beginning of training, especially with large datasets.
- **Storage Limitations:** The amount of data you can use is limited by the storage capacity of the training instance.

Example: Suppose you have a dataset of images for a computer vision model stored in S3. In File Mode, you would download all these images to your SageMaker instance before starting the model training. This is useful when your model needs to access different parts of the data multiple times during training.

Pipe Mode

What It Is: Pipe Mode streams data directly into the algorithm while it's running, without being stored on the disk. It's a more complex way to handle input data but can start processing the data immediately as it's received.

Advantages:

- **Faster Start:** Training can start as soon as the first batch of data is available, which can significantly reduce the start time for training jobs.

- **Scalability:** Can handle datasets that are larger than the available disk space since the data is processed on-the-fly and not stored locally.

Disadvantages:

- **Complexity:** Implementing and debugging can be more challenging since data handling and processing must be managed carefully to ensure the streaming data is processed correctly.
- **One-Time Use:** Data can typically only be used once. If the algorithm needs to access the data multiple times, Pipe Mode might not be suitable.

Example: If you're training a large-scale machine learning model on a very large dataset of text data for natural language processing, Pipe Mode allows you to stream text data directly from S3 into your SageMaker training algorithm. This is particularly useful when the dataset is too large to fit on the local disk of the training instance.

Choosing Between File and Pipe Mode

The choice between File and Pipe Mode depends largely on the size of your dataset and the specifics of your training process:

- Use **File Mode** when your dataset is relatively small, or your training process requires multiple passes over the data.
- Use **Pipe Mode** when dealing with very large datasets, or when training speed and starting times are critical, and the training algorithm can process data in a single pass.

AWS SageMaker's support for both modes provides flexibility to optimize data handling according to the needs of your specific machine learning project.

Data Warehouse vs Data Lake

A data warehouse can only store structured data whereas a data lake can store structured, semi-structured and unstructured data

A data lake is a centralized repository that allows you to store all your structured and unstructured data at any scale. You can store your data as-is, without having to first structure the data, and run different types of analytics—from dashboards and visualizations to big data processing, real-time analytics, and machine learning to guide better decisions.

A data warehouse is a large collection of business data used to help an organization make decisions. A data warehouse periodically pulls data from the business apps and systems; then, the data goes through formatting and import processes to match the data already in the warehouse.

Data lakes and data warehouses are both widely used for storing big data, but they are not interchangeable terms. A data lake is a vast pool of raw data, the purpose for which is not yet defined. A data warehouse is a repository for structured, filtered data that has already been processed for a specific purpose.

A data warehouse can only store structured data whereas a data lake can store structured, semi-structured and unstructured data.

DATA WAREHOUSE	vs.	DATA LAKE
structured, processed	DATA	structured / semi-structured / unstructured, raw
schema-on-write	PROCESSING	schema-on-read
hierarchically archived	STORAGE	object-based, no hierarchy
less agile, fixed configuration	AGILITY	highly agile, configure and reconfigure as needed
mature	SECURITY	maturing
business professionals	USERS	data scientists et. al.

AWS Sagemaker Instances. Which instance to use for training and inference?

1. Compute Optimized Instances (C Series)

- **Use Case:** If your seq2seq model doesn't heavily rely on GPU acceleration but needs robust CPU performance for tasks like inference or light training.

Explanation: Compute optimized instances offer high performance processors that can handle demanding computational tasks efficiently. They are suitable for applications where CPU capability is more important than GPU.

2. GPU Instances (P Series and G Series)

- **Use Case:** For training more complex seq2seq models, such as those used in deep learning frameworks that benefit from GPU acceleration.
- **Explanation:**
 - **P Series:** These instances are designed primarily for general-purpose GPU computing. They are ideal for heavy machine learning training and deep learning tasks. Using these can drastically reduce the time it takes to train complex seq2seq models.
 - **G Series:** These are good for both machine learning and graphics-intensive applications. If your seq2seq tasks involve rendering or real-time interactive applications alongside ML computations, G instances can be beneficial.

3. Memory Optimized Instances (R Series and X Series)

- **Use Case:** Useful when your seq2seq models require handling large datasets in memory or need to manage large vocabularies and context windows, which is common in NLP tasks.
- **Explanation:** Memory optimized instances provide a high ratio of RAM to CPU, which helps in processing large in-memory databases or performing high-throughput data processing tasks. They help reduce the time to train and infer by keeping larger portions of data in fast-access memory.

4. General Purpose Instances (M Series and T Series)

- **Use Case:** Suitable for development, testing, or if your seq2seq models are relatively simple and do not require extensive CPU, GPU, or memory resources.
- **Explanation:**
 - **M Series:** Offers a balance of compute, memory, and networking resources and is good for general-purpose tasks that do not have specialized requirements.
 - **T Series:** Provides a baseline level of CPU performance with the capability to burst to higher levels when needed. These are cost-effective and suitable for small or medium databases, and low-latency interactive applications.

Selecting the Right Instance

When choosing an instance for seq2seq tasks in AWS, consider the following:

- **Model Complexity:** More complex models, such as those with larger datasets or more layers, generally benefit from GPU instances.
- **Budget Constraints:** While GPU instances offer significant speed in training, they are also more expensive. Balance your budget with your need for speed and performance.
- **Development vs. Production:** Use more cost-effective instances like the T Series for development and testing. Scale up to more powerful instances for training and production workloads.

Using the right AWS instance for your seq2seq models can optimize both performance and cost, ensuring efficient training and inference for your machine learning applications.

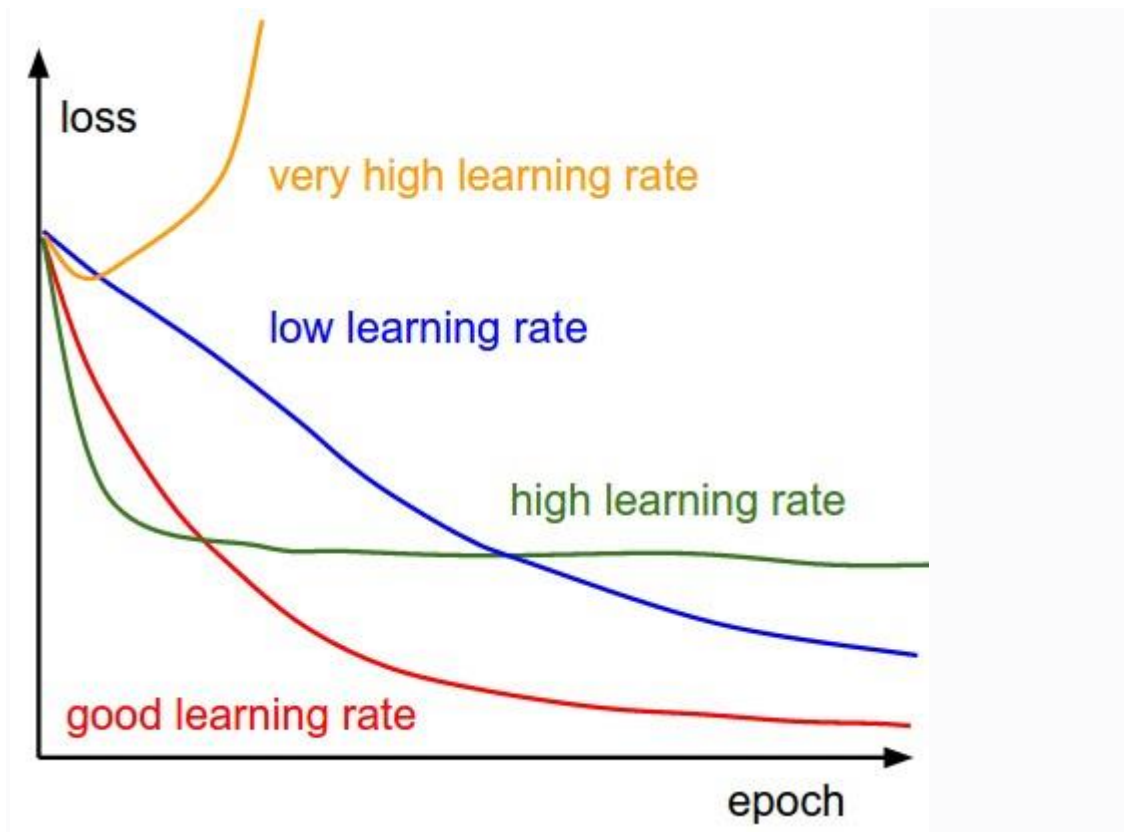
Generic Model Hyperparameters

Fine-tuning a neural network involves adjusting its hyperparameters to improve its performance. Hyperparameters are the settings or configurations that govern the entire training process but are not changed during training itself. Here are some of the most common hyperparameters you might adjust when fine-tuning a neural network, explained in simple, layman terms:

1. Learning Rate

- **What It Does:** The learning rate is perhaps the most crucial hyperparameter in neural network training. It determines how much the model's weights should change in response to the error it sees after each batch of data.

The learning rate is a constant value used in the Stochastic Gradient Descent (SGD) algorithm. Learning rate affects the speed at which the algorithm reaches (converges to) the optimal weights. The SGD algorithm makes updates to the weights of the linear model for every data example it sees. The size of these updates is controlled by the learning rate.



- Too large a learning rate might prevent the weights from approaching the optimal solution. Too small a value results in the algorithm requiring many passes to approach the optimal weights. In Amazon ML, the learning rate is auto-selected based on your data.
- A Loss function evaluates how effective an algorithm is at modeling the data. For example, if a model consistently predicts values that are very different from the actual values, it returns a large loss. Depending on the training algorithm, more than one loss function might be used.

- **Layman's Term:** Think of it like adjusting the size of steps you take; too large, and you might overshoot your destination, too small, and it might take forever to get there.

2. Number of Epochs

- **What It Does:** An epoch is a complete pass through the entire training dataset. The number of epochs is the number of times the learning algorithm will work through the entire dataset.
- **Layman's Term:** It's like reading the same book several times. Each time you read it, you might pick up something new, but read it too many times, and it becomes a waste of time.

3. Batch Size

- **What It Does:** The batch size is the number of samples processed before the model is updated.
- **Layman's Term:** Imagine you're learning new words by reading a bunch of flashcards. Batch size is how many flashcards you look at before trying to recall and consolidate what you've learned.

The batch size in machine learning training significantly impacts several key aspects of the learning process. Smaller batch sizes tend to improve generalization due to the noise in gradient updates, which can help avoid local minima and promote better model performance on unseen data. However, they may result in slower convergence and less stable training progress. On the other hand, larger batch sizes offer more stable and faster convergence by utilizing the computational power of modern hardware like GPUs more efficiently, but they can lead to overfitting due to their tendency to converge to sharp minimizers. Choosing the right batch size typically involves balancing these factors, requiring experimentation to optimize model accuracy, training stability, and computational efficiency.

4. Activation Functions

- **What It Does:** Activation functions help decide whether a neuron should be activated or not, by calculating a weighted sum and further adding bias with it. This determines the output of that neuron, whether it should fire (activate) or not.
- **Layman's Term:** Think of it as a gate at a park entrance. If the person has a ticket (input signal strong enough), the gate allows them in (neuron activates).

5. Dropout Rate

- **What It Does:** Dropout is a regularization method where randomly selected neurons are ignored during training. They are “dropped-out” randomly. This means that their contribution to the activation of downstream neurons is temporally removed on the forward pass and any weight updates are not applied to the neuron on the back pass.
- **Layman's Term:** Imagine you're doing a group project but randomly give some team members a day off. This ensures that no one person becomes too crucial, preventing any one pattern (or mistake) from dominating.

6. Optimization Algorithms

- **What It Does:** These algorithms are how the model adjusts its weights based on the error it measures. Common optimizers include SGD (Stochastic Gradient Descent), Adam, RMSprop, etc.
- **Layman's Term:** Think of these as different strategies for searching through a maze to find the quickest exit. Some strategies might be better depending on the layout of the maze.

7. Weight Initialization

- **What It Does:** This hyperparameter defines the method to set the initial random weights of neural network layers.
- **Layman's Term:** It's like setting up the starting blocks for a race. A good position can give you a head start, while a bad one can hinder your progress right from the beginning.

These hyperparameters are critical as they directly influence how quickly a network learns, how well it performs, and its ability to generalize from training data to unseen data. Fine-tuning these parameters can significantly improve your model's accuracy and efficiency.

All about Databases

Redshift vs. Aurora

Amazon Redshift and **Amazon Aurora** are both part of Amazon Web Services (AWS), but they serve different purposes and are optimized for different types of tasks.

Amazon Redshift: This is a data warehousing service, which means it's designed for analyzing large volumes of data using complex queries. Imagine it like a large library where you can go to find answers to complicated questions that require pulling information from various books. It is optimized for Online Analytical Processing (OLAP), which supports the complex querying of large datasets.

Amazon Aurora: This is a relational database service optimized for Online Transaction Processing (OLTP). You can think of it as a high-performance cashier system designed to handle a lot of rapid, small transactions like recording sales, updating inventory, etc. Aurora can support a variety of database engines, but it's particularly known for being highly scalable and durable, making it a good fit for applications that require reliable, consistent database performance.

OLAP vs. OLTP

OLTP (Online Transaction Processing): This is all about handling a large number of short online transactions (INSERT, UPDATE, DELETE). The main emphasis is on fast query processing, maintaining data integrity in multi-access environments, and an effective response to large numbers of users performing small transactions. Think of a busy e-commerce site during a flash sale where thousands of people are placing orders simultaneously.

OLAP (Online Analytical Processing): This deals with systems designed to answer complex queries, not to process transactions. OLAP is used in scenarios where you need to analyze large amounts of data to make decisions. For example, a business analyst might use an OLAP system to determine sales trends over a year. These systems are optimized for reading and querying large amounts of data in a multidimensional way, such as comparing sales across different regions and time periods.

Redshift Spectrum

Using Amazon Redshift Spectrum, you can efficiently query and retrieve structured and semistructured data from files in Amazon S3 without having to load the data into Amazon Redshift tables. Redshift Spectrum queries employ massive parallelism to execute very fast against large datasets. Much of the processing occurs in the Redshift Spectrum layer, and most of the data remains in Amazon S3. Multiple clusters can concurrently query the same dataset in Amazon S3 without the need to make copies of the data for each cluster.

Redshift Spectrum is a built-in feature of Amazon Redshift, and your existing queries and BI tools will continue to work seamlessly. Under the covers, AWS manages a fleet of thousands of Redshift Spectrum nodes spread across multiple Availability Zones. These are transparently scaled and allocated to your queries based on the data that you need to process, with no provisioning or commitments. Redshift Spectrum is also highly concurrent—you can access your Amazon S3 data from any number of Amazon Redshift clusters.

AWS Athena

Amazon Athena supports a wide variety of data formats like CSV, TSV, JSON, or Textfiles and also supports open-source columnar formats such as Apache ORC and Apache Parquet. Athena also supports compressed data in Snappy, Zlib, LZO, and GZIP formats. By compressing, partitioning, and using columnar formats, you can improve performance and reduce your costs.

Parquet and ORC file formats both support predicate pushdown (also called predicate filtering). Parquet and ORC both have blocks of data that represent column values. Each block holds statistics for the block, such as max/min values. When a query is being executed, these statistics determine whether the block should be read or skipped.

Athena charges you by the amount of data scanned per query. You can save on costs and get better performance if you partition the data, compress data, or convert it to columnar formats such as Apache Parquet.

Dataset	Size on Amazon S3	Query Run time	Data Scanned	Cost
Data stored as CSV files	1 TB	236 seconds	1.15 TB	\$5.75
Data stored in Apache Parquet format*	130 GB	6.78 seconds	2.51 GB	\$0.01
Savings / Speedup	87% less with Parquet	34x faster	99% less data scanned	99.7% savings

Apache Parquet is an open-source columnar storage format that is 2x faster to unload and takes up 6x less storage in Amazon S3 as compared to other text formats. One can COPY Apache Parquet and Apache ORC file formats from Amazon S3 to your Amazon Redshift cluster. Using AWS Glue, one can configure and run a job to transform CSV data to Parquet. Parquet is a columnar format that is well suited for AWS analytics services like Amazon Athena and Amazon Redshift Spectrum.

AWS Athena is a service that allows you to perform analyses and run queries on large amounts of data stored in AWS S3, without having to set up and manage complex databases. Here's how it simplifies working with big data:

Simplified Overview of AWS Athena

1. **Serverless Query Service:** Athena is serverless, which means you don't have to worry about setting up or managing servers. You just use the service to run queries; AWS handles all the underlying infrastructure.
2. **Direct Operation on Data in S3:** You don't need to move your data into a separate storage system or database. Athena works directly on data stored in Amazon S3. Whether your data is in formats like CSV, JSON, Parquet, or others, Athena can handle it right where it is.
3. **SQL-Based:** If you know SQL (Structured Query Language), you can query your data in S3 immediately. This makes it accessible not just to data engineers but to anyone familiar with SQL, including business analysts and other non-engineering roles.
4. **Integration with AWS Glue:** Athena integrates seamlessly with AWS Glue, which is a service for cataloging data. Glue automatically discovers and prepares data for analytics, creating a data catalog that Athena can query.

How Athena Works - A Simple Explanation

- **Store Your Data:** Your data resides in AWS S3 in whatever format it's already in.
- **Write a SQL Query:** Use standard SQL to write queries for analyzing your data.
- **Run the Query in Athena:** Execute the query in Athena. Athena reads the data directly from S3 and returns the results.
- **Visualization and Analysis:** You can connect Athena to AWS QuickSight or other analytics tools for visualizing the results or integrating them into your business reports and dashboards.

Business Use Cases for AWS Athena

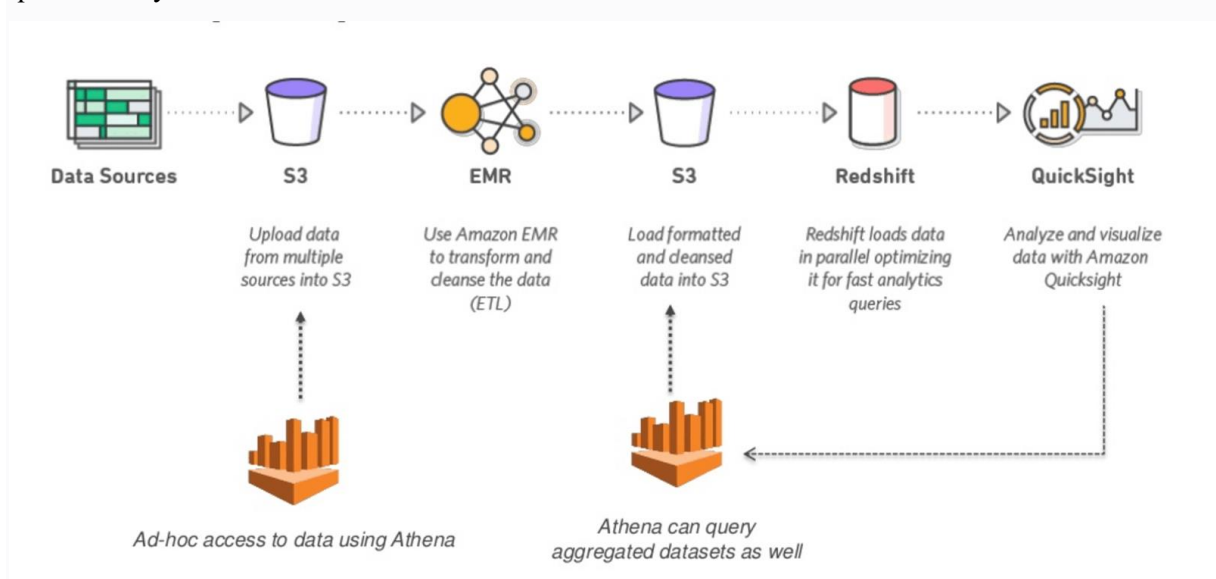
1. **Log Analysis:** Companies store vast amounts of log data (from web servers, applications, etc.) in S3. Athena can quickly query these logs for troubleshooting, security analyses, and operational monitoring.
2. **Ad-hoc Business Intelligence:** Business users can perform ad-hoc analysis on business data (like sales, customer behavior) stored in S3. With SQL queries, they can uncover insights without waiting for IT to prepare and load data into a traditional data warehouse.
3. **Data Lake Exploration:** For organizations using a data lake architecture, where a vast amount of data in various formats is stored in S3, Athena allows quick exploration of this data. This helps in making data-driven decisions faster.
4. **Interactive Querying for Quick Decisions:** Whenever quick, interactive querying of data is required, Athena provides a powerful solution without the overhead of traditional data warehouses.

Advantages of Using Athena

- **Cost-Effective:** You pay only for the queries you run. The cost depends on the amount of data scanned by each query. You can reduce costs by compressing, partitioning, or converting data into columnar formats which help scan less data.
- **No Infrastructure Management:** As a serverless service, Athena removes the complexity of scaling, managing, and operating the infrastructure for data querying and analysis.
- **Flexibility:** Easily integrate with a variety of AWS and external services for further processing, data transformation, or visualization.

In summary, AWS Athena offers a flexible, powerful, and cost-effective way to perform data analysis directly on data stored in S3 using standard SQL, making it an ideal tool for businesses looking to leverage their data without significant upfront infrastructure investment

Amazon Athena is an interactive query service that makes it easy to analyze data in Amazon S3 using standard SQL. Athena is serverless, so there is no infrastructure to manage, and you pay only for the queries that you run.



Athena is easy to use. Simply point to your data in Amazon S3, define the schema, and start querying using standard SQL. Most results are delivered within seconds. With Athena, there's no need for complex ETL jobs to prepare your data for analysis. This makes it easy for anyone with SQL skills to quickly analyze large-scale datasets.

AWS Data Pipeline

AWS DataPipeline is an orchestration service that helps you define, schedule, and manage complex data processing workflows. It allows you to combine various AWS services and third-party applications to build end-to-end data processing pipelines. DataPipeline is a versatile choice when you need to integrate and move data between different services and systems.

Use DataPipeline when:

You require an orchestration service to manage complex data workflows.

You want to integrate and transform data across multiple AWS services.

You need to coordinate data processing tasks with external systems.

Example: Use DataPipeline to schedule regular data imports from an S3 bucket to a Redshift cluster, perform data transformations using Glue, and trigger an AWS Lambda function for further processing or notifications.

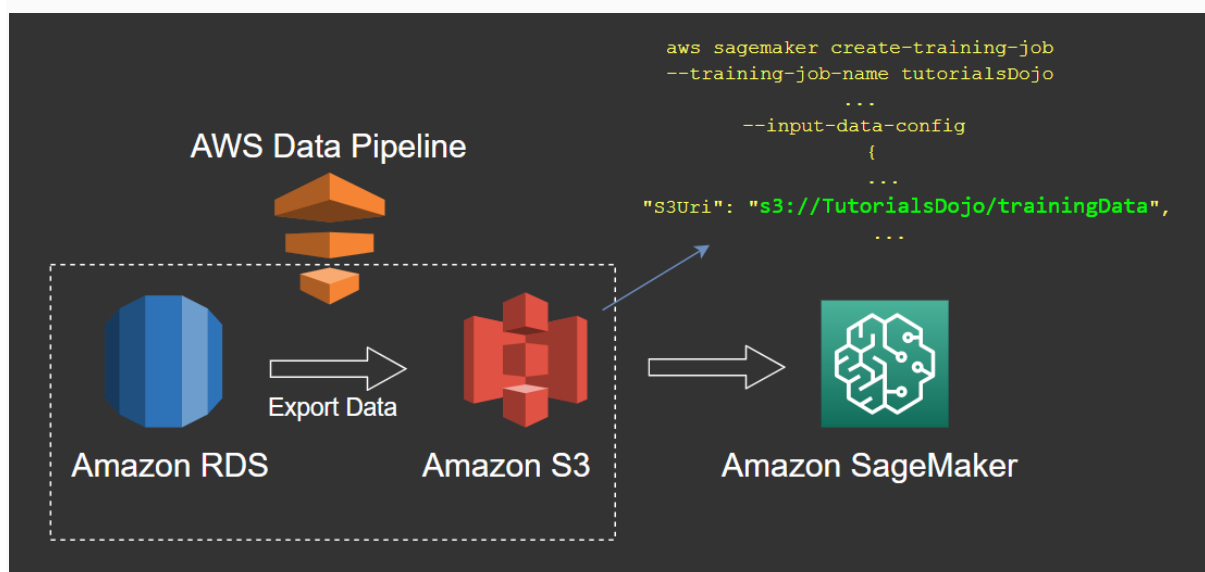
When to Use Data Pipeline:

Automating ETL Jobs: When you need to regularly move and transform data for analysis or storage, especially if you require complex transformations or processing.

Data Workflow Automation: When managing complex data workflows that involve multiple steps and dependencies, ensuring that data processing tasks are carried out efficiently.

Periodic Data Processing: For scenarios where data needs to be processed on a regular schedule, such as daily sales reports or monthly inventory updates.

AWS Data Pipeline is a web service that helps you reliably process and move data between different AWS compute and storage services, as well as on-premises data sources, at specified intervals. With AWS Data Pipeline, you can regularly access your data where it's stored, transform and process it at scale, and efficiently transfer the results to AWS services such as Amazon S3, Amazon RDS, Amazon DynamoDB, and Amazon EMR.



To train a model in SageMaker, you create a training job. The training job includes the following information:

- **The URL of the Amazon Simple Storage Service (Amazon S3)** bucket where you've stored the training data.
- The compute resources that you want SageMaker to use for model training. Compute resources are ML compute instances that are managed by SageMaker.
- The URL of the S3 bucket where you want to store the output of the job.
- The Amazon Elastic Container Registry path where the training code is stored.

AWS DMS (Database Migration Service)

AWS DMS is a fully managed service that helps migrate, replicate, and continuously capture data changes between different databases. It supports various source and target databases, making it an ideal choice for database migrations, data synchronization, and real-time data replication.

Use DMS when:

- You need to migrate data between different databases or data warehouses.
- You want to replicate data changes in real-time.
- You require minimal downtime during database migrations.

Example: Use DMS to migrate data from an on-premises Oracle database to an Amazon RDS instance, ensuring data consistency and minimal downtime.

When to Use DMS:

- **Migrating Databases to AWS:** When you need to move your database to AWS to take advantage of cloud scalability and flexibility.
- **Database Consolidation:** When consolidating multiple databases into a single database system for easier management and cost efficiency.
- **Disaster Recovery:** Setting up a secondary database in a different location for disaster recovery purposes.

AWS Lake Formation

AWS Lake Formation makes it easier to centrally govern, secure, and globally share data for analytics and machine learning (ML). With Lake Formation, you can centralize data security and governance using the AWS Glue Data Catalog, letting you manage metadata and data permissions in one place with familiar database-style features. It also delivers fine-grained data access control, so you can help ensure users have access to the right data, down to the row and column level. You can then scale permissions across your users. Lake Formation also makes it easier to share data internally across your organization, across Regions, and externally using AWS Data Exchange, letting you create a data mesh or meet other data sharing needs with no data movement. And, because Lake Formation tracks data interactions by role and user, it provides comprehensive data access auditing to help ensure the right data was accessed by the right users at the right time.

AWS Lake Formation is a service designed to simplify the setup, security, and management of data lakes on Amazon Web Services. It automates the process of collecting, cataloging, and securing data from various sources directly into Amazon S3, allowing for easy integration with analytical tools like Amazon Athena and Redshift. Lake Formation enhances data management by providing tools for data cleaning, transformation, and adding metadata for easier classification and access.

One of the key benefits of Lake Formation is its robust security model, which simplifies access controls down to the column level within your data, ensuring that only authorized users can access sensitive information. This service is particularly useful for organizations looking to analyze large

amounts of data from diverse sources, securely share data across departments, or leverage structured data for machine learning projects, making it an essential tool for efficient and secure data management in the cloud.

Fastest way to load Data from S3 to Redshift

Amazon Redshift is an enterprise-level, petabyte scale, fully managed data warehousing service. Amazon Redshift achieves efficient storage and optimum query performance through a combination of massively parallel processing, columnar data storage, and very efficient, targeted data compression encoding schemes.

The COPY command loads data in parallel from Amazon S3, Amazon EMR, Amazon DynamoDB, or multiple data sources on remote hosts. COPY loads large amounts of data much more efficiently than using INSERT statements, and stores the data more effectively as well.

AWS DataSync

AWS DataSync is a secure, online service that automates and accelerates moving data between on premises and AWS Storage services. DataSync can copy data between Network File System (NFS) shares, Server Message Block (SMB) shares, Hadoop Distributed File Systems (HDFS), self-managed object storage, AWS Snowcone, Amazon Simple Storage Service (Amazon S3) buckets, Amazon Elastic File System (Amazon EFS) file systems, Amazon FSx for Windows File Server file systems, Amazon FSx for Lustre file systems, Amazon FSz for OpenZFS file systems, and Amazon FSx for NetApp ONTAP file systems.

A social media company is experiencing poor accuracy while training on SageMaker's default built-in image classification algorithm. The Deep Learning team at the company wants to use an Inception neural network architecture rather than a ResNet one.

Which of the following options represent the most effective way to do this? (Select two)

The Amazon SageMaker image classification algorithm is a supervised learning algorithm that supports multi-label classification. It takes an image as input and outputs one or more labels assigned to that image. It uses a convolutional neural network (ResNet) that can be trained from scratch or trained using transfer learning when a large number of training images are not available.

If you need a different combination of the pre-trained network and the image data on which it has been trained, Amazon SageMaker also supports transfer learning for image classification through the built-in image classification algorithm.

In Transfer Learning, when you build a new model to classify your original dataset, you reuse the feature extraction part and re-train the classification part with your dataset. Since you don't have to train the feature extraction part (which is the most complex part of the model), you can train the model with less computational resources and training time.

For the given use case, you can take an inception v3 network pre-trained on an ImageNet dataset and bundle your own container and import it into Amazon Elastic Container Registry (Amazon ECR). You can also customize the container provided by Amazon SageMaker with our own transfer learning code in the TensorFlow framework. Then you can import this container into Amazon ECR, and use it for model training and inferencing.

Exam Alert:

If you plan to use GPU devices, make sure that your containers are nvidia-docker compatible. Only the CUDA toolkit should be included on containers. Don't bundle NVIDIA drivers with the docker image.

Exploratory Data Analysis

Probability mass function vs Probability density function

Understanding the difference between the Probability Density Function (PDF) and the Probability Mass Function (PMF) is fundamental in statistics, particularly when dealing with different types of data—continuous and discrete, respectively.

Probability Mass Function (PMF)

Definition: The Probability Mass Function applies to discrete random variables. This means the variable can take on a countable number of distinct values. The PMF gives the probability that a discrete random variable is exactly equal to some value.

Key Points:

- The PMF is defined at discrete points and sums to 1.
- It can be represented as a table, graph, or formula.

Probability Density Function (PDF)

Definition: The Probability Density Function is used with continuous random variables. These are variables that can take on any value within a continuous range. The PDF, when integrated over a range of values, gives the probability that the random variable falls within that range.

Key Points:

- The PDF itself is not a probability. Instead, the area under the curve of the PDF over an interval gives the probability.
- The total area under the curve of a PDF equals 1.

Distributions

In data science, understanding different types of distributions is crucial because they help explain how data is spread or varies, which in turn influences decision-making. Here are some of the common distributions you'll encounter, explained in simple business terms along with relatable examples:

1. Uniform Distribution

- **What It Is:** In a uniform distribution, every value has equal probability of occurring. The distribution is flat because no single value is more likely than others within a defined range.
- **Business Example:** Imagine a bakery that makes an equal number of five different types of pastries every day. The probability of selling any type of pastry is the same if the preference is uniformly distributed among customers.

2. Normal Distribution (Gaussian Distribution)

- **What It Is:** This is one of the most important distributions in statistics, characterized by its bell-shaped curve, symmetrical about the mean. Most of the data points cluster around a central point and it has predictable proportions within range intervals.
- **Business Example:** Consider the heights of adult men in a large city. The heights tend to cluster around the average (mean) height, with fewer men being extremely tall or extremely short.

3. Binomial Distribution

- **What It Is:** The binomial distribution represents the number of successes in a fixed number of independent experiments or trials, with only two possible outcomes (commonly referred to as a "success" and a "failure").
- **Business Example:** If a call center agent makes 50 calls a day and the probability of making a sale per call is 10%, the number of sales made per day by the agent follows a binomial distribution.

4. Poisson Distribution

- **What It Is:** Poisson distribution measures the probability of a given number of events happening in a fixed interval of time or space when these events happen with a known constant mean rate and independently of the time since the last event.
- **Business Example:** A car dealership sells an average of 3 cars on Saturdays. The number of cars sold can be modeled by a Poisson distribution to understand the probability of selling exactly 0, 1, 2, or more cars.

5. Exponential Distribution

- **What It Is:** The exponential distribution describes the time between events in a Poisson process, where events occur continuously and independently at a constant average rate.
- **Business Example:** The amount of time until the next customer enters a store can be modeled with an exponential distribution if customers come in at a steady average rate throughout the day.

6. Geometric Distribution

- **What It Is:** This distribution models the number of trials needed to achieve the first success in repeated experiments with two possible outcomes (success or failure), where each trial is independent of the others.
- **Business Example:** If a telemarketer has a 5% success rate of making a sale per call, the geometric distribution can model the probability of how many calls will be needed to make the first sale.

7. Log-Normal Distribution

- **What It Is:** A log-normal distribution occurs when the logarithm of a variable is normally distributed. It is used to describe distribution shapes where data is positively skewed, commonly where the size of the data grows exponentially.
- **Business Example:** The distribution of incomes within a population often fits a log-normal distribution, as incomes cannot be negative and high-value incomes stretch out the tail of the distribution.

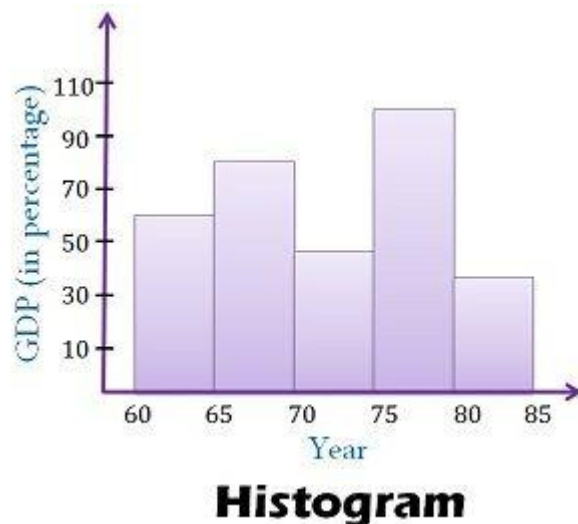
Understanding these distributions allows data scientists and business analysts to model real-world phenomena accurately, predict future outcomes, and make informed decisions based on the likely behavior of various business metrics and events.

Different Types of Plots to Remember

The main differences between a histogram and a bar chart are:

1. **Purpose:**
 - **Histogram:** Displays the distribution of numerical data by showing the frequency of data within certain ranges (bins).
 - **Bar Chart:** Compares different categories of data by displaying individual bars for each category.
2. **Data Type:**
 - **Histogram:** Used for continuous data.
 - **Bar Chart:** Used for categorical data.
3. **Bars:**
 - **Histogram:** Bars touch each other to indicate continuous data.
 - **Bar Chart:** Bars are separated to indicate discrete categories.

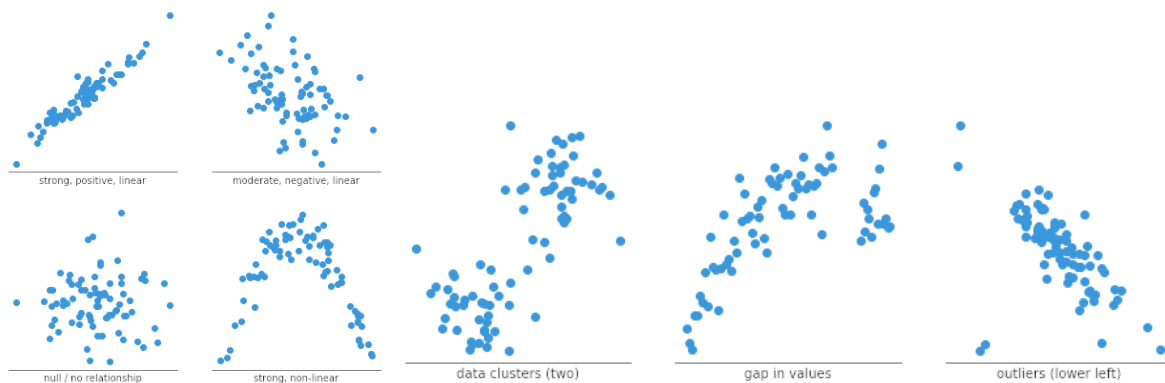
These visual distinctions help in understanding and interpreting the nature of the data being presented.



Scatterplots

are used for:

1. **Identifying Relationships:** They show the relationship between two continuous variables, helping identify patterns, correlations, or trends.
2. **Detecting Outliers:** Scatterplots can reveal outliers or anomalies in the data.
3. **Visualizing Distribution:** They help in visualizing the distribution and spread of data points.
4. **Assessing Model Fit:** In regression analysis, scatterplots help assess the fit of a model by plotting observed vs. predicted values.
5. **Comparing Variables:** Multiple scatterplots can compare relationships across different pairs of variables.

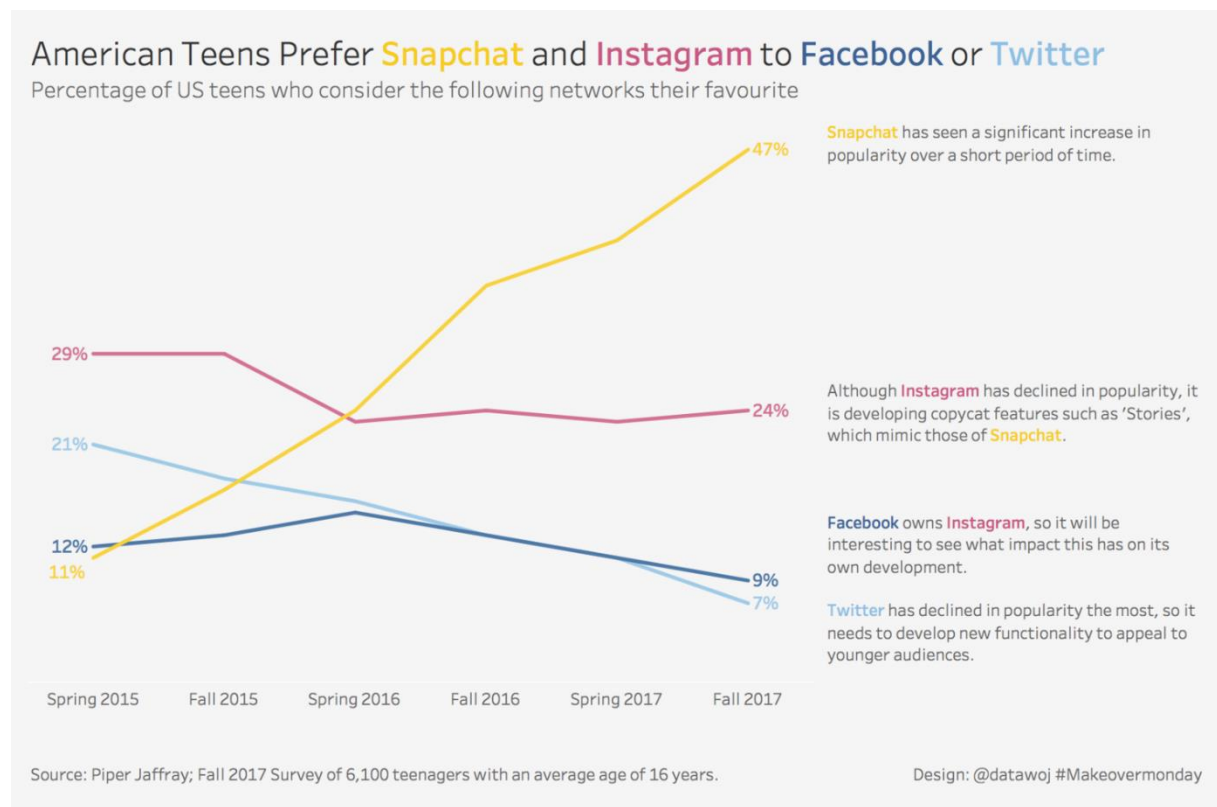


Line charts

can be used for:

1. **Trend Analysis:** Show trends over time or other continuous intervals.
2. **Comparing Data:** Compare multiple data sets on the same chart.
3. **Forecasting:** Predict future values by extrapolating trends.
4. **Identifying Patterns:** Detect patterns, cycles, and seasonal effects in data.
5. **Performance Tracking:** Monitor changes in performance metrics over time.

Line charts are particularly useful for visualizing how variables change continuously and for highlighting overall trends.



One Hot Encoding and Categorical Encoding

Categorical data should be converted into numerical forms to be used in machine learning algorithms. This conversion is usually known as encoding.

One hot encoding is a process by which categorical variables are converted into numerical values that could be provided to ML algorithms to do a better job in prediction. These numerical values are binary digits composed of 0 and 1. It creates a number (n) of numerical columns, where n is the number of unique values in a selected categorical variable.

For example, consider a column named `shirt_size`. Shirts are available in small, medium, large, or extra-large. In this scenario, there are four distinct values for `shirt_size`.

Therefore, `ONE_HOT_ENCODING` generates four new columns. Each new column is named `shirt_size_x`, where x represents a distinct `shirt_size` value.

Categorical data is usually composed of a finite number of categories, where each category is represented with a string. For example, if you have a table of customer data, a column that indicates the country a person lives in is categorical. The categories would be Afghanistan, Albania, Algeria, and so on. Categorical data can be *nominal* or *ordinal*. Ordinal categories have an inherent order, and nominal categories do not. The highest degree obtained (High school, Bachelors, Master) is an example of ordinal categories.

Encoding categorical data is the process of creating a numerical representation for categories. For example, if your categories are Dog and Cat, you may encode this information into two vectors, `[1,0]` to represent dog, and `[0,1]` to represent cat.

When you encode ordinal categories, you may need to translate the natural order of categories into your encoding. For example, you can represent highest degree obtained with the following map: `{"High school": 1, "Bachelors": 2, "Masters":3}`.

Use categorical encoding to encode categorical data that is in string format into arrays of integers.

Missing Values Imputation

In Statistics, imputation is the process of replacing missing data with substituted values. This concept is also applicable to Machine Learning.

A common issue in time-series forecasting data is the presence of missing values. Your data might contain missing values for a number of reasons, including measurement failures, formatting problems, human errors, or a lack of information to record. For instance, if you're forecasting product demand for a retail store and an item is sold out or unavailable, there would be no sales data to record while that item is out of stock. If prevalent enough, missing values can significantly impact a model's accuracy.

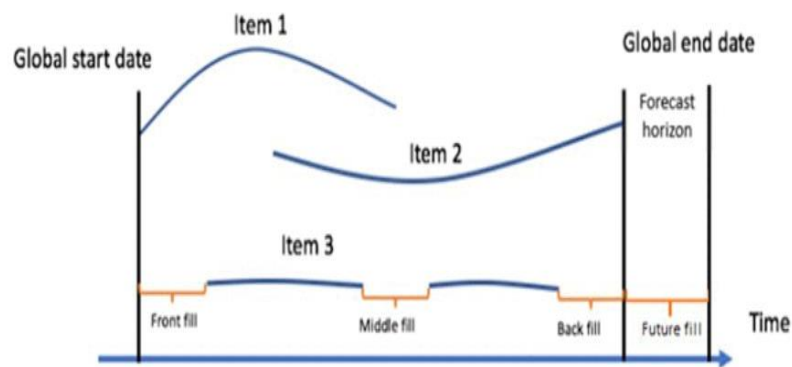
Amazon Forecast provides several filling methods to help handle missing values in your datasets. During backtesting, Forecast assumes the filled values (barring NaNs) to be true values and uses them in evaluating metrics. Forecast supports the following filling methods:

Middle filling – Fills any missing values between the item start and item end date of a data set.

Back filling – Fills any missing values between the last recorded data point and the global end date of a dataset.

Future filling (related time series only) – Fills any missing values between the global end date and the end of the forecast horizon.

The following image provides a visual representation of different filling methods.



A missing value in a time series simply denotes that the corresponding true value isn't available for further processing for a plethora of reasons. For a time series representing product sales, a missing value can mean it's not available for sale; for example, the periods outside the existence of the product (before its launch, after its deprecation) or when it wasn't available within its period of existence (partially out of stock). A missing value can also mean you don't have sales data recorded for this time range.

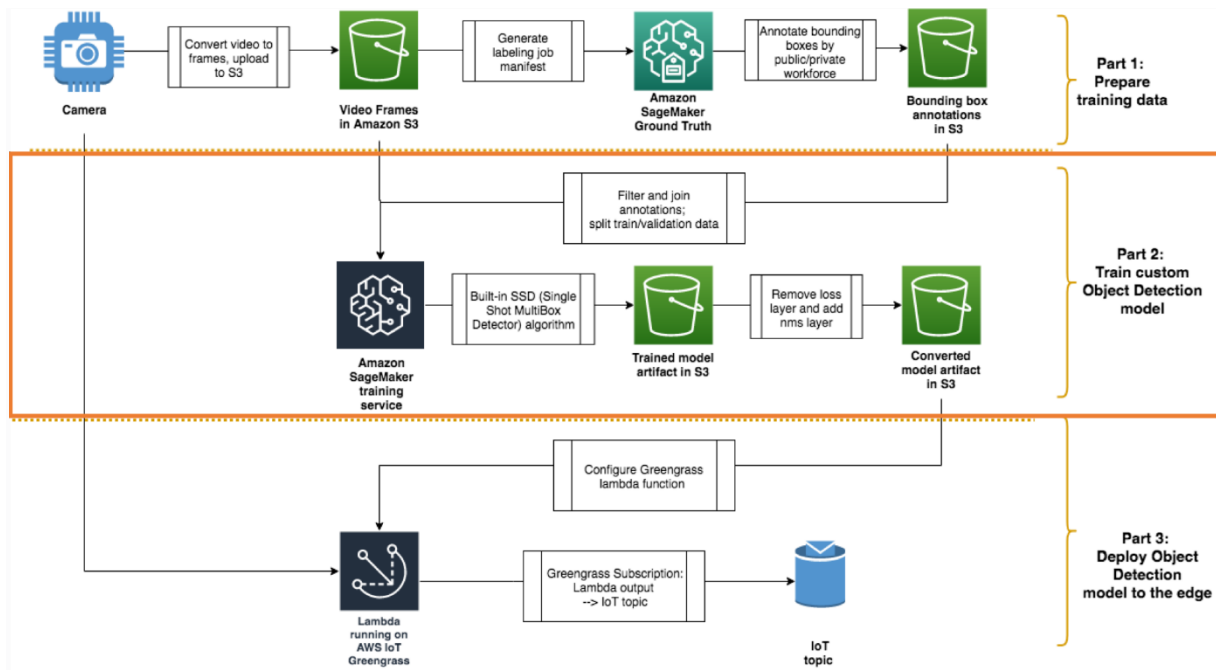
The **Multiple Imputations by Chained Equations (MICE)** algorithm is a robust, informative method of dealing with missing data in your datasets. This procedure imputes or 'fills in' the missing data in a dataset through an iterative series of predictive models. Each specified variable in the dataset is imputed in each iteration using the other variables in the dataset. These iterations will be run continuously until convergence has been met. In General, MICE is a better imputation method than naive approaches (filling missing values with 0, dropping columns).

Data Augmentation

The performance of deep learning neural networks often improves with the amount of data available.

Data augmentation is a technique to artificially create new training data from existing training data. This is done by applying domain-specific techniques to examples from the training data that create new and different training examples.

Image data augmentation is perhaps the most well-known type of data augmentation and involves creating transformed versions of images in the training dataset that belong to the same class as the original image.



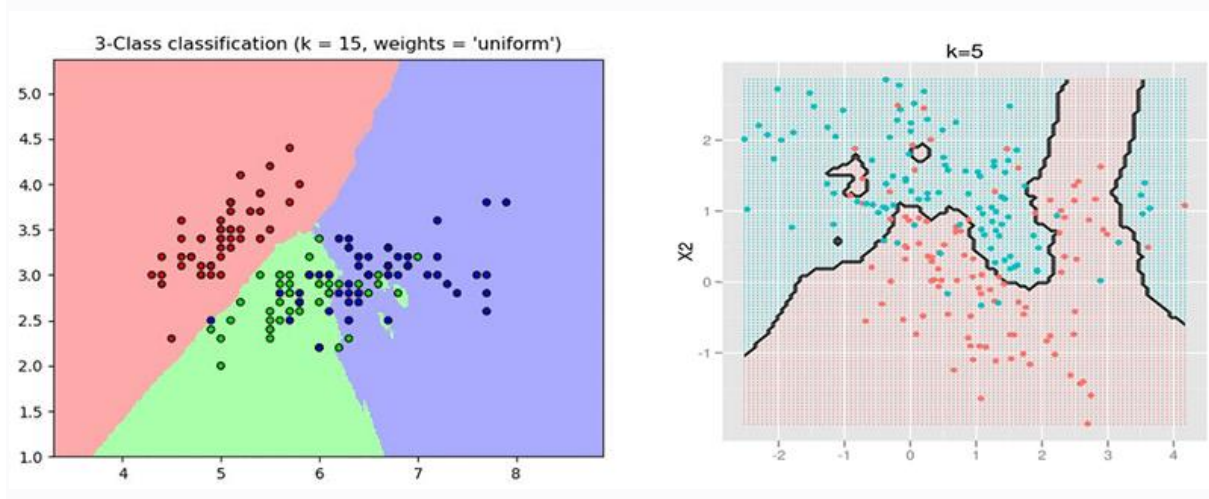
Training deep learning neural network models on more data can result in more skillful models, and the augmentation techniques can create variations of the images that can improve the ability of the fit models to generalize what they have learned to new images.

The **Amazon SageMaker image classification algorithm** is a supervised learning algorithm that supports multi-label classification. It takes an image as input and outputs one or more labels assigned to that image. It uses a convolutional neural network (ResNet) that can be trained from scratch or trained using transfer learning when a large number of training images are not available. The recommended input format for the Amazon SageMaker image classification algorithms is Apache MXNet RecordIO. However, you can also use raw images in .jpg or .png format.

You can also use a data augmentation type (augmentation_type) hyperparameter to configure your input images to be augmented in multiple ways. You can randomly crop the image and flip the image horizontally and alter the color using the Hue-Saturation-Lightness channels.

Normalization for K-NN

Normalization of numeric variables can help the learning process if there are very large range differences between numeric variables because variables with the highest magnitude could dominate the ML model, no matter if the feature is informative with respect to the target or not.



Most k-NN models use the Euclidean distance to measure how similar the target data is to a specific class. Since the Euclidean distance is a function of data points in a graph, the position of our data points greatly influences the resulting Euclidean distance. Consider a dataset with a feature called 'age' that ranges between 18-35 and a 'product price' that ranges between \$50 – \$5,000. Since the product price has a significantly larger value than the age, the model will treat the product price with “more importance”. This would have a negative impact on the model’s ability to classify data correctly.

Correlation coefficients

Correlation coefficients are statistical measures that quantify the degree to which two variables are related to each other. In machine learning, understanding the correlation between different features can help in feature selection, providing insights into which variables are important, and how they interact with each other. Here are some commonly used correlation coefficients, when to use them, and a business use case for each:

1. Pearson Correlation Coefficient

- **What It Is:** The Pearson correlation coefficient measures the linear relationship between two continuous variables. It ranges from -1 to +1, where +1 indicates a perfect positive linear relationship, -1 indicates a perfect negative linear relationship, and 0 indicates no linear relationship.
- **When to Use:** Use the Pearson correlation when you need to measure the strength of association between variables that are both continuous and approximately normally distributed.
- **Business Use Case:** In a real estate business, the Pearson correlation could be used to determine the relationship between the size of homes (in square feet) and their selling prices. This could help in pricing strategies or in identifying the most influential features for home valuation.

2. Spearman's Rank Correlation Coefficient

- **What It Is:** Spearman's Rank correlation measures the monotonic relationship between two variables based on the rank values instead of the raw data. Like Pearson, it ranges from -1 to +1.
- **When to Use:** It is best used when the data does not meet the assumptions necessary for Pearson's correlation, particularly when dealing with ordinal variables or when the data is not normally distributed.
- **Business Use Case:** In human resources, Spearman's correlation could be useful to analyze the relationship between the rank of an employee's performance score and their satisfaction level. This helps to understand if higher performance correlates with higher satisfaction irrespective of the actual score values.

Visualizations to help understand model evaluations

1. Confusion Matrix

Scenario: Evaluating classification model performance.

- **Purpose:** A confusion matrix visually displays the count of correct and incorrect predictions compared to the actual outcomes. It breaks down predictions into true positives, true negatives, false positives, and false negatives.

- **Example:** In a binary classification model predicting whether an email is spam or not, a confusion matrix helps you see how many spam emails were correctly identified, how many were missed, and how many non-spam emails were incorrectly labeled as spam.

2. ROC Curve and AUC

Scenario: Comparing classifier performance.

Purpose: The Receiver Operating Characteristic (ROC) curve plots the true positive rate against the false positive rate at various threshold settings. The Area Under the Curve (AUC) provides a single metric to compare different classifiers.

Example: In a medical diagnosis tool, the ROC curve can help in determining the best threshold for maximizing the true positive rate (correctly identifying a disease) while minimizing the false positive rate (incorrectly diagnosing a disease in healthy patients).

AWS recommends an industry-standard accuracy metric for binary classification models called Area Under the (Receiver Operating Characteristic - ROC) Curve (AUC). AUC measures the ability of the model to predict a higher score for positive examples as compared to negative examples. Because it is independent of the score cut-off, you can get a sense of the prediction accuracy of your model from the AUC metric without picking a threshold.

AUC/ROC is the correct choice. AUC/ROC metric does not require you to set a classification threshold.

Measuring ML Model Accuracy

Amazon ML provides an industry-standard accuracy metric for binary classification models called Area Under the (Receiver Operating Characteristic) Curve (AUC). AUC measures the ability of the model to predict a higher score for positive examples as compared to negative examples. Because it is independent of the score cut-off, you can get a sense of the prediction accuracy of your model from the AUC metric without picking a threshold.

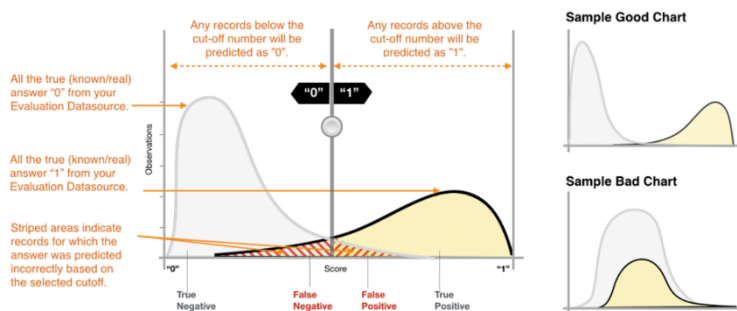
The AUC metric returns a decimal value from 0 to 1. AUC values near 1 indicate an ML model that is highly accurate. Values near 0.5 indicate an ML model that is no better than guessing at random. Values near 0 are unusual to see, and typically indicate a problem with the data. Essentially, an AUC near 0 says that the ML model has learned the correct patterns, but is using them to make predictions that are flipped from reality (0's are predicted as 1's and vice versa). For more information about AUC, go to the [Receiver operating characteristic](#) page on Wikipedia.

The baseline AUC metric for a binary model is 0.5. It is the value for a hypothetical ML model that randomly predicts a 1 or 0 answer. Your binary ML model should perform better than this value to begin to be valuable.

Using the Performance Visualization

To explore the accuracy of the ML model, you can review the graphs on the **Evaluation** page on the Amazon ML console. This page shows you two histograms: a) a histogram of the scores for the actual positives (the target is 1) and b) a histogram of scores for the actual negatives (the target is 0) in the evaluation data.

An ML model that has good predictive accuracy will predict higher scores to the actual 1s and lower scores to the actual 0s. A perfect model will have the two histograms at two different ends of the x-axis showing that actual positives all got high scores and actual negatives all got low scores. However, ML models make mistakes, and a typical graph will show that the two histograms overlap at certain scores. An extremely poor performing model will be unable to distinguish between the positive and negative classes, and both classes will have mostly overlapping histograms.



Using the visualizations, you can identify the number of predictions that fall into the two types of correct predictions and the two types of incorrect predictions.

Correct Predictions

- True positive (TP): Amazon ML predicted the value as 1, and the true value is 1.
- True negative (TN): Amazon ML predicted the value as 0, and the true value is 0.

Erroneous Predictions

- False positive (FP): Amazon ML predicted the value as 1, but the true value is 0.
- False negative (FN): Amazon ML predicted the value as 0, but the true value is 1.

For imbalanced datasets, you are better off using another metric called - **PR AUC** - that is also used in production systems for a highly imbalanced dataset, where the fraction of positive class is small, such as in case of credit card fraud detection.

3. Precision-Recall Curve

Scenario: Evaluating classifiers in situations with significant class imbalance.

- **Purpose:** This curve plots precision (the accuracy of positive predictions) and recall (the ability to find all positive instances) against each other at various threshold levels.
- **Example:** For a fraud detection model where fraudulent transactions are rare, the precision-recall curve is useful for understanding the trade-off between catching as many frauds as possible (recall) and making sure the flagged transactions are truly fraudulent (precision).

4. Residual Plots

Scenario: Diagnosing regression models.

- **Purpose:** Residual plots display the difference between observed and predicted values. Ideally, residuals should be randomly distributed and show no pattern.
- **Example:** In a house price prediction model, a residual plot can show if there are any systematic errors in predictions, such as underestimating prices for high-value houses or overestimating for low-value houses.

5. Feature Importance Plot

Scenario: Understanding which features are driving the predictions in tree-based models.

- **Purpose:** This plot ranks the features based on their importance in making accurate predictions.
- **Example:** In a credit scoring model, a feature importance plot can reveal which factors (e.g., income level, credit history, debt-to-income ratio) most significantly impact a person's credit score according to the model.

6. Learning Curves

Scenario: Diagnosing issues with learning in model training.

- **Purpose:** Learning curves plot training and validation error as a function of the number of training instances. They help in identifying problems like overfitting or underfitting.
- **Example:** If a model's training error decreases (improves) with more data but the validation error does not decrease, this could indicate overfitting. Conversely, if both training and validation errors remain high, the model may be underfitting.

7. Heat Maps

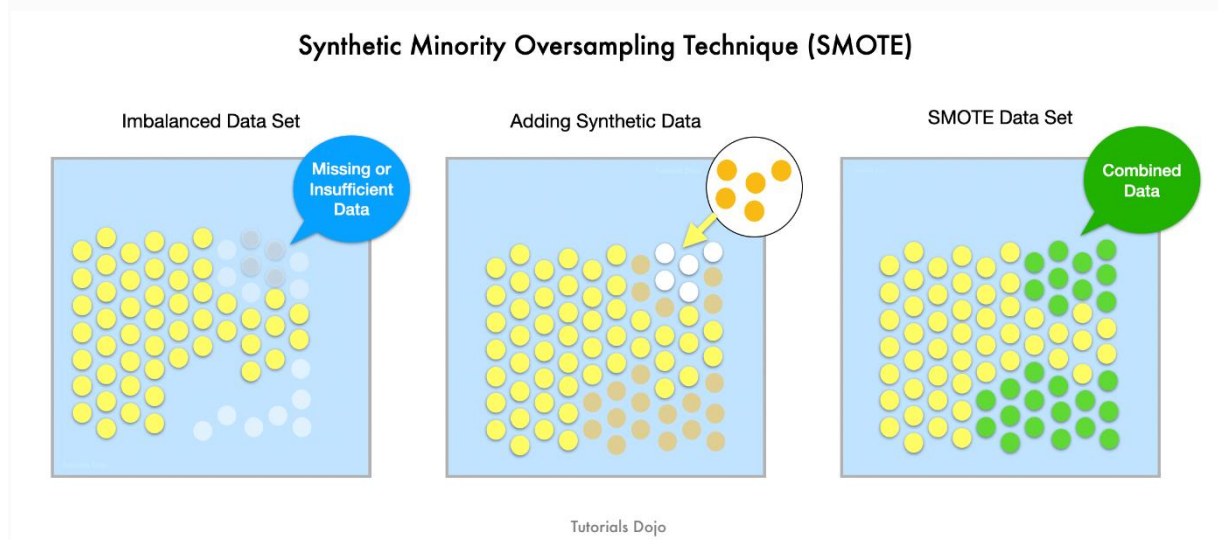
Scenario: Visualizing complex relationships in data or between predicted and actual values.

- **Purpose:** Heat maps can be used to show the density or intensity of data, revealing patterns that might be missed in other types of plots.
- **Example:** In a clustering model, a heat map can help visualize the intensity of data points across different clusters or categories.

These visualizations are tools to help data scientists and analysts understand and improve their models by providing a visual interpretation of the model's performance and any issues that might need attention. Choosing the right type of visualization depends on the specific questions you want to answer about your model and the type of model you are evaluating.

Oversampling

The **Synthetic Minority Oversampling Technique (SMOTE)** is an oversampling approach in which the minority class is over-sampled by creating “synthetic” examples rather than by over-sampling with replacement.



The minority class is oversampled by taking each minority class sample and introducing synthetic examples along the line segments joining any/all of the minority class nearest neighbors. Depending upon the amount of over-sampling required, neighbors from the k nearest neighbors are randomly chosen.

ML Algorithms

Sagemaker Folder Structure

When SageMaker trains a model, it creates the following file folder structure in the container's `/opt/ml` directory:

```
/opt/ml
├── input
│   ├── config
│   │   ├── hyperparameters.json
│   │   └── resourceConfig.json
│   ├── data
│   ├── <channel_name>
│   └── <input data>
├── model
├── code
├── output
└── failure
```

The SageMaker containers library places the scripts that the container will run in the `/opt/ml/code/` directory. Your script should write the model generated by your algorithm to the `/opt/ml/model/` directory.

After copying the script to the location inside the container that is expected by SageMaker, you must define it as the script entry point in the `SAGEMAKER_PROGRAM` environment variable. When training starts, the interpreter executes the entry point defined by `SAGEMAKER_PROGRAM`.

Regularization

A regression model that uses L1 regularization technique is called **Lasso Regression** and model which uses L2 is called **Ridge Regression**.

The key difference between these two is the penalty term.

Ridge regression adds “*squared magnitude*” of coefficient as penalty term to the loss function. Here the *highlighted* part represents L2 regularization element.

$$\sum_{i=1}^n (y_i - \sum_{j=1}^p x_{ij} \beta_j)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

Cost function

Here, if *lambda* is zero then you can imagine we get back OLS. However, if *lambda* is very large then it will add too much weight and it will lead to under-fitting. Having said that it's important how *lambda* is chosen. This technique works very well to avoid over-fitting issue.

Lasso Regression (Least Absolute Shrinkage and Selection Operator) adds “*absolute value of magnitude*” of coefficient as penalty term to the loss function.

$$\sum_{i=1}^n (Y_i - \sum_{j=1}^p X_{ij} \beta_j)^2 + \lambda \sum_{j=1}^p |\beta_j|$$

Cost function

Again, if *lambda* is zero then we will get back OLS whereas very large value will make coefficients zero hence it will under-fit.

The **key difference** between these techniques is that Lasso shrinks the less important feature's coefficient to zero thus, removing some feature altogether. So, this works well for **feature selection** in case we have a huge number of features.

Traditional methods like cross-validation, stepwise regression to handle overfitting and perform feature selection work well with a small set of features but these techniques are a great alternative when we are dealing with a large set of features.

Linear Learner

Linear models are supervised learning algorithms used for solving either classification or regression problems. For input, you give the model labeled examples (x, y). x is a high-dimensional vector and y is a numeric label. For binary classification problems, the label must be either 0 or 1. For multiclass classification problems, the labels must be from 0 to `num_classes - 1`. For regression problems, y is a real number. The algorithm learns a linear function, or, for classification problems, a linear threshold function, and maps a vector x to an approximation of the label y.

Hyperparameters

At a minimum, Amazon SageMaker linear learner requires you to specify input and output data locations, and objective type (classification or regression) as arguments. The feature dimension is also required.

If you're using [Managed Spot Training](#), the linear learner algorithm supports using [checkpoints to take a snapshot of the state of the model](#).

The Amazon SageMaker linear learner algorithm supports three data channels: train, validation (optional), and test (optional). If you provide validation data, the `S3DataDistributionType` should be `FullyReplicated`. The algorithm logs validation loss at every epoch, and uses a sample of the validation data to calibrate and select the best model. If you don't provide validation data, the algorithm uses a sample of the training data to calibrate and select the model. If you provide test data, the algorithm logs include the test score for the final model.

For training, the linear learner algorithm supports both recordIO-wrapped protobuf and CSV formats. For the `application/x-recordio-protobuf` input type, only Float32 tensors are supported. For the `text/csv` input type, the first column is assumed to be the label, which is the target variable for prediction. You can use either File mode or Pipe mode to train linear learner models on data that is formatted as recordIO-wrapped-protobuf or as CSV.

For inference, the linear learner algorithm supports the `application/json`, `application/x-recordio-protobuf`, and `text/csv` formats. When you make predictions on new data, the format of the response depends on the type of model. **For regression** (`predictor_type='regressor'`), the score is the prediction produced by the model. **For classification** (`predictor_type='binary_classifier'` or `predictor_type='multiclass_classifier'`), the model returns a score and also a `predicted_label`. The `predicted_label` is the class predicted by the model and the score measures the strength of that prediction.

- **For binary classification**, `predicted_label` is 0 or 1, and score is a single floating point number that indicates how strongly the algorithm believes that the label should be 1.
- **For multiclass classification**, the `predicted_class` will be an integer from 0 to `num_classes-1`, and score will be a list of one floating point number per class.

To interpret the score in classification problems, you have to consider the loss function used. If the loss hyperparameter value is `logistic` for binary classification or `softmax_loss` for multiclass classification, then the score can be interpreted as the probability of the corresponding class. These are the loss values used by the linear learner when the loss value is auto default value. But if the loss is set to `hinge_loss`, then the score cannot be interpreted as a probability. This is because hinge loss corresponds to a Support Vector Classifier, which does not produce probability estimates

The following are the available output formats for the SageMaker linear learner algorithm.

Binary Classification

```
let response = {  
  "predictions": [  
    {  
      "score": 0.4,  
      "predicted_label": 0
```

```
}  
]  
}
```

Multiclass Classification

```
let response = {  
  "predictions": [  
    {  
      "score": [0.1, 0.2, 0.4, 0.3],  
      "predicted_label": 2  
    }  
  ]  
}
```

Regression

```
let response = {  
  "predictions": [  
    {  
      "score": 0.4  
    }  
  ]  
}
```

DeepAR

The Amazon SageMaker DeepAR forecasting algorithm is a supervised learning algorithm for forecasting scalar (one-dimensional) time series using recurrent neural networks (RNN). Classical forecasting methods, such as autoregressive integrated moving average (ARIMA) or exponential smoothing (ETS), fit a single model to each individual time series. They then use that model to extrapolate the time series into the future.

In many applications, however, you have many similar time series across a set of cross-sectional units. For example, you might have time series groupings for demand for different products, server loads, and requests for webpages. For this type of application, you can benefit from training a single model jointly over all of the time series. DeepAR takes this approach. When your dataset contains hundreds of related time series, DeepAR outperforms the standard ARIMA and ETS methods. *You can also use*

the trained model to generate forecasts for new time series that are similar to the ones it has been trained on.

Based on historical data for a behavior, you can predict future behavior using DeepAR algorithm. For example, you can predict sales on a new product based on previous sales data. SageMaker DeepAR algorithm specializes in forecasting new product performance.

AWS Ground truth

[Amazon SageMaker Ground Truth](#) helps you quickly build highly accurate training datasets for machine learning (ML). To get your data labeled, you can use your own workers, a choice of vendor-managed workforces that specialize in data labeling, or a public workforce powered by Amazon Mechanical Turk.

The public workforce is large and economical but as with any set of diverse workers, can be prone to errors. One way to produce a high quality label from these lower quality annotations is to systematically combine the responses from different workers for the same item into a single label. Amazon SageMaker Ground Truth has built-in annotation consolidation algorithms that perform this aggregation so that you can get high accuracy labels as a result of a labeling job.

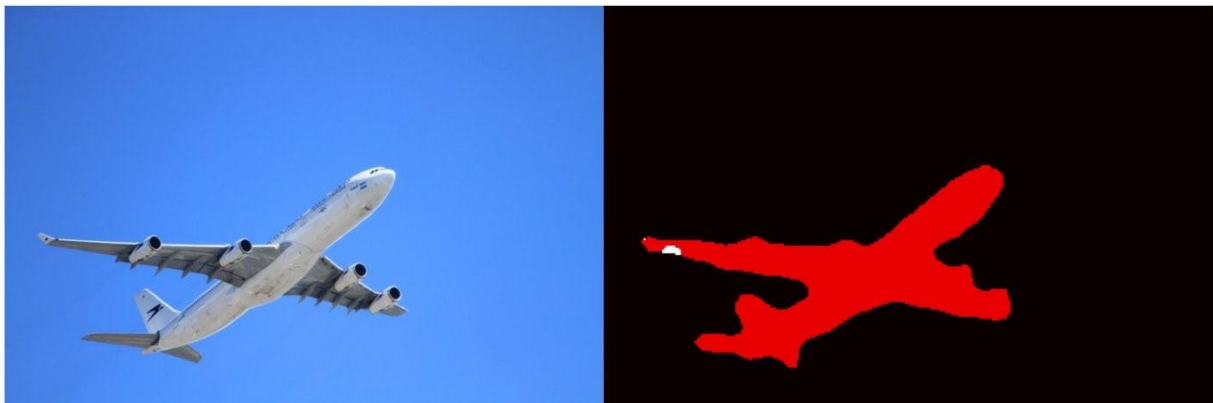
This blog post focuses on the consolidation algorithm for the case of classification (e.g. labeling an image as that of an “owl,” “falcon,” or “parrot”), and shows its benefit over two competing baseline approaches of single responses and majority voting.

- **Option A is incorrect.** Ground Truth chaining labelling jobs allows you to reuse datasets from previous labelling jobs. This feature would not help you address mislabeled images or videos.
 - **Option B is incorrect.** The Ground Truth label verification and adjustment feature allows you to have workers verify and correct mislabeled labels. This would help you correct mislabeled items, but it is not an automated process. It is manual.
 - **Option C is incorrect.** The Ground Truth batches for labelling tasks feature is used to send objects to your workers in batches. This would not help you correct mislabeled objects.
 - **Option D is correct.** The Ground Truth annotation consolidation feature allows you to combine the annotations of multiple workers to produce an automated probabilistic estimate of what the correct label should be.
-
- **Option A is correct.** The bounding box finds the most similar bounding boxes from workers and averages them, thus using the power of multiple workers to annotate your images more accurately.
 - **Option B is correct.** The semantic segmentation feature fuses the pixel annotations of multiple workers and applying a smoothing function to the image, thus using the power of multiple workers to annotate your images more accurately.
 - **Option C is incorrect.** The named entity feature is used with text annotation work, not image annotation.
 - **Option D is incorrect.** The Ground Truth output manifest allows the output of a labelling job to be used as the input to a machine learning model. This feature will not help ensure the accuracy of worker annotations.
 - **Option E is incorrect.** The Ground Truth Mechanical Turk feature gives you access to a large pool of labelling workers. While increasing the number of workers at your disposal, this feature will not help ensure the accuracy of worker annotations.

References

Semantic Segmentation

The semantic segmentation algorithm provides a fine-grained, pixel-level approach to developing computer vision applications. It tags every pixel in an image with a class label from a predefined set of classes. Tagging is fundamental for understanding scenes, which is critical to an increasing number of computer vision applications such as self-driving vehicles, medical imaging diagnostics, and robot sensing.



Because the semantic segmentation algorithm classifies every pixel in an image, it also provides information about the shapes of the objects contained in the image. The segmentation output is represented as a grayscale image called a *segmentation mask*. A segmentation mask is a grayscale image with the same shape as the input image.

LDA

The **Amazon SageMaker Latent Dirichlet Allocation (LDA) algorithm** is an unsupervised learning algorithm that attempts to describe a set of observations as a mixture of distinct categories. LDA is most commonly used to discover a user-specified number of topics shared by documents within a text corpus. Here each observation is a document, the features are the presence (or occurrence count) of each word, and the categories are the topics. Since the method is unsupervised, the topics are not specified up front and are not guaranteed to align with how a human may naturally categorize documents. The topics are learned as a probability distribution over the words that occur in each document. Each document, in turn, is described as a mixture of topics.

Feedforward Neural Networks

What It Is: A feedforward neural network is the simplest type of artificial neural network. In this architecture, the information moves in only one direction—forward—from the input nodes, through the hidden nodes (if any), and to the output nodes. There are no cycles or loops in the network; hence, the data flows one way.

How It Works: Feedforward neural networks process input data through one or more layers of nodes, each of which applies a set of weights to the inputs, adds a bias, and passes the result through an activation function. The output of each layer becomes the input for the next layer until the final output is produced.

Multilayer Perceptron (MLP)

What It Is: A multilayer perceptron is a class of feedforward neural network. An MLP consists of at least three layers of nodes: an input layer, one or more hidden layers, and an output layer. Unlike other types of feedforward neural networks, MLPs use at least one hidden layer.

How It Works: The MLP utilizes a technique called backpropagation for training the network. Backpropagation is an algorithm used for minimizing the error by updating the weights iteratively. This makes MLPs suitable for complex pattern recognition tasks, where the relationship between the input data and the output is non-linear.

Lex

Amazon Lex is a service for building conversational interfaces using voice and text. Powered by the same conversational engine as Alexa, Amazon Lex provides high quality speech recognition and language understanding capabilities, enabling addition of sophisticated, natural language ‘chatbots’ to new and existing applications.

Amazon Q

Amazon Q generates code, tests, debugs, and has multistep planning and reasoning capabilities that can transform and implement new code generated from developer requests. Amazon Q also makes it easier for employees to get answers to questions across business data—such as company policies, product information, business results, code base, employees, and many other topics—by connecting to enterprise data repositories to summarize the data logically, analyze trends, and engage in dialogue about the data.

Amazon Q Business

Amazon Q Business is a generative AI-powered assistant that can answer questions, provide summaries, generate content, and securely complete tasks based on data and information in your enterprise systems. It empowers employees to be more creative, data-driven, efficient, prepared, and productive.

Amazon Q Developer

Amazon Q Developer assists developers and IT professionals with all their tasks—from coding, testing, and upgrading applications, to diagnosing errors, performing security scanning and fixes, and optimizing AWS resources. Amazon Q has advanced, multistep planning and reasoning capabilities that can transform (for example, perform Java version upgrades) and implement new features generated from developer requests.

Amazon QuickSight

Amazon Q brings its advanced generative AI technology to Amazon QuickSight, the AWS unified business intelligence (BI) service built for the cloud. With Amazon Q in QuickSight, customers get a generative BI assistant that allows business analysts to use natural language to build BI dashboards in minutes and easily create visualizations and complex calculations. Additionally, business users can get AI-driven executive summaries of dashboards, ask questions of data beyond what is presented in the dashboards, and create detailed and customizable data stories highlighting key insights, trends, and drivers.

Amazon Connect

In Amazon Connect, our contact center service, Amazon Q helps your customer service agents provide better customer service. Amazon Q in Connect uses the real-time conversation with the customer along with relevant company content to automatically recommend what to say or what actions an agent should take to better assist customers.

AWS Supply Chain

Using Amazon Q in AWS Supply Chain (coming soon), inventory managers, supply and demand planners, and others will be able to ask and get intelligent answers about what is happening in their supply chain, why it is happening, and what actions to take. They will also be able to explore what-if scenarios to understand the trade-offs between different supply chain choices.

Amazon Comprehend

Amazon Comprehend uses a pre-trained model to gather **insights** about a document or a set of documents. This model is continuously trained on a large body of text so that there is no need for you to provide training data.

You can use Amazon Comprehend to build your own **custom models** for custom classification and custom entity recognition.

Amazon Comprehend provides **topic modeling** using a built-in model. Topic modeling examines a corpus of documents and organizes the documents based on similar keywords within them.

Amazon Comprehend provides synchronous and asynchronous **document processing modes**. Use synchronous mode for processing one document or a batch of up to 25 documents. Use an asynchronous job to process a large number of documents.

Amazon Comprehend can analyze a document or set of documents to gather insights about it. Some of the insights that Amazon Comprehend develops about a document include:

- [Entities](#) – Amazon Comprehend returns a list of entities, such as people, places, and locations, identified in a document.
- [Events](#) – Amazon Comprehend detects specific types of events and related details.
- [Key phrases](#) – Amazon Comprehend extracts key phrases that appear in a document. For example, a document about a basketball game might return the names of the teams, the name of the venue, and the final score.
- [Personally identifiable information \(PII\)](#) – Amazon Comprehend analyzes documents to detect personal data that identify an individual, such as an address, bank account number, or phone number.
- [Dominant language](#) – Amazon Comprehend identifies the dominant language in a document. Amazon Comprehend can identify 100 languages.
- [Sentiment](#) – Amazon Comprehend determines the dominant sentiment of a document. Sentiment can be positive, neutral, negative, or mixed.
- [Targeted Sentiment](#) – Amazon Comprehend determines the sentiment of specific entities mentioned in a document. The sentiment of each mention can be positive, neutral, negative, or mixed.
- [Syntax analysis](#) – Amazon Comprehend parses each word in your document and determines the part of speech for the word. For example, in the sentence "It is raining today in Seattle," "it" is identified as a pronoun, "raining" is identified as a verb, and "Seattle" is identified as a proper noun.

AWS Rekognition

Amazon Rekognition Video has a feature called Pathing that can create a track of the path people take in videos and provide information such as:

The location of the person in the video frame at the time their path is tracked.

Facial landmarks such as the position of the left eye, when detected.

Amazon Transcribe

Amazon Transcribe makes it easy for developers to add speech to text capabilities to their applications. Amazon Transcribe uses a deep learning process called automatic speech recognition (ASR) to convert speech to text quickly and accurately. Amazon Transcribe can be used to transcribe customer service calls, automate subtitling, and generate metadata for media assets to create a fully searchable archive. You can use Amazon Transcribe Medical to add medical speech to text capabilities to clinical documentation applications.

Amazon Transcribe uses advanced machine learning technologies to recognize speech in audio files and transcribe them into text. You can use Amazon Transcribe to convert audio to text and to create applications that incorporate the content of audio files.

Content removal

Content removal conceals information in the resulting transcript from your source audio file. Amazon Transcribe changes items in the transcript and does not modify the source audio.

☐ Automatic content redaction [Info](#)
Automatic content redaction removes personally identifiable information (PII) in your transcripts. Redactions in transcripts show up as [PII].

☒ Vocabulary filtering [Info](#)
Vocabulary filtering can remove or mask specified words in the final transcript.

Filter selection

Choose an existing vocabulary filter collection or [create a new one](#). [↗](#)

offensive_words ▼

Vocabulary filtering method

Use a vocabulary filter to filter vocabularies from your transcript. For example, in the sentence "The quick brown fox jumps over the lazy dog," you remove the word "lazy" using the following options.

☒ Mask vocabulary
Example: The quick brown fox jumps over the *** dog.

☐ Remove vocabulary
Example: The quick brown fox jumps over the dog.

You can mask or remove words that you don't want to appear in your transcription results with **vocabulary filtering**. For example, you can use vocabulary filtering to prevent the display of offensive or profane terms. This enables you to generate family-friendly captions of a TV show or transcripts of conferences that are appropriate for your audiences. Use vocabulary filtering for any word that you consider profane, obscene, offensive, or otherwise unsuitable for the readers of your transcripts.

Amazon Polly

With Amazon Polly's custom lexicons or vocabularies, you can modify the pronunciation of particular words, such as company names, acronyms, foreign words, and neologisms (e.g., "ROTFL", "C'est la vie" when spoken in a non-French voice). To customize these pronunciations, you upload an XML file with lexical entries. For example, you can customize the pronunciation of the Filipino word: "Pilipinas" by using the phoneme element in your input XML.

Pronunciation lexicons enable you to customize the pronunciation of words. Amazon Polly provides API operations that you can use to store lexicons in an AWS region. Those lexicons are then specific to that particular region.

Amazon Polly

Text-to-Speech

Lexicons

S3 synthesis tasks

Lexicons

You can customize the pronunciation of specific words and phrases by uploading lexicon files in the PLS format. [Learn more](#)

[Upload](#) [Download](#) [Delete](#)

Filter All languages: 1/100 Selected items: 0

☐	Name	Language	Entities	Uploaded
☐	load	English, US	1	01/03/2021

Defining the acronym of BGC which stands for: Bonifacio Global City

```
<?xml version="1.0" encoding="UTF-8"?>
<lexicon version="1.0"
  xmlns="http://www.w3.org/2005/01/pronunciation-lexicon"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.w3.org/2005/01/pronunciation-lexicon
    http://www.w3.org/TR/2007/CR-pronunciation-lexicon-20071212/pls.xsd"
  alphabet="ipa"
  xml:lang="en-US">
  <lexeme>
    <grapheme>BGC</grapheme>
    <alias>Bonifacio Global City</alias>
  </lexeme>
</lexicon>
```

Uploaded custom lexicons in Amazon Polly

Tutorials Dojo

The following are examples of ways to use lexicons with speech synthesis engines:

- Common words are sometimes stylized with numbers taking the place of letters, as with “g3t sm4rt” (get smart). Humans can read these words correctly. However, a Text-to-Speech (TTS) engine reads the text literally, pronouncing the name exactly as it is spelled. This is where you can leverage lexicons to customize the synthesized speech by using Amazon Polly. In this example, you can specify an alias (get smart) for the word “g3t sm4rt” in the lexicon.

- Your text might include an acronym, such as W3C. You can use a lexicon to define an alias for the word W3C so that it is read in the full, expanded form (World Wide Web Consortium).

Lexicons give you additional control over how Amazon Polly pronounces words uncommon to the selected language. For example, you can specify the pronunciation using a phonetic alphabet.

Use a viseme Speech Mark : this feature is used to synchronize speech with facial animation (lip-syncing) or to highlight written words as they’re spoken.

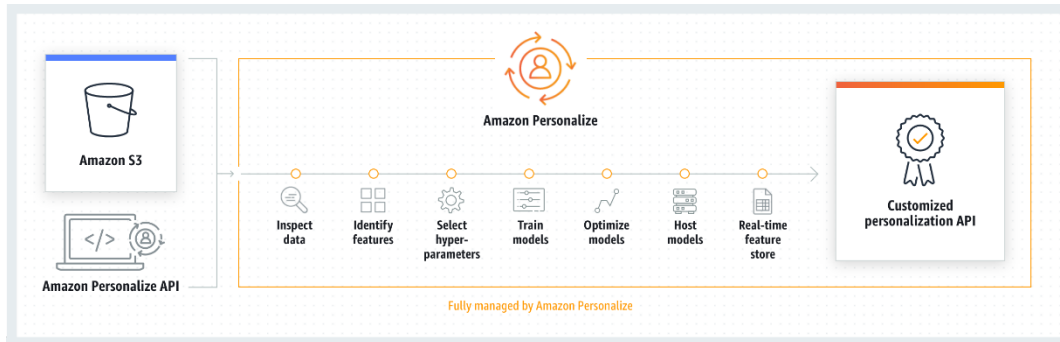
Speech Synthesis Markup Language (SSML) and use the emphasis tag to guide the pronunciation - this type of tag is simply used to emphasize words by changing the speaking rate and volume of the speech.

Amazon Personalize

Amazon Personalize is a machine learning service that makes it easy for developers to create individualized recommendations for customers using their applications.

Amazon Personalize makes it easy for developers to build applications capable of delivering a wide array of personalization experiences, including specific product recommendations, personalized product re-ranking, and customized direct marketing.

There is no need for development, training and testing of custom models when using Personalize.



Amazon Personalize Terminology

Amazon Personalize is an AWS service that makes it easy for developers to create individualized recommendations for customers using their applications. To understand how it works, let's explore two key terms used in Amazon Personalize: datasets and recipes.

Datasets

In Amazon Personalize, a **dataset** refers to the collection of data that you provide to the service. This data includes information about users, items, and interactions between users and items, which Personalize uses to train a model.

- **Users Dataset:** Contains attributes about the users, such as age or region.
- **Items Dataset:** Contains information about the items you want to recommend, like products, movies, or articles.
- **Interactions Dataset:** Records the interactions between users and items, which might include clicking, purchasing, or rating.

Recipes

A **recipe** in Amazon Personalize is a pre-defined algorithm or a set of data processing instructions and machine learning algorithms that you use to create personalized recommendations. Each recipe is suited for different types of recommendation problems, such as user personalization, similar items, or personalized ranking.

- **User-Personalization Recipe:** Provides recommendations tailored to individual users.
- **SIMS (Similar Items) Recipe:** Finds items similar to a given item, useful for "customers who viewed this item also viewed" scenarios.
- **Personalized-Ranking Recipe:** Ranks a list of items for a user, based on predicted preferences.

Example Business Scenario

Question: How can a retail company use Amazon Personalize to enhance the shopping experience for its customers?

Answer: A retail company can utilize Amazon Personalize to improve its online shopping platform by creating personalized user experiences. Here's a simple explanation of how they can implement this:

1. **Collect Data:** The company starts by collecting data that will form the datasets:
 - **Users Dataset:** Includes customer profiles with information such as age, gender, and purchasing history.
 - **Items Dataset:** Comprises details of the products like category, price, brand, and descriptions.
 - **Interactions Dataset:** Contains data on how users interact with these products, such as views, adds to cart, and purchases.
2. **Choose a Recipe:** Depending on the company's specific needs, they might choose different recipes:
 - If they want to recommend products based on a user's unique preferences, they can use the **User-Personalization Recipe**.
 - To show similar products on a product detail page, they could use the **SIMS Recipe**.
3. **Train and Deploy:** Amazon Personalize will use the data in the datasets to train a model based on the selected recipe. Once trained, the model can be deployed to start generating recommendations.
4. **Deliver Personalized Experiences:** When a user logs in to the retail site, the system can display personalized product recommendations tailored to their tastes and preferences. Additionally, on product pages, it can show other similar items, enhancing user engagement and increasing the likelihood of additional purchases.

Business Benefits: By implementing Amazon Personalize, the retail company can enhance customer satisfaction by providing a more tailored shopping experience. It can also lead to increased sales through higher conversion rates and improved customer retention by keeping the shopping experience relevant and engaging.

This approach demonstrates the practical application of Amazon Personalize in a retail scenario, leveraging datasets and recipes to effectively meet business objectives in enhancing customer experiences.

Amazon Personalize Hyperparameters:

Amazon Personalize allows you to customize the behavior of the machine learning models through hyperparameters. These hyperparameters help you fine-tune how the models train on your data, influencing the quality and relevance of the recommendations they generate. Below, I'll explain some of the key hyperparameters specific to various recipes in Amazon Personalize, using simple layman terms:

1. min_user_history_length_percentile and max_user_history_length_percentile

- **What They Are:** These hyperparameters are used to filter out users based on their interaction history when training the model. The "min_user_history_length_percentile" sets a lower threshold, and "max_user_history_length_percentile" sets an upper threshold.
- **Layman Explanation:** Imagine you're only interested in customers who are neither too new nor too frequent, to tailor your model's learning. These settings help you focus on customers within a specific range of activity, like those who visit your store a moderate amount of times.

2. min_item_interaction_count_percentile and max_item_interaction_count_percentile

- **What They Are:** These parameters control which items are included in the training data based on how many times users have interacted with them.
- **Layman Explanation:** It's like deciding to only stock items that are not too unpopular but also not overexposed. You choose items that have been interacted with a "just right" number of times to ensure your recommendations are relevant but also provide some variety.

3. hidden_dimension

- **What It Is:** This hyperparameter specifies the number of dimensions used to represent user and item data internally in the model.
- **Layman Explanation:** Think of this as adjusting the complexity of the model's thought process. A higher number means the model can consider more factors or nuances when learning about users and items, potentially leading to more nuanced recommendations.

4. bptt

- **What It Is:** Backpropagation through time (bptt) is a technique for training certain types of neural networks on sequences of data. In Amazon Personalize, it specifies how far back in a user's history the model should look to learn dependencies.
- **Layman Explanation:** This is akin to how far back in time you should remember someone's preferences. If they liked action movies last year, should that influence your recommendations today? This setting helps decide that temporal influence.

5. learning_rate

- **What It Is:** The rate at which the model learns from new data during training. A higher rate makes the model learn faster but can lead to overshooting, while a lower rate makes learning steadier but slower.
- **Layman Explanation:** Imagine you're teaching someone new skills. If you rush the lessons, they might miss some nuances; too slow, and it might take forever for them to learn. This rate adjusts how quickly the model adapts based on new data it receives.

Use Cases for Tuning Hyperparameters

Adjusting these hyperparameters can be crucial in various scenarios:

- **E-commerce:** Tailoring recommendations to match seasonal buying patterns by adjusting bptt and hidden_dimension to reflect short-term changes in user preferences.
- **Media Streaming:** Adapting min_user_history_length_percentile to ensure that only users with a reliable viewing history are considered, ensuring that the recommendations are based on well-established preferences.
- **Online Publishing:** Using min_item_interaction_count_percentile to avoid recommending articles that are either too niche or too viral, aiming for a balanced content strategy that keeps readers engaged.

By tweaking these hyperparameters, businesses can ensure that their recommendation systems are not only more effective but also aligned with specific business goals and user engagement strategies.

Visualizations for Model Evaluation:

Visualizations play a crucial role in evaluating and interpreting machine learning models. They provide insight into the performance, behavior, and potential issues with models that raw numbers alone might not reveal. Here are some commonly used visualizations in different scenarios when evaluating a model, along with examples of their use:

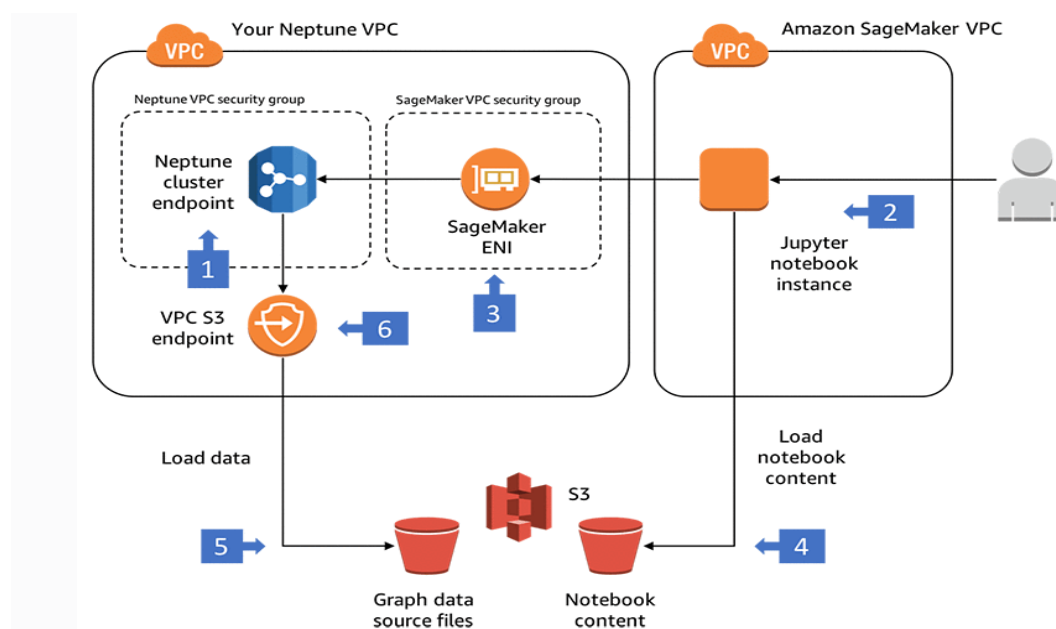
Factorization Machines

The Factorization Machines algorithm is a general-purpose supervised learning algorithm that you can use for both classification and regression tasks. It is an extension of a linear model that is designed to capture interactions between features within high dimensional sparse datasets economically. For example, in a click prediction system, the Factorization Machines model can capture click rate patterns observed when ads from a certain ad-category are placed on pages from a certain page-category. *Factorization machines are a good choice for tasks dealing with high dimensional sparse datasets, such as click prediction and item recommendation.*

Collaborative Filtering

Collaborative filtering is based on (user, item, rating) tuples. So, unlike content-based filtering, it leverages other users' experiences. The main concept behind collaborative filtering is that users with similar tastes (based on observed user-item interactions) are more likely to have similar interactions with items they haven't seen before.

Compared to content-based filtering, collaborative filtering provides better results for diversity (how dissimilar recommended items are); serendipity (a measure of how surprising the successful or relevant recommendations are); and novelty (how unknown recommended items are to a user). However, collaborative filtering is more computationally expensive and more complex and costly to implement and manage. Though some algorithms used for collaborative filtering such as factorization machines are more lightweight than others. Collaborative filtering has a cold start problem as well, since it has difficulty recommending new items without a large amount of interaction data to train a model.



In addition to these two “classic” categories of recommender systems, various neural net architectures are common in recommender systems. Some implement a form of collaborative filtering. Others expand recommender systems to handle temporal data to make recommendations based on a sequence of user actions that reflect the evolution of user interests. These systems were originally based on various kinds of Recurrent Neural Nets (RNNs). They now leverage Transformer-based models with self-attention to learn dependencies among items in users' behavior sequences.

Seq2seq

Sequence-to-sequence learning (Seq2Seq) is about training models to convert sequences from one domain (e.g. sentences in English) to sequences in another domain (e.g. the same sentences translated to French). In simple terms, the way seq2seq works is that it reads in a sentence, gets a sense of the entire sentence's "meaning" or context and then performs its assigned task. So, for the given use case, instead of translating from one language to another, you can "translate" from possibly misspelled lines in English to their corrected versions. The model reads in a possibly misspelled line, encodes it into a representation of its "meaning" and then outputs the corrected line.

Object2Vec

The Amazon SageMaker Object2Vec algorithm is a general-purpose neural embedding algorithm that is highly customizable. It can learn low-dimensional dense embeddings of high-dimensional objects. The embeddings are learned in a way that preserves the semantics of the relationship between pairs of objects in the original space in the embedding space. You can use the learned embeddings to efficiently compute nearest neighbors of objects and to visualize natural clusters of related objects in low-dimensional space, for example. You can also use the embeddings as features of the corresponding objects in downstream supervised tasks, such as classification or regression.

Object2Vec can be used to find semantically similar objects such as tickets. For example, in a customer support workflow, you might need to identify duplicate support tickets or route tickets to the correct support queue based on similarity of the text found in the ticket.

Built-in Forecast Algorithms

Amazon Forecast provides six built-in algorithms for you to choose from. These range from commonly used statistical algorithms like Autoregressive Integrated Moving Average (ARIMA), to complex neural network algorithms like CNN-QR and DeepAR+.

CNN-QR

arn:aws:forecast:::algorithm/CNN-QR

Amazon Forecast CNN-QR, Convolutional Neural Network - Quantile Regression, is a proprietary machine learning algorithm for forecasting time series using causal convolutional neural networks (CNNs). CNN-QR works best with large datasets containing hundreds of time series. It accepts item metadata, and is the only Forecast algorithm that accepts related time series data without future values.

DeepAR+

arn:aws:forecast:::algorithm/Deep_AR_Plus

Amazon Forecast DeepAR+ is a proprietary machine learning algorithm for forecasting time series using recurrent neural networks (RNNs). DeepAR+ works best with large datasets containing hundreds of feature time series. The algorithm accepts forward-looking related time series and item metadata.

Prophet

arn:aws:forecast:::algorithm/Prophet

Prophet is a time series forecasting algorithm based on an additive model where non-linear trends are fit with yearly, weekly, and daily seasonality. It works best with time series with strong seasonal effects and several seasons of historical data.

NPTS

arn:aws:forecast:::algorithm/NPTS

The Amazon Forecast Non-Parametric Time Series (NPTS) proprietary algorithm is a scalable, probabilistic baseline forecaster. NPTS is especially useful when working with sparse or intermittent time series. Forecast provides four algorithm variants: Standard NPTS, Seasonal NPTS, Climatological Forecaster, and Seasonal Climatological Forecaster.

ARIMA

arn:aws:forecast:::algorithm/ARIMA

Autoregressive Integrated Moving Average (ARIMA) is a commonly used statistical algorithm for time-series forecasting. The algorithm is especially useful for simple datasets with under 100 time series.

ETS

arn:aws:forecast:::algorithm/ETS

Exponential Smoothing (ETS) is a commonly used statistical algorithm for time-series forecasting. The algorithm is especially useful for simple datasets with under 100 time series, and datasets with seasonality patterns. ETS computes a weighted average over all observations in the time series dataset as its prediction, with exponentially decreasing weights over time.

Warm Start

In machine learning, a "warm start" refers to the practice of starting the training of a model from an already trained model's state rather than starting from scratch. This approach leverages previously learned weights and biases as initial conditions, which can lead to faster convergence and sometimes improved model performance, particularly in complex scenarios where training from scratch is computationally expensive and time-consuming.

What are Recurrent Neural Networks (RNNs)?

Recurrent Neural Networks (RNNs) are a type of neural network specialized for processing sequences of data. Imagine you're watching a movie and trying to understand the story. Each scene makes more sense when you remember what happened in the previous scenes. RNNs work similarly; they have a "memory" that captures information about what has been processed so far, essentially remembering past inputs to influence the current output.

How Do RNNs Work?

To simplify, think of RNNs as having a loop within them that allows information to be passed from one step of the sequence to the next. This loop helps the RNN keep track of all the previous data it has seen, making it very effective for tasks where the context from earlier input is crucial to understanding the current input.

For example, if you're trying to predict the next word in a sentence, it helps to remember the words that came before. RNNs do this by maintaining a state (think of it as a kind of memory) that updates as they process each word.

Key Characteristics of RNNs:

1. **Memory:** RNNs remember past information and use it to inform the current output, which is critical when the sequence of the data matters.
2. **Parameters Sharing:** Unlike other neural networks, RNNs share the same weights and biases across all steps. This means they perform the same task on every element of the sequence, but with the output dependent on the previous computations — which is a big plus in terms of efficiency.

Use Cases of RNNs

1. Language Translation (Machine Translation)

- RNNs can be used to translate text from one language to another. By remembering previous words in a sentence, RNNs can maintain the grammatical structure and context of the sentence, making the translation more accurate.

2. Speech Recognition

- Systems like voice-controlled assistants use RNNs to convert spoken language into text. RNNs can recognize speech patterns and the context in which words are spoken, which helps in understanding and transcribing speech accurately.

3. Time Series Prediction

- RNNs are ideal for predicting future values in a sequence like stock prices, weather forecasting, or sales forecasting. They can analyze the time series data and remember important events like seasonal effects or unexpected spikes.

4. Sentiment Analysis

- RNNs can analyze text data from social media or reviews to determine the sentiment expressed by the user. By understanding the sequence of words, RNNs can accurately identify emotions and opinions in the text.

5. Video Tagging and Anomaly Detection

- In video processing, RNNs can help tag scenes based on the activities occurring over time or detect unusual patterns in surveillance videos. They can process video frames as sequences, understanding ongoing events and providing real-time alerts.

Conclusion

RNNs are uniquely suited for sequential data or any data where time and context significantly impact the output. Their ability to remember previous inputs makes them powerful tools for any tasks involving sequences, from language processing and speech recognition to more complex sequence prediction tasks. Their use in business and technology continues to grow as more applications for their capabilities are discovered.

SageMaker Image Classification

The Amazon SageMaker image classification algorithm is a supervised learning algorithm that supports multi-label classification. It takes an image as input and outputs one or more labels assigned to that image. It uses a convolutional neural network (ResNet) that can be trained from scratch or trained using transfer learning when a large number of training images are not available.

SageMaker Object Detection

The Amazon SageMaker Object Detection algorithm detects and classifies objects in images using a single deep neural network. It is a supervised learning algorithm that takes images as input and identifies all instances of objects within the image scene. The object is categorized into one of the classes in a specified collection with a confidence score that it belongs to the class.

The Object Detection algorithm detects and classifies objects in images using a single deep neural network. It is a supervised learning algorithm that takes images as input and **identifies all instances of objects within the image scene.**



The object is categorized into one of the classes in a specified collection with a confidence score that it belongs to the class. Its location and scale in the image are indicated by a rectangular bounding box.

SageMaker semantic segmentation

The SageMaker semantic segmentation algorithm provides a fine-grained, pixel-level approach to developing computer vision applications. It tags every pixel in an image with a class label from a predefined set of classes. Tagging is fundamental for understanding scenes, which is critical to an

increasing number of computer vision applications, such as self-driving vehicles, medical imaging diagnostics, and robot sensing.

Word2Vec

mode represents the Word2vec architecture used for training. Valid values are: `batch_skipgram`, `skipgram`, or `cbow`.

mode is the only mandatory hyperparameter for both the Word2Vec (unsupervised) and Text Classification (supervised) modes of the SageMaker BlazingText algorithm.

The Amazon SageMaker BlazingText algorithm provides highly-optimized implementations of the Word2vec and text classification algorithms. The Word2vec algorithm is useful for many downstream natural language processing (NLP) tasks, such as sentiment analysis, named entity recognition, machine translation, etc. Text classification is an important task for applications that perform web searches, information retrieval, ranking, and document classification.

Word embedding is a vector representation of a word. Words that are semantically similar correspond to vectors that are close together. That way, word embeddings capture the semantic relationships between words.



Many natural language processing (NLP) applications learn word embeddings by training on large collections of documents. These pre-trained vector representations provide information about semantics and word distributions that typically improve the generalizability of other models that are later trained on a more limited amount of data.

Lifecycle Configuration Script to automate installation of packages

Amazon SageMaker provides you the ability to manually install additional libraries on your notebook instances. However, once your notebook instance is terminated, these additional customizations are removed as well, requiring that you manually add them again when you restart your notebook instances.

Scripts

Start notebook

Create notebook

This script will be run each time an associated notebook instance is started, including during initial creation. If the associated notebook instance is already started, it will be run the next time it is stopped and started. See [a curated list of sample scripts](#)

```
1  #!/bin/bash
2
3  set -e
4
5  # OVERVIEW
6  # This script installs a single pip package in a single SageMaker conda environments.
7
8  sudo -u ec2-user -i <<'EOF'
9  # PARAMETERS
10 PACKAGE=scipy
11 ENVIRONMENT=python3
12 source /home/ec2-user/anaconda3/bin/activate "$ENVIRONMENT"
13 pip install --upgrade "$PACKAGE"
14 source /home/ec2-user/anaconda3/bin/deactivate
15 EOF
```

Cancel

Create configuration

With the **Lifecycle configuration** feature in Amazon SageMaker, you can automate these customizations to be applied at different phases of the lifecycle of an instance. For example, you can write a script to install a list of libraries and, using the Lifecycle configuration feature, configure the scripts to automatically execute every time your notebook instance is started. Similarly, you can choose to automatically run the script only once when the notebook instance is created.

BoW and TF-IDF

The **Bag of Words** (BoW) and **Term Frequency-Inverse Document Frequency** (TF-IDF) are two fundamental text representation techniques used in natural language processing (NLP) for transforming text data into a format that can be easily analyzed and processed by algorithms. Here's a breakdown of what each does and how they are typically used:

Bag of Words (BoW)

The Bag of Words model is a way of representing text data when modeling text with machine learning algorithms. It simplifies text by breaking it down into a set of words and then counting their occurrences. Amazon SageMaker LDA is an unsupervised learning algorithm that attempts to describe a set of observations as a mixture of different categories. These categories are themselves a probability distribution over the features. LDA is a generative probability model, which means it attempts to provide a model for the distribution of outputs and inputs based on latent variables. This is opposed to discriminative models, which attempt to learn how inputs map to outputs. LDA is a "bag-of-words" model, which means that the order of words does not matter.

Here's how it works:

1. **Tokenization:** Split the text into words (tokens), using spaces and punctuation as delimiters.
2. **Vocabulary Building:** Create a vocabulary of all unique words in the text corpus.

3. **Vectorization:** For each document, create a vector where each element corresponds to a word in the vocabulary. The value in each element of the vector is the count of the word in the document.

Advantages:

- Simple and easy to understand.
- Efficient to build and model.

Disadvantages:

- Loses all information about the order of words (context and grammar), treating each word independently.
- High dimensionality: The vocabulary can grow large with the size of the corpus, leading to sparse matrices that are computationally expensive to handle.
- Ignores the relative importance of words in different documents.

Term Frequency-Inverse Document Frequency (TF-IDF)

TF-IDF improves on the Bag of Words model by not only counting the number of times a word appears in a document but also scaling this count based on how common the word is across all documents. It aims to reflect how important a word is to a document in a collection. The computation of TF-IDF is a two-step process:

1. **Term Frequency (TF):** This is the same as in BoW, measuring how frequently a term occurs in a document.

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

2. **Inverse Document Frequency (IDF):** This measures how important a term is. While computing TF, all terms are considered equally important. However, certain terms, like "is" or "the", may appear a lot of times but have little importance. IDF is calculated as:

$$IDF(t) = \log \left(\frac{\text{Total number of documents}}{\text{Number of documents with term } t \text{ in them}} \right)$$

3. **TF-IDF Score:** The TF-IDF score is the product of TF and IDF:

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

Advantages:

- Reduces the problem of common words that appear frequently in most documents but aren't informative.
- Prioritizes words uniquely characteristic of a document.

Disadvantages:

- Still ignores the order of words.
- Can be more computationally intense than simple BoW, especially on large corpora.

Both methods are foundational in text mining and information retrieval, often used for tasks like document classification, clustering, and sentiment analysis. TF-IDF generally provides better feature space representation by addressing some of the key limitations of the Bag of Words model.

Incremental Training

Over time, you might find that a model generates inferences that are not as good as they were in the past. With incremental training, you can use the artifacts from an existing model and use an expanded dataset to train a new model. Incremental training saves both time and resources.

You can use incremental training to:

Train a new model using an expanded dataset that contains an underlying pattern that was not accounted for in the previous training and which resulted in poor model performance.

Use the model artifacts or a portion of the model artifacts from a popular publicly available model in a training job. You don't need to train a new model from scratch.

Resume a training job that was stopped.

Train several variants of a model, either with different hyperparameter settings or using different datasets.

You can read more on this reference link -

<https://docs.aws.amazon.com/sagemaker/latest/dg/incremental-training.html>

Batch size

Batch size is a term used in machine learning and refers to the number of training examples utilized in one iteration. The batch size can be one of three options:

batch mode: where the batch size is equal to the total dataset thus making the iteration and epoch values equivalent

mini-batch mode: where the batch size is greater than one but less than the total dataset size. Usually, a number that can be divided into the total dataset size.

stochastic mode: where the batch size is equal to one. Therefore the gradient and the neural network parameters are updated after each sample.

Beta Testing

Beta Testing is one of the Acceptance Testing types used in traditional software engineering, which adds value to the product as the end-user (intended real user) validates the product for functionality, usability, reliability, and compatibility.

Transfer Learning

This is a technique used in **image classification algorithms**. The image classification algorithm takes an image as input and classifies it into one of the output categories. Image classification in Amazon SageMaker can be run in two modes: full training and transfer learning. In full training mode, the network is initialized with random weights and trained on user data from scratch. In transfer learning mode, the network is initialized with pre-trained weights and just the top fully connected layer is initialized with random weights. Then, the whole network is fine-tuned with new data. In this mode, training can be achieved even with a smaller dataset. This is because the network is already trained and therefore can be used in cases without sufficient training data.

Transfer learning for custom labels using a TensorFlow container and “bring your own algorithm” in Amazon SageMaker

by Amit Sharma | on 27 JUL 2018 | in [Amazon SageMaker](#), [Artificial Intelligence](#) | [Permalink](#) | [Comments](#) | [Share](#)

Data scientists and developers can use the Amazon SageMaker fully managed machine learning service to build and train machine learning (ML) models, and then directly deploy them into a production-ready hosted environment.

In this blog post we'll show you how to use Amazon SageMaker to do transfer learning using a TensorFlow container with our own code for training and inference.

Transfer learning is a well-known technique that is used in computer vision problems to re-train an existing trained neural network, such as AlexNet or ResNet[1], for additional custom labels. Amazon SageMaker also supports transfer learning for image classification through the built-in image classification algorithm and you could use your own labelled image data to re-train a ResNet[1] network. See [this image classification documentation](#) for more details on Amazon SageMaker. To understand when to use transfer learning and related guidelines, refer to [this](#).

Although the Amazon SageMaker built-in image classification algorithm is great for a variety of use cases, you might have scenarios where you need a different combination of the pre-trained network and the image data on which it has been trained. For example, some of the criteria to keep in mind are – similarity of new data set to the original data set, size of the new data set, number of labels required, accuracy of the model, size of the trained model, and, last but not least, the amount of compute power needed to re-train. If, for example, you are trying to deploy the trained model on a handheld device, you might be better off with a smaller footprint model such as MobileNet. Alternatively, if you want more compute-efficient models, Xception would be better than VGG16 or Inception.

In this blog post, we take an inception v3 network pre-trained on an ImageNet dataset and re-train it using the Caltech-256 dataset (Griffin, G. Holub, AD. Perona, P. The Caltech 256. Caltech Technical Report). Amazon SageMaker makes it very easy to bundle your own container and import it into Amazon Elastic Container Registry (Amazon ECR). Alternatively, you could use the container provided by Amazon SageMaker: <https://github.com/aws/sagemaker-tensorflow-containers>. We will customize the Amazon SageMaker TensorFlow container with our own transfer learning code in TensorFlow framework. Then we'll import this container into Amazon ECR, and use it for model training and inferencing.

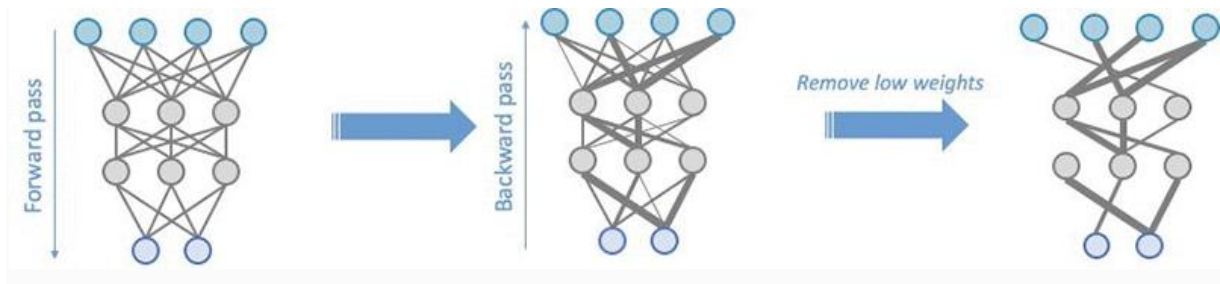
I hope this gives you sufficient background, now let us start putting it together.

Load data from Hadoop to Sagemaker model for training

Amazon SageMaker Spark is an open source Spark library that helps you construct Spark machine learning (ML) pipelines with SageMaker. This simplifies the integration of Spark ML stages with SageMaker stages, like model training and hosting.

Model Pruning

Model pruning aims to remove weights that don't contribute much to the training process. *Weights* are learnable parameters: they are randomly initialized and optimized during the training process. During the forward pass, data passes through the model. The loss function evaluates model output given the labels; during the backward pass, weights are updated to minimize the loss. To do so, the gradients of the loss with respect to the weights are computed, and each weight receives a different update.



After a few iterations, certain weights are typically more impactful than others; the goal of pruning is to remove the useless ones without significantly reducing model accuracy.

Common data formats for SageMaker

To prepare for training, you can preprocess your data using a variety of AWS services, including AWS Glue, Amazon EMR, Amazon Redshift, Amazon Relational Database Service, and Amazon Athena. After preprocessing, publish the data to an Amazon S3 bucket. For training, the data must go through a series of conversions and transformations, including:

- Training data serialization (handled by you)
- Training data deserialization (handled by the algorithm)
- Training model serialization (handled by the algorithm)
- Trained model deserialization (optional, handled by you)

When using Amazon SageMaker in the training portion of the algorithm, make sure to upload all data at once. If more data is added to that location, a new training call would need to be made to construct a brand new model.

The following table lists some of the commonly supported [ContentType](#) values and the algorithms that use them:

ContentType	Algorithms
<code>~application/x-image~</code>	Object Detection Algorithm, Semantic Segmentation
<code>~application/x-recordio~</code>	Object Detection Algorithm
<code>~application/x-recordio-protobuf~</code>	Factorization Machines, K-Means, k-NN, Latent Dirichlet Allocation, Linear Learner, NTM, PCA, RCF, Sequence-to-Sequence
<code>~application/jsonlines~</code>	BlazingText, DeepAR
<code>~image/jpeg~</code>	Object Detection Algorithm, Semantic Segmentation
<code>~image/png~</code>	Object Detection Algorithm, Semantic Segmentation
<code>~text/csv~</code>	IP Insights, K-Means, k-NN, Latent Dirichlet Allocation, Linear Learner, NTM, PCA, RCF, XGBoost
<code>~text/libsvm~</code>	XGBoost

Using Pipe Mode

In *Pipe mode*, your training job streams data directly from Amazon Simple Storage Service (Amazon S3). Streaming can provide faster start times for training jobs and better throughput. This is in contrast to *File mode*, in which your data from Amazon S3 is stored on the training instance volumes. File mode uses disk space to store both your final model artifacts and your full training dataset. By streaming in your data directly from Amazon S3 in Pipe mode, you reduce the size of Amazon Elastic

Block Store volumes of your training instances. Pipe mode needs only enough disk space to store your final model artifacts. See the [AlgorithmSpecification](#) for additional details on the training input mode.

Using CSV Format

Many Amazon SageMaker algorithms support training with data in CSV format. To use data in CSV format for training, in the input data channel specification, specify **text/csv** as the **ContentType**. Amazon SageMaker requires that a CSV file does not have a header record and that the target variable is in the first column. To run unsupervised learning algorithms that don't have a target, specify the number of label columns in the content type. For example, in this case **'content_type=text/csv;label_size=0'**. For more information, see [Now use Pipe mode with CSV datasets for faster training on Amazon SageMaker built-in algorithms](#).

Using RecordIO Format

In the protobuf recordIO format, SageMaker converts each observation in the dataset into a binary representation as a set of 4-byte floats, then loads it in the protobuf values field. If you are using Python for your data preparation, we strongly recommend that you use these existing transformations. However, if you are using another language, the protobuf definition file below provides the schema that you use to convert your data into SageMaker protobuf format.

Trained Model Deserialization

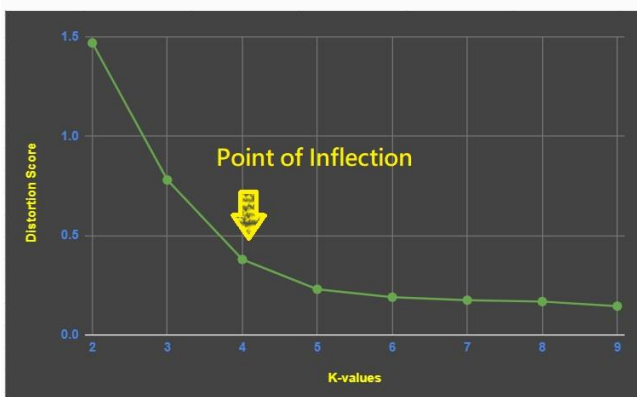
Amazon SageMaker models are stored as model.tar.gz in the S3 bucket specified in OutputDataConfig S3OutputPath parameter of the create_training_job call. The S3 bucket must be in the same AWS Region as the notebook instance. You can specify most of these model artifacts when creating a hosting model. You can also open and review them in your notebook instance. When model.tar.gz is untarred, it contains model_algo-1, which is a serialized Apache MXNet object.

K-Means Hyperparameters

Parameter Name	Description	Required	Valid Values	Default Value
feature_dim	The number of features in the input data.	Yes	Positive integer	-
k	The number of required clusters.	Yes	Positive integer	-
epochs	The number of passes done over the training data.	No	Positive integer	1
eval_metrics	A JSON list of metric types used to report a score for the model. Allowed metrics are 'msd' and 'ssd'.	No	Either ['msd'], ['ssd'], or ['msd', 'ssd']	['msd']
extra_center_factor	Determines the initial number of cluster centers ($K = \text{num_clusters} * \text{extra_center_factor}$) and reduces to k during the finalization.	No	Either a positive integer or auto	auto
half_life_time_size	Used to determine the decay weight for an observation in computing a cluster mean, with the weight halving after half_life_time_size.	No	Non-negative integer	0
init_method	Method for choosing initial cluster centers. 'random' selects centers randomly, 'kmeans++' optimizes cluster compactness.	No	Either random or kmeans++	random
local_lloyd_init_method	Initialization method for Lloyd's EM procedure in building the final model.	No	Either random or kmeans++	kmeans++
local_lloyd_max_iter	Maximum number of iterations for Lloyd's EM procedure.	No	Positive integer	300
local_lloyd_num_trials	Number of times to run Lloyd's EM procedure to find the least loss configuration.	No	Either a positive integer or auto	auto
local_lloyd_tol	Tolerance for the change in loss for early stopping in Lloyd's EM procedure.	No	Float, range [0, 1]	0.0001
mini_batch_size	Number of observations per mini-batch for data processing.	No	Positive integer	5000

The elbow method runs k-means clustering on the dataset for a range of values for k (e.g., 1-10), and then for each value of k computes an average score for all clusters. In this scenario, the distortion score is computed – the sum of square distances from each point to its assigned center.

When the resulting metrics for each model are plotted, it is possible to determine the best value for k visually. If the line chart looks like an “arm”, then the “elbow” (the point of inflection on the curve) is the best value of k. The “arm” can be either up or down, but if there is a strong inflection point, it is a good indication that the underlying model fits best at that point.



XGBoost Hyperparameters

Parameter Name	Description
num_class	Required if objective is set to multi or multi . Valid values: Integer.
num_round	The number of rounds to run the training. Required. Valid values: Integer.
alpha	L1 regularization term on weights. Increasing this value makes models more conservative. Optional. Valid values: Float. Default value: 0
base_score	The initial prediction score of all instances, global bias. Optional. Valid values: Float. Default value: 0.5
booster	Which booster to use. The gbtrees and dart values use a tree-based model, while gblinear uses a linear function. Optional. Valid values: String. One of "gbtree", "gblinear", or "dart". Default value: "gbtree"
colsample_bylevel	Subsample ratio of columns for each split, in each level. Optional. Valid values: Float. Range: [0,1]. Default value: 1
colsample_bynode	Subsample ratio of columns from each node. Optional. Valid values: Float. Range: (0,1). Default value: 1
colsample_bytree	Subsample ratio of columns when constructing each tree. Optional. Valid values: Float. Range: [0,1]. Default value: 1
csv_weights	When enabled, differentiates the importance of instances for csv input by taking the second column in training data as the instance weights. Optional. Valid values: 0 or 1 Default value: 0
deterministic_histogram	Builds histogram on GPU deterministically if tree_method is set to gpu_hist. Optional. Valid values: String. Range: "true" or "false". Default value: "true"
early_stopping_rounds	Trains until the validation score stops improving. Optional. Valid values: Integer.

eta	Step size shrinkage used in updates to prevent overfitting. Optional. Valid values: Float. Range: [0,1]. Default value: 0.3
eval_metric	Evaluation metrics for validation data, assigned according to the objective. Optional. Valid values: String. Default value: Default according to objective.
gamma	Minimum loss reduction required to make a further partition on a leaf node of the tree. Optional. Valid values: Float. Range: [0,∞). Default value: 0
grow_policy	Controls how new nodes are added to the tree, supported only if tree_method is set to hist. Optional. Valid values: String. Either "depthwise" or "lossguide". Default value: "depthwise"
interaction_constraints	Specify groups of variables that are allowed to interact. Optional. Valid values: Nested list of integers. Default value: None
lambda	L2 regularization term on weights. Optional. Valid values: Float. Default value: 1
lambda_bias	L2 regularization term on bias. Optional. Valid values: Float. Range: [0.0, 1.0]. Default value: 0
max_bin	Maximum number of discrete bins to bucket continuous features, used only if tree_method is set to hist. Optional. Valid values: Integer. Default value: 256
max_delta_step	Maximum delta step allowed for each tree's weight estimation. Optional. Valid values: Integer. Range: [0,∞). Default value: 0
max_depth	Maximum depth of a tree. Optional. Valid values: Integer. Range: [0,∞) Default value: 6
max_leaves	Maximum number of nodes to be added, relevant only if grow_policy is set to lossguide. Optional. Valid values: Integer. Default value: 0
min_child_weight	Minimum sum of instance weight needed in a child. Optional. Valid values: Float. Range: [0,∞). Default value: 1

monotone_constraints	Specifies monotonicity constraints on any feature. Optional. Valid values: Tuple of Integers. Default value: (0, 0)
normalize_type	Type of normalization algorithm. Optional. Valid values: Either tree or forest. Default value: tree
nthread	Number of parallel threads used to run xgboost. Optional. Valid values: Integer. Default value: Maximum number of threads.
objective	Specifies the learning task and the corresponding learning objective. Optional. Valid values: String Default value: "reg"
one_drop	When enabled, at least one tree is always dropped during the dropout. Optional. Valid values: 0 or 1 Default value: 0
process_type	The type of boosting process to run. Optional. Valid values: String. Either "default" or "update". Default value: "default"
rate_drop	The dropout rate that specifies the fraction of previous trees to drop during the dropout. Optional. Valid values: Float. Range: [0.0, 1.0]. Default value: 0.0
refresh_leaf	Parameter of the 'refresh' updater plug-in. Optional. Valid values: 0/1 Default value: 1
sample_type	Type of sampling algorithm. Optional. Valid values: Either uniform or weighted. Default value: uniform
scale_pos_weight	Controls the balance of positive and negative weights. Optional. Valid values: float Default value: 1
seed	Random number seed. Optional. Valid values: integer Default value: 0
single_precision_histogram	Uses single precision to build histograms if tree_method is set to hist or gpu_hist. Optional. Valid values: String. Range: "true" or "false" Default value: "false"

sketch_eps	Used for approximate greedy algorithm. Optional. Valid values: Float. Range: [0, 1]. Default value: 0.03
skip_drop	Probability of skipping the dropout procedure during a boosting iteration. Optional. Valid values: Float. Range: [0.0, 1.0]. Default value: 0.0
subsample	Subsample ratio of the training instance. Optional. Valid values: Float. Range: [0,1]. Default value: 1
tree_method	The tree construction algorithm used in XGBoost. Optional. Valid values: One of auto, exact, approx, hist, or gpu_hist. Default value: auto
tweedie_variance_power	Controls the variance of the Tweedie distribution. Optional. Valid values: Float. Range: (1, 2). Default value: 1.5
updater	Defines the sequence of tree updaters to run. Optional. Valid values: comma-separated string. Default value: grow_colmaker, prune
use_dask_gpu_training	Set to "true" for running distributed GPU training with Dask. Optional. Valid values: String. Range: "true" or "false" Default value: "false"
verbosity	Verbosity of printing messages. Valid values: 0 (silent), 1 (warning), 2 (info), 3 (debug). Optional. Default value: 1

Amazon Augmented AI (Amazon A2I)

Some machine learning applications need human oversight to ensure accuracy with sensitive data, to help provide continuous improvements and, retrain models with updated predictions. However, in these situations, you're often forced to choose between a machine learning only or human only system. Companies are looking for the best of both worlds — integrating machine learning systems into your workflow while keeping a human eye on the results to guarantee a needed precision.

Amazon Augmented AI (Amazon A2I) enables you to build the workflows that are required for human review of machine learning predictions. Amazon Textract is directly integrated with Amazon A2I so that you can easily get low-confidence results from Amazon Textract's AnalyzeDocument API operation reviewed by humans.

Task type

Human reviewers can review data extracted from Amazon Textract forms, moderate images in Amazon Rekognition, or review data using a custom workflow. [Learn more about task types](#)

Choose task type

☒ **Textract - Key-value pair extraction**

Get human review of key-value pairs extracted by Amazon Textract.

Employment Application

Application Information

Full Name: Jane Doe

Phone number: 550-0100

Home address: 123 Any Street, Any town, USA

Mail address: same as home address

☐ **Rekognition - Image moderation**

Get human review of unsafe content, such as explicit adult content or violent content.

Safe

Appropriate



☐ **Custom**

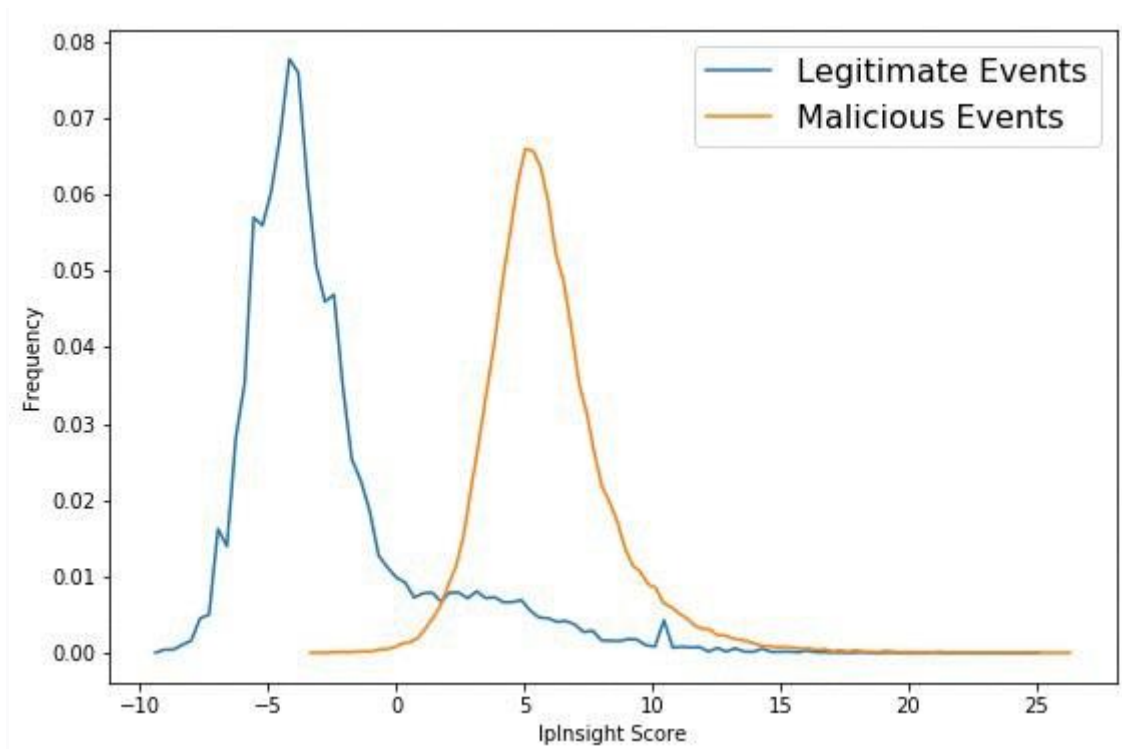
Get human review of your data with a custom machine learning workflow.



You can use Amazon Textract’s AnalyzeDocument API for form data extraction and the Amazon A2I console to specify the conditions under which Amazon A2I routes predictions to reviewers. The conditions are set based on the confidence threshold of important form keys. For example, you can send a document to a human to review if the key “Name” or its associated value “Jane Doe” was detected with low confidence.

IP Insights

The **Amazon SageMaker IP Insights algorithm** uses statistical modeling and neural networks to capture associations between online resources (for example, online bank accounts) and IPv4 addresses. Under the hood, the algorithm learns vector representations for the online resources and IP addresses where each point is close together if they have been used together. The algorithm itself can learn and incorporate many of the latent factors without requiring us to explicitly model them.



It can be used to analyze access logs and make predictions about whether an access attempt (such as a login event or an online transaction) is suspicious based on the IP address and a user's access history. This is even the case when an IP address has not been seen before.

What is Amazon Fraud Detector?

Amazon Fraud Detector is a fully managed service provided by AWS designed to identify potentially fraudulent activities in real time. This service utilizes supervised machine learning (ML) and more than 20 years of fraud detection expertise from AWS and Amazon.com to automatically identify potentially fraudulent activities such as online payment fraud and fake account creations.

Key Features of Amazon Fraud Detector:

- **Ease of Use:** It enables users to create a fraud detection model using pre-built templates and based on historical data they provide. Users do not need to be ML experts.
- **Automated Machine Learning:** The service uses machine learning under the hood to train, test, and deploy fraud detection models.
- **Real-Time Prediction:** Once set up, it can detect fraud in milliseconds by evaluating activities in real-time.

Use Cases:

- **Online Payment Fraud:** Detecting fraudulent transactions during checkout processes.
- **New Account Frauds:** Identifying fake account creations that might be set up for malicious purposes.
- **Guest Checkout Fraud:** Spotting fraud in transactions that occur through guest checkouts in e-commerce scenarios.

What is the Random Cut Forest Algorithm?

The Random Cut Forest (RCF) algorithm is a machine learning algorithm used for anomaly detection on streaming data. It is available as one of the built-in algorithms in Amazon SageMaker and is effective in identifying outliers or anomalous data points within data.

Key Features of Random Cut Forest

- **Unsupervised Learning:** RCF is an unsupervised algorithm, meaning it doesn't require labeled data to function.
- **Scalability:** Efficiently handles streaming data, making it ideal for real-time anomaly detection.
- **Versatility:** Can be applied to a wide range of data types and is not specific to fraud detection.

Use Cases

- **Anomaly Detection in IT Operations:** Identifying unusual patterns in system metrics that might indicate issues like a system outage or breach.
- **Monitoring IoT Device Outputs:** Detecting anomalies in IoT device functionalities which could indicate malfunction or external tampering.
- **Financial Monitoring:** Spotting unusual financial transactions that deviate from typical patterns.

Differences Between Amazon Fraud Detector and Random Cut Forest Algorithm

- **Purpose and Specialization:**
 - **Amazon Fraud Detector** is specifically tailored for detecting types of fraud and is built on specific models that AWS has developed for this purpose. It's a high-level service that requires minimal setup from the user to start detecting various kinds of fraud.
 - **Random Cut Forest** is a more general-purpose algorithm focused on anomaly detection, which can be used in many different contexts beyond fraud, such as detecting anomalies in IT, healthcare monitoring, or industrial systems.
- **Ease of Deployment:**
 - **Amazon Fraud Detector** provides a user-friendly interface with pre-built templates and automatic model training, evaluation, and deployment, making it accessible even to those without deep technical expertise.
 - **Random Cut Forest** requires users to set up and manage their models within Amazon SageMaker, which offers flexibility but also requires a bit more setup and understanding of machine learning principles.
- **Real-Time Operation:**
 - Both services can operate in real-time, but Amazon Fraud Detector is optimized specifically for real-time fraud detection use cases, while Random Cut Forest is designed for real-time anomaly detection in a broader range of data streaming applications.

Conclusion

While both Amazon Fraud Detector and the Random Cut Forest algorithm can be used to detect irregular patterns and potential fraud, Amazon Fraud Detector is a specialized service designed specifically for easy setup and use in fraud detection across various scenarios. In contrast, Random Cut Forest is a versatile, unsupervised algorithm suitable for detecting anomalies across a broad spectrum of data types and industries. The choice between them would depend on the specific

requirements of the task at hand, including the need for specialization, ease of use, and context of the data.

Parameters used by Amazon Fraud Detector

AWS Fraud Detector is designed to help users easily implement effective fraud detection solutions without requiring deep technical expertise in machine learning. When setting up a fraud detection model with AWS Fraud Detector, there are several parameters and model types to choose from, depending on your specific needs.

When configuring a fraud detection solution in AWS Fraud Detector, you typically need to specify several parameters:

1. **Event Type:** Defines the schema that includes the variables you will use in your fraud detection model, such as email address, IP address, transaction amount, etc. This schema correlates to the data points you have about the interactions or transactions you are monitoring.
2. **Entity Type:** Specifies the type of entity that the event involves, which could be a customer, a merchant, or any other principal you're monitoring for fraud.
3. **Labels:** You need to define what constitutes fraudulent and legitimate behavior in your dataset. Labels are used to categorize historical events you provide to train your model, such as "fraud" and "legit".
4. **Variables:** These are the features used to detect fraud. Variables can be derived from your data fields, and AWS Fraud Detector also allows you to create derived variables using SQL-like expressions.
5. **Models:** Depending on your needs, you choose a model type and configure its specific parameters, such as training data split, model type, and detectors.
6. **Training Data Split:** You decide how your data is split between training and validation, which impacts how the model learns and validates its accuracy.
7. **Detectors:** After your model is trained, you create a detector, which uses the model to evaluate new activity for fraud. Detectors include decision logic such as rules to interpret the model's score.

Model Types in AWS Fraud Detector

AWS Fraud Detector supports several model types, each designed for different aspects of fraud detection:

1. **Online Fraud Insights (OFI):** This is the primary model used for detecting potential fraud in online activities in real-time. It's suitable for various use cases like online payments, account registrations, and guest checkouts.
2. **Transaction Fraud Insights (TFI):** Specifically designed for more complex transactional fraud scenarios, TFI models are tuned to process transaction-specific data points and are ideal for payment fraud, banking transactions, and insurance claim fraud.

Business Use Cases for Model Types

- **Online Fraud Insights (OFI):**
 - **Use Case:** E-commerce platforms can use OFI models to analyze transactions at the point of sale, detecting potential fraud based on factors like transaction amount, frequency, and the device used for the purchase.
 - **Benefits:** Rapid detection and low latency are key benefits, allowing real-time decisions that prevent fraud before it impacts the business.

- **Transaction Fraud Insights (TFI):**
 - **Use Case:** Financial institutions might use TFI to monitor for unusual patterns in account transfers or loan applications, leveraging more complex data relationships and historical patterns.
 - **Benefits:** TFI models offer high accuracy in environments where transactions have high variability and the cost of fraud can be significant.

Conclusion

Choosing the right parameters and model type in AWS Fraud Detector involves understanding your specific data, the type of fraud you are dealing with, and how quickly you need to detect potential issues. Each model type provides a different level of complexity and insight, allowing you to tailor your fraud detection mechanism to your operational needs and risk management strategies.

AWS Sagemaker Debugger

AWS SageMaker Debugger is a powerful tool designed to help machine learning practitioners understand, analyze, and improve the training of machine learning models by providing insights into their internal operations. As you're preparing for the AWS Machine Learning Specialty exam, understanding the features and capabilities of SageMaker Debugger is crucial. Here's a detailed breakdown of its primary features:

1. Real-Time Monitoring of Model Parameters

What It Does: SageMaker Debugger allows you to monitor various aspects of your model training in real-time. This includes tracking the gradients, weights, biases, and other tensors as the model trains.

Benefits: By monitoring these metrics in real-time, you can quickly identify and diagnose training issues such as overfitting, underfitting, or poor convergence, saving time and computational resources.

2. Automatic Detection of Common Training Issues

What It Does: The tool can automatically detect common problems that occur during the training of machine learning models, such as vanishing or exploding gradients.

Benefits: This automated anomaly detection helps in quickly pinpointing issues without manually analyzing complex tensor data, making the debugging process more efficient.

3. Custom Rules for Monitoring

What It Does: Besides built-in rules provided by AWS, Debugger allows you to create custom rules to specify which conditions to monitor for your specific use case.

Benefits: Custom rules enable the tool to be flexible and tailored to your specific needs, allowing deeper insights into model behavior based on custom-defined criteria.

4. Analysis and Visualization

What It Does: SageMaker Debugger integrates with other AWS services like Amazon CloudWatch and SageMaker Studio for visualization and further analysis of the debug data.

Benefits: Visualizing the training process can make it easier to understand complex patterns and anomalies in model behavior. Integration with tools like CloudWatch allows for setting up alerts based on the metrics monitored by Debugger.

5. Integration with Popular Frameworks

What It Does: Debugger supports popular machine learning frameworks like TensorFlow, PyTorch, XGBoost, and Apache MXNet.

Benefits: This makes it widely applicable to a variety of machine learning projects and environments, ensuring that you can use it regardless of the chosen framework for model development.

6. Saving Data for Offline Analysis

What It Does: Debugger can save the tensors during training to Amazon S3, allowing for offline analysis after the training job has completed.

Benefits: Offline analysis can provide deeper insights without the constraints of real-time processing, allowing for more thorough examinations of model behavior over time.

7. Model Profiling

What It Does: Debugger includes a profiling feature that helps you understand how your training job utilizes the underlying hardware resources.

Benefits: Profiling can identify bottlenecks in computation, memory usage, or data loading processes. This helps optimize model training to be more cost-effective and faster, by suggesting improvements in resource allocation and utilization.

Key points about Amazon SageMaker Debugger:

- **Automatic Monitoring:** Tracks and profiles training jobs in real-time.
- **Debugging Rules:** Pre-built and custom rules to detect issues.
- **Insights and Alerts:** Provides insights and sends alerts on detected issues.
- **Visualization:** Interactive charts and graphs for analyzing training metrics.
- **Integration:** Works with built-in and custom algorithms.

Amazon SageMaker Debugger supports the following frameworks:

- **TensorFlow:** Monitors training jobs, providing detailed insights and debugging capabilities.
 - **Apache MXNet:** Captures and visualizes training data, detecting potential issues.
 - **PyTorch:** Offers built-in rules for detecting issues in training jobs.
 - **XGBoost:** Provides debugging and profiling for training jobs to optimize performance.
 - **Hugging Face:** Integrates with SageMaker for enhanced model training monitoring.
-
- **Automatic Data Capture:** SageMaker Debugger captures training data tensors and metrics in real-time.

- **Hooks and Rules:** Uses hooks to integrate with training jobs and predefined or custom rules to identify issues.
- **Analysis and Visualization:** Provides tools for visualizing and analyzing captured data within SageMaker Studio.
- **Early Issue Detection:** Facilitates early detection and diagnosis of training issues, improving model performance.

SageMaker Debugger Built-In Rules

- **Predefined Rules:** Includes rules for common issues like vanishing gradients, overfitting, and loss not decreasing.
- **Rule Examples:** LossNotDecreasing, VanishingGradient, Overtraining.
- **Customization:** Users can customize and create new rules based on specific needs.
- **Integration:** Configured via configuration files or SDKs.
- **Alerts and Insights:** Provides alerts and insights during training when rules are triggered.

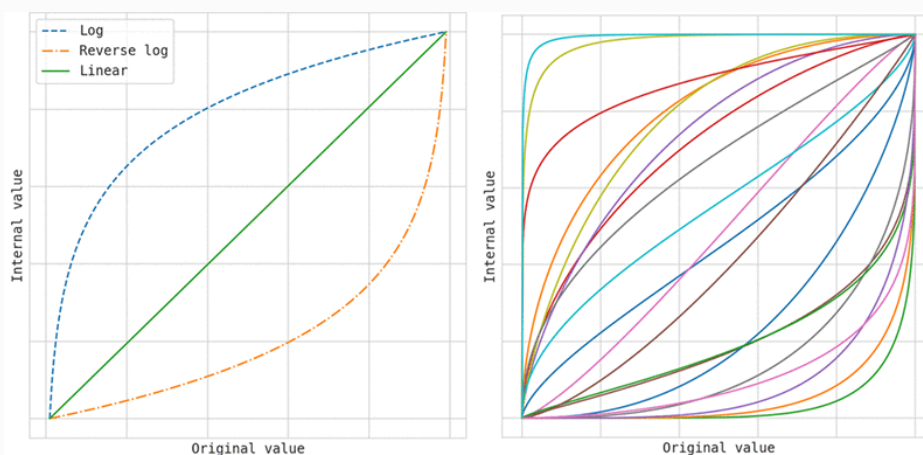
SageMaker Debugger Built-In Rules

- **AllZero:** Detects if all tensor values are zero.
- **ClassImbalance:** Identifies imbalances in class distributions.
- **Confusion:** Generates a confusion matrix for classification models.
- **ExplodingTensor:** Detects exploding tensor values during training.
- **LossNotDecreasing:** Monitors if loss does not decrease over time.
- **Overtraining:** Identifies if a model is overtrained.
- **OvertrainingWithValidation:** Similar to Overtraining but includes validation data.
- **PoorWeightInitialization:** Detects poor initialization of model weights.
- **SaturatedActivation:** Identifies saturation in activation functions.
- **StalledTrainingRule:** Detects if training progress stalls.

Hyperparameter tuning

By default, SageMaker assumes a uniform distribution of hyperparameter values and uses linear scaling to select values in a search range. However, this may not be the most effective approach for some types of hyperparameters, such as a learning rate whose typical value spans multiple orders of magnitude and is not uniformly distributed. Customers can either rely on SageMaker to automatically determine the scaling method or manually select it, for each hyperparameter to be tuned.

During hyperparameter tuning, SageMaker attempts to figure out if your hyperparameters are log-scaled or linear-scaled. Initially, it assumes that hyperparameters are linear-scaled. If they are in fact log-scaled, it might take some time for SageMaker to discover that. If you know that a hyperparameter is log-scaled and can convert it yourself, doing so could improve hyperparameter optimization.



Logarithmic scaling works only for ranges that have only values greater than 0. Choose logarithmic scaling when you are searching for a range that spans several orders of magnitude. For example, if you are tuning a linear learner model and you specify a range of values between .0001 and 1.0 for the `learning_rate` hyperparameter, searching uniformly on a logarithmic scale gives you a better sample of the entire range than searching on a linear scale would. This is because searching on a linear scale would, on average, devote 90 percent of your training budget to only the values between .1 and 1.0, leaving only 10 percent of your training budget for the values between .0001 and .1.

Grid Search

When using grid search, hyperparameter tuning chooses combinations of values from the range of categorical values that you specify when you create the job. Only categorical parameters are supported when using the grid search strategy. You do not need to specify the `MaxNumberOfTrainingJobs`. The number of training jobs created by the tuning job will be automatically calculated to be the total number of distinct categorical combinations possible. If specified, the value of `MaxNumberOfTrainingJobs` should equal the total number of distinct categorical combinations possible.

Random Search

When using random search, hyperparameter tuning chooses a random combination of values from within the ranges that you specify for hyperparameters for each training job it launches. Because the choice of hyperparameter values doesn't depend on the results of previous training jobs, you can run the maximum number of concurrent training jobs without affecting the performance of the tuning.

For an example notebook that uses random search, see the [Random search and hyperparameter scaling with SageMaker XGBoost and Automatic Model Tuning](#) notebook.

Bayesian Optimization

Bayesian optimization treats hyperparameter tuning like a [regression](#) problem. Given a set of input features (the hyperparameters), hyperparameter tuning optimizes a model for the metric that you choose. To solve a regression problem, hyperparameter tuning makes guesses about which hyperparameter combinations are likely to get the best results, and runs training jobs to test these values. After testing a set of hyperparameter values, hyperparameter tuning uses regression to choose the next set of hyperparameter values to test.

Hyperparameter tuning uses an Amazon SageMaker implementation of Bayesian optimization.

When choosing the best hyperparameters for the next training job, hyperparameter tuning considers everything that it knows about this problem so far. Sometimes it chooses a combination of hyperparameter values close to the combination that resulted in the best previous training job to incrementally improve performance. This allows hyperparameter tuning to exploit the best known results. Other times, it chooses a set of hyperparameter values far removed from those it has tried. This allows it to explore the range of hyperparameter values to try to find new areas that are not yet well understood. The explore/exploit trade-off is common in many machine learning problems.

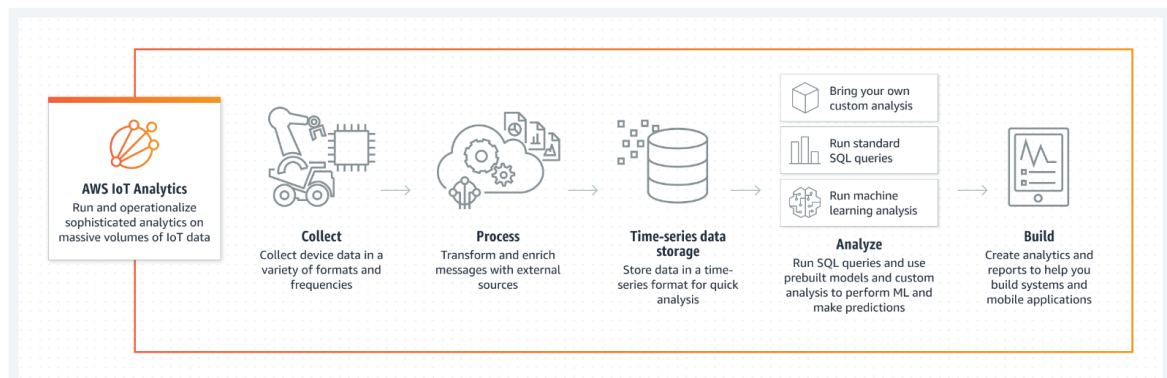
Sagemaker Automatic Model Tuning

When you use automatic model tuning, the linear learner internal tuning mechanism is turned off automatically. This sets the number of parallel models, `num_models`, to 1

Automatic model tuning, also known as hyperparameter tuning, finds the best version of a model by running many jobs that test a range of hyperparameters on your dataset. You choose the tunable hyperparameters, a range of values for each, and an objective metric. You choose the objective metric from the metrics that the algorithm computes. Automatic model tuning searches the hyperparameters chosen to find the combination of values that result in the model that optimizes the objective metric. *When you use automatic model tuning, the linear learner internal tuning mechanism is turned off automatically. This sets the number of parallel models, num_models, to 1.*

AWS IoT Analytics

AWS IoT Analytics simplifies the difficult steps required to analyze massive volumes of IoT data, without the cost and complexity of building an IoT analytics platform.



Configure label for unsupervised algorithm in sagemaker

Many Amazon SageMaker algorithms support training with data in CSV format. To use data in CSV format for training, in the input data channel specification, specify `text/csv` as the `ContentType`. Amazon SageMaker requires that a CSV file does not have a header record and that the target variable is in the first column.

To run unsupervised learning algorithms that don't have a target, specify the number of label columns in the content type. For example, in this case `'content_type=text/csv;label_size=0'`.

Tackle increased workload

Amazon SageMaker supports automatic scaling (autoscaling) for your hosted models. *Autoscaling* dynamically adjusts the number of instances provisioned for a model in response to changes in your workload. When the workload increases, autoscaling brings more instances online. When the workload decreases, autoscaling removes unnecessary instances so that you don't pay for provisioned instances that you aren't using.

When hosting models in production, you can configure the endpoint to elastically scale the deployed ML compute instances. For each production variant, you specify the number of ML compute instances that you want to deploy. When you specify two or more instances, SageMaker launches them in multiple Availability Zones. This ensures continuous availability.

Huge Data Dimensionality reduction

For datasets with a large number of observations and features, you should use PCA in randomized mode as this mode uses an approximation algorithm.

Principal Component Analysis (PCA) Algorithm

[PDF](#) | [Kindle](#) | [RSS](#)

PCA is an unsupervised machine learning algorithm that attempts to reduce the dimensionality (number of features) within a dataset while still retaining as much information as possible. This is done by finding a new set of features called *components*, which are composites of the original features that are uncorrelated with one another. They are also constrained so that the first component accounts for the largest possible variability in the data, the second component the second most variability, and so on.

In Amazon SageMaker, PCA operates in two modes, depending on the scenario:

- **regular:** For datasets with sparse data and a moderate number of observations and features.
- **randomized:** For datasets with both a large number of observations and features. This mode uses an approximation algorithm.

PCA uses tabular data.

The rows represent observations you want to embed in a lower dimensional space. The columns represent features that you want to find a reduced approximation for. The algorithm calculates the covariance matrix (or an approximation thereof in a distributed manner), and then performs the singular value decomposition on this summary to produce the principal components.

Validate a ML Model

After training a model, evaluate it to determine whether its performance and accuracy enable you to achieve your business goals. You might generate multiple models using different methods and evaluate each. For example, you could apply different business rules for each model, and then apply various measures to determine each model's suitability. You might consider whether your model needs to be more sensitive than specific (or vice versa).

You can evaluate your model using historical data (offline) or live data:

- **Offline testing**—Use historical, not live, data to send requests to the model for inferences. Deploy your trained model to an alpha endpoint, and use historical data to send inference requests to it. To send the requests, use a Jupyter notebook in your Amazon SageMaker notebook instance and either the AWS SDK for Python (Boto) or the high-level Python library provided by SageMaker.
- **Online testing with live data**—SageMaker supports A/B testing for models in production by using production variants. Production variants are models that use the same inference code and are deployed on the same SageMaker endpoint. You configure the production variants so that a small portion of the live traffic goes to the model that you want to validate. For example, you might choose to send 10% of the traffic to a model variant for evaluation. After you are satisfied with the model's performance, you can route 100% traffic to the updated model. For an example of testing models in production, see [Production variants](#).

Monitor and Visualize metrics while training models on Sagemaker

Data scientists and developers can now quickly and easily access, monitor, and visualize metrics that are computed while training machine learning models on Amazon SageMaker. You can now specify the metrics you want to track by using the AWS Management Console for Amazon SageMaker or by using the Amazon SageMaker Python SDK APIs. After the model training starts, Amazon SageMaker will automatically monitor and stream the specified metrics in real time to the Amazon CloudWatch console for visualizing time-series curves, such as loss curves and accuracy curves. You can also access the metrics programmatically using Amazon SageMaker Python SDK APIs.

Model training is an iterative process of teaching a model to make predictions by presenting examples from a training dataset. Typically a training algorithm computes several metrics such as training loss and prediction accuracy that help diagnose whether the model is learning well and will generalize well for making predictions on unseen data. This diagnosis is especially helpful when you are tuning your model's hyperparameters or evaluating whether your model has the potential for deploying to production.

SageMaker Neo

Neo is a capability of Amazon SageMaker that enables machine learning models to train once and run anywhere in the cloud and *at the edge*.

Since the robot should be able to classify the images of marine life forms in an autonomous way with low latency, Neo is the right fit for the given use case.

Sagemaker with built-in algorithms

Provide the Amazon Resource Name (ARN) of an IAM role that SageMaker can assume to perform tasks on your behalf

You must provide the Amazon Resource Name (ARN) that SageMaker assumes to perform tasks on your behalf during model training. You must grant this role the necessary permissions so that SageMaker can successfully complete model training. You can use the mandatory RoleArn parameter to configure this option.

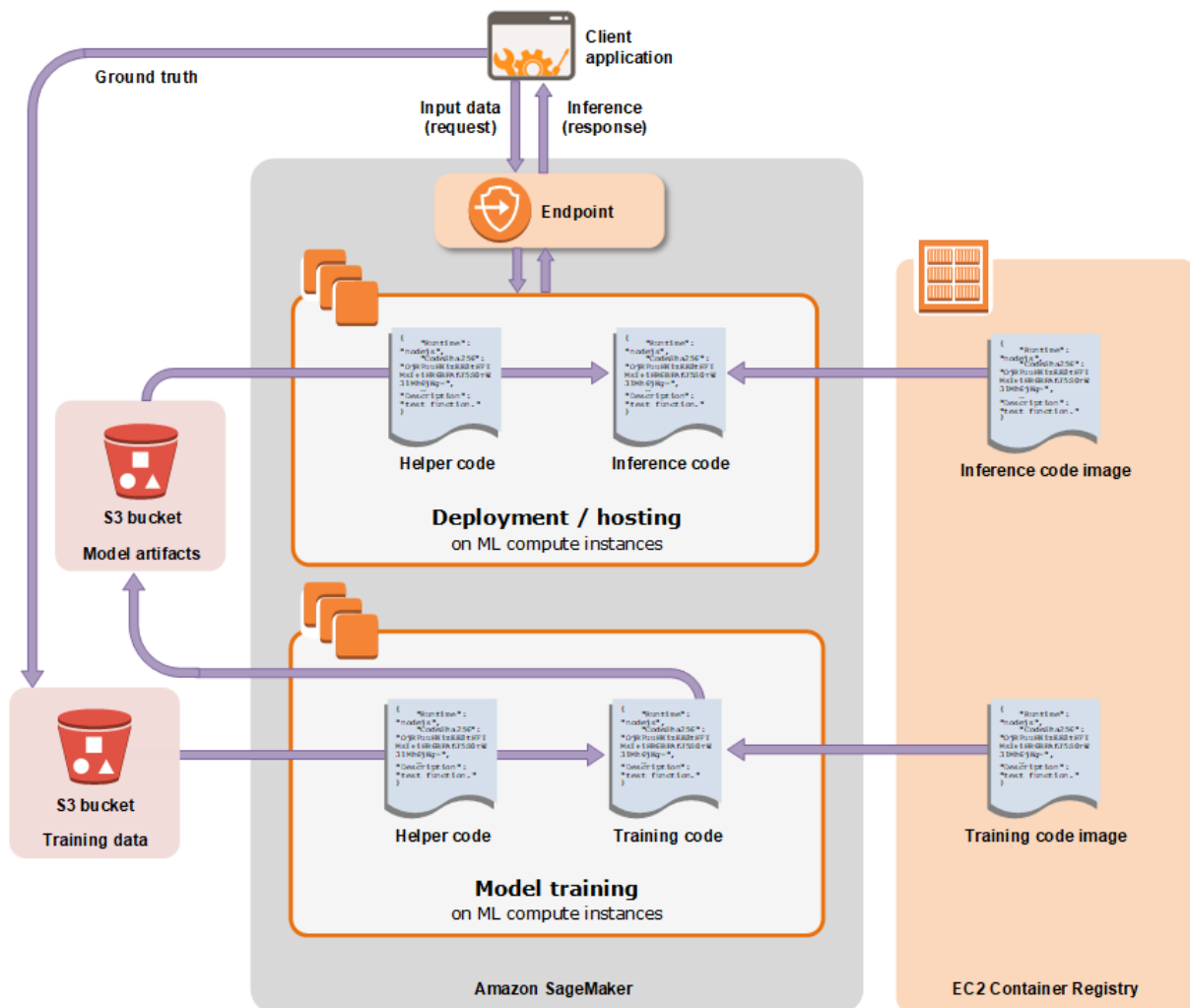
Specify the resources, ML compute instances, and ML storage volumes to deploy for model training

You need to specify the resources, ML compute instances, and ML storage volumes to deploy for model training. In distributed training, you specify more than one instance. ML storage volumes store model artifacts and incremental states. Training algorithms might also use ML storage volumes for scratch space. If you want SageMaker to use the ML storage volume to store the training data, choose File as the TrainingInputMode in the algorithm specification. For distributed training algorithms, specify an instance count greater than 1. You can use the mandatory ResourceConfig parameter to configure this option.

Specify the output path on an Amazon S3 bucket where the trained model will persist

Specifies the path to the S3 location where you want to store model artifacts. SageMaker creates subfolders for the artifacts. You can use the mandatory OutputDataConfig parameter to configure this option.

Training a model with SageMaker:



AWS Multimodel Endpoints

Cost-Effective: Multi-model endpoints host multiple models in one container, reducing costs and improving resource utilization.

Dynamic Loading: Models are loaded into memory from S3 on invocation, with caching to reduce latency.

Auto Scaling: Supports auto-scaling and time-sharing of memory resources.

Instance Types: Supports both CPU and GPU instances, with recommendations based on performance and cost needs.

Use Cases: Ideal for large numbers of models with similar sizes and invocation patterns.

For creating multimodel endpoints:

- **Create Model:** Define a model with the MultiModel mode and specify the S3 location of the model artifacts.
- **Deploy Endpoint:** Use SageMaker console or Boto3 SDK to deploy the endpoint with the defined model.

Adding and Removing Models from MultiModel Endpoints

Add Models: Upload new model artifacts to the S3 location without updating or stopping the endpoint.

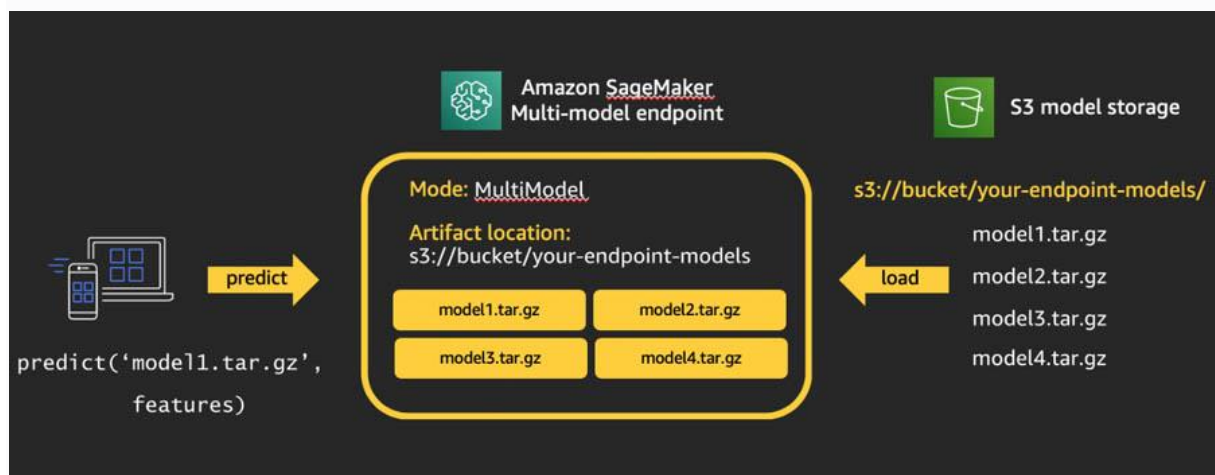
Invocation: First invocation of a new model may take longer due to loading from S3.

Model Update: Use unique names for new versions to avoid invoking old models.

Remove Models: Stop invocations and delete model artifacts from S3.

Inference Pipelines

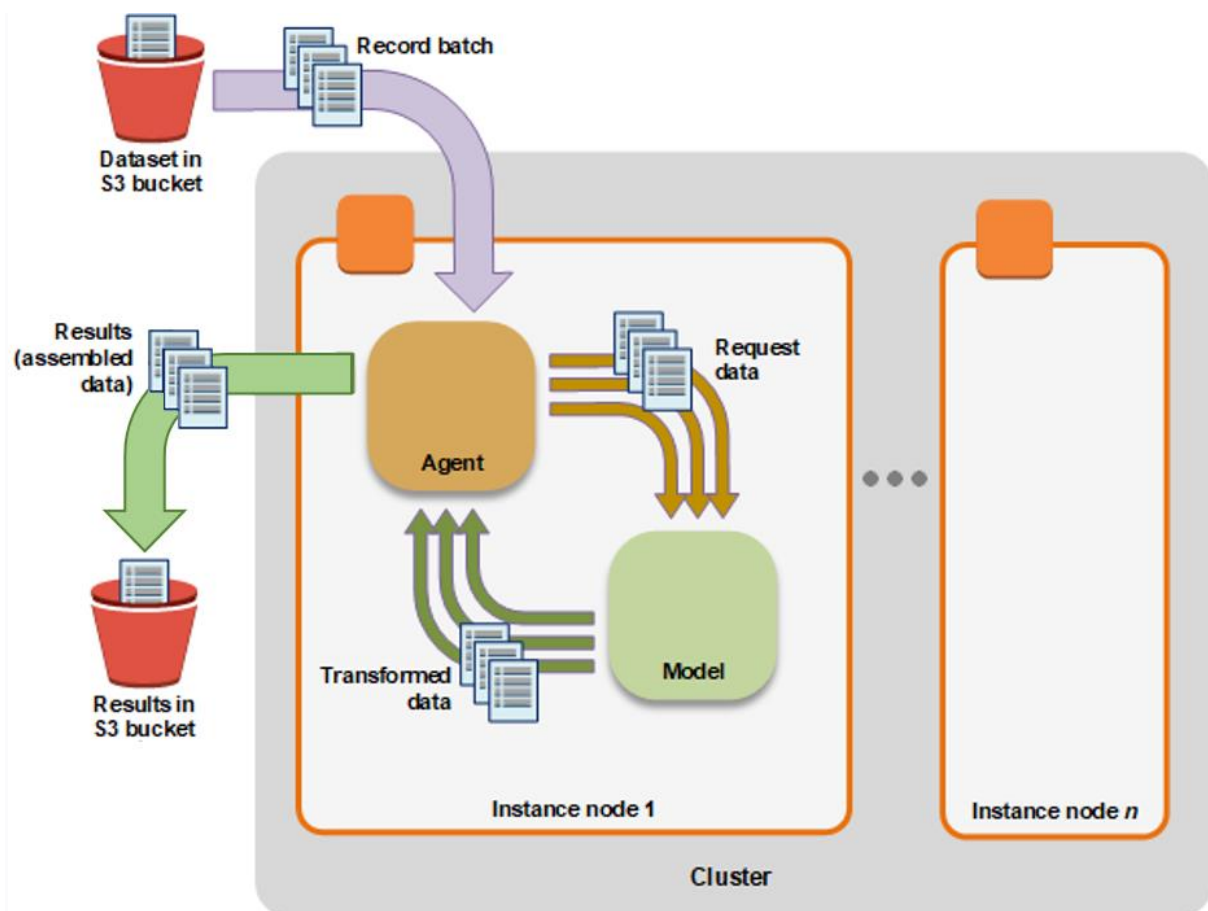
An **inference pipeline** is an Amazon SageMaker model that is composed of a linear sequence of two to fifteen containers that process requests for inferences on data. You use an inference pipeline to define and deploy any combination of pre-trained SageMaker built-in algorithms and your own custom algorithms packaged in Docker containers. You can use an inference pipeline to combine preprocessing, predictions, and post-processing data science tasks. Inference pipelines are fully managed.



You can add SageMaker Spark ML Serving and Scikit-learn containers that reuse the data transformers developed for training models. The entire assembled inference pipeline can be considered as a SageMaker model that you can use to make either real-time predictions or to process batch transforms directly without any external preprocessing.

AWS Batch Transform

To get inferences for an entire dataset, use batch transform. With batch transform, you create a batch transform job using a trained model and the dataset, which must be stored in Amazon S3. Amazon SageMaker saves the inferences in an S3 bucket that you specify when you create the batch transform job.



You can use Amazon SageMaker Batch Transform to exclude attributes before running predictions. You can also join the prediction results with partial or entire input data attributes when using data that is in CSV, text, or JSON format. This eliminates the need for any additional pre-processing or post-processing and accelerates the overall ML process.

Sagemaker Feature Store

One of the challenging parts of machine learning (ML) is feature engineering, the process of transforming data to create features for ML. Features are processed data signals used for training ML models and for deployed models to make accurate predictions. Data scientists and ML engineers can spend up to 60-70% of their time on feature engineering. It's also typical to have this work repeated by different teams within an organization who use the same data to build ML models for different solutions, further increasing effort levels for feature engineering. Moreover, it's important that the generated features should be available for both training and real-time inference use cases, to ensure consistency between model training and inference serving.

A purpose-built feature store for ML is needed to ensure both high-quality ML predictions with a consistent set of features, and cost reduction by eliminating duplicate feature engineering effort and storage overhead. Consistent features are needed between different parts of an organization, and between training and inference for any given ML model. There is also a need for feature stores to meet the high performance, scale, and availability requirements to serve features in near-real time for inferences. Because of this, organizations are often forced to do the heavy lifting of building and maintaining feature store systems, which can become expensive and difficult to maintain.

At AWS we are constantly listening to our customers and building solutions and services that delight them. We heard from many customers about the pain their data science and data engineering teams face when managing features, and used those inputs to build the [Amazon SageMaker Feature Store](#), which was launched at re:Invent on December 1, 2020. Amazon SageMaker Feature Store is a fully managed, purpose-built repository to securely store, update, retrieve, and share ML features.

Although there is a lot to unpack in terms of the capabilities that SageMaker Feature Store brings to the table, in this post, we focus on key capabilities for data ingestion, data access, and security and access control.

Overview of SageMaker Feature Store

As a purpose-built feature store for ML, SageMaker Feature Store is designed to enable you to do the following:

Securely store and serve features for real-time and batch applications – SageMaker Feature Store enables you to use ML to make near-real time decisions by enabling feature vector retrievals with low latencies.

Accelerate model development by sharing and reusing features – Feature engineering is a long and tedious process that often gets repeated by multiple teams within an organization working on the same data. SageMaker Feature Store enables data scientists to spend less time on data preparation and feature computation, and more time on ML innovation, by letting them discover and reuse existing engineered features across the organization.

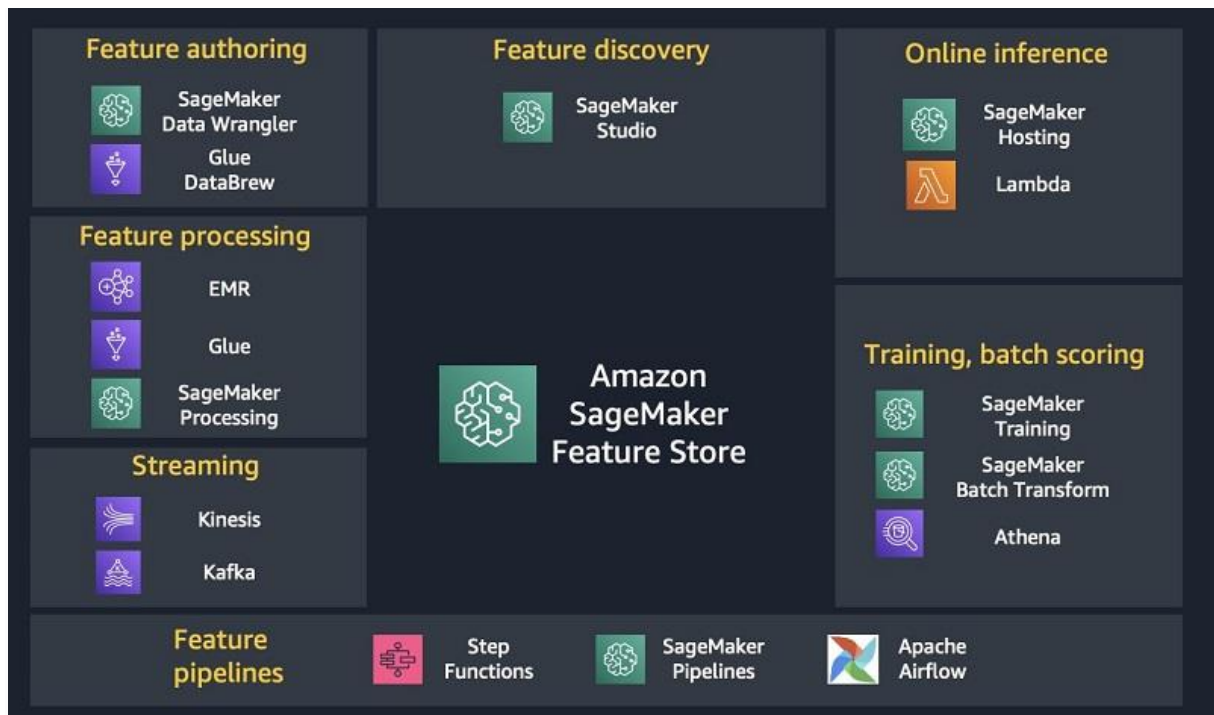
Provide historical data access – Features are used for training purposes, and a good feature store should provide easy and quick access to historical feature values to recreate training datasets at a given point in time in the past. Amazon SageMaker Feature Store enables this with support for time-travel queries—querying data at a point in time—which enables you to re-create features at specific points of time in the past.

Reduce training-serving skew – Data science teams often struggle with training-serving skew caused by data discrepancy between model training and inference serving, which can cause models to perform worse than expected in production. SageMaker Feature Store reduces training-serving skew by keeping feature consistency between training and inference.

Enable data encryption and access control – As with other data assets, ML feature data security is paramount in all organizations. At AWS, security and operational performance are our top priorities, and SageMaker Feature Store provides a suite of capabilities for enterprise-grade data security and access control, including encryption at rest and in transit, and role-based access control using [AWS Identity and Access Management](#) (IAM).

Guarantee a robust service level – Managed feature store production use cases need service-level guarantees, ensuring that you get the desired performance and availability, and you can rely on expert help should something go wrong. SageMaker Feature Store is backed by AWS's unmatched reliability, scale and operational efficiency.

SageMaker Feature Store is designed to play a central role in ML architectures, helping you streamline the ML lifecycle, and integrating seamlessly with many other services. For example, you can use tools like [AWS Glue DataBrew](#) and [SageMaker Data Wrangler](#) for feature authoring. You can use [Amazon EMR](#), [AWS Glue](#), and [SageMaker Processing](#) in conjunction with SageMaker Feature Store for performing feature transformation tasks. You can use a suite of tools, including [SageMaker Pipelines](#), [AWS Step Functions](#), or [Apache AirFlow](#) for scheduling and orchestrating feature pipelines to automate feature engineering process flow. When you have features in the feature store, you can pull them with low latency from the online store to feed models hosted with services like [SageMaker Hosting](#). You can use existing tools like [Amazon Athena](#), [Presto](#), [Spark](#), and EMR to extract datasets from the offline store for use with [SageMaker Training](#) and batch transforms. Lastly, you can use [Amazon Kinesis](#), [Apache Kafka](#), and [AWS Lambda](#) for streaming feature engineering and ingestion. The following diagram illustrates some of the services that can be integrated with SageMaker Feature Store.



Before we go into more detail, we briefly introduce some SageMaker Feature Store concepts:

Feature group – a logical grouping of ML features

Record – a set of values for features in a feature group

Online store – the low latency, high availability store that enables real-time lookup of records

Offline store – the store that manages historical data in your [Amazon Simple Storage Service](#) (Amazon S3) bucket, and is used for exploration and model training use cases

Sagemaker Clarify

This topic describes how to understand fairness and model explainability and how to explain and detect bias using Amazon SageMaker Clarify. You can configure an SageMaker Clarify processing job to compute bias metrics and feature attributions and generate reports for model explainability. SageMaker Clarify processing jobs are implemented using a specialized SageMaker Clarify container image. The following instructions show you how to configure, run, and troubleshoot a SageMaker Clarify processing job and how to configure an analysis.

What is fairness and model explainability for machine learning predictions?

Machine learning (ML) models are helping make decisions in domains including financial services, healthcare, education, and human resources. Policymakers, regulators, and advocates have raised awareness about the ethical and policy challenges posed by ML and data-driven systems. Amazon SageMaker Clarify can help you understand why your ML model made a specific prediction and whether this bias impacts this prediction during training or inference. SageMaker Clarify also provides tools that can help you build less biased and more understandable machine learning models. SageMaker Clarify can also generate model governance reports that you can provide to risk and compliance teams and external regulators. With SageMaker Clarify, you can do the following:

- Detect bias in and help explain your model predictions.

- Identify types of bias in pre-training data.

- Identify types of bias in post-training data that can emerge during training or when your model is in production.

Data Engineering

S3 Lifecycle

To manage your objects so that they are stored cost-effectively throughout their lifecycle, configure their **Amazon S3 Lifecycle**. An S3 Lifecycle configuration is a set of rules that define actions that Amazon S3 applies to a group of objects.

The screenshot shows the Amazon S3 console interface for configuring lifecycle rules. The sidebar on the left includes links for Buckets, Access points, Batch Operations, Access analyzer for S3, Account settings for Block Public Access, and a Feature spotlight. The main content area is titled 'Transition current versions of objects between storage classes'. It features a table for 'Storage class transitions' with columns for 'Storage class transitions' and 'Days after object creation'. The table shows two transitions: Standard-IA at 180 days and Glacier at 730 days. There are 'Remove transition' and 'Add transition' buttons. Below the table is a warning box about transitioning small objects to Glacier or Glacier Deep Archive, with a checkbox for acknowledgment. At the bottom, there is a 'Timeline summary' section showing 'Current version actions' and 'Previous version actions'. An orange callout bubble labeled 'Object transition timeline' points to the 'Timeline summary' section. The bottom of the console shows 'Tutorials Dojo' and 'Cancel'/'Create rule' buttons.

Storage class transitions	Days after object creation	
Standard-IA	180	Remove transition
Glacier	730	Remove transition

Timeline summary

Current version actions	Previous version actions
Day 0 Objects uploaded	Day 0 Objects become noncurrent
↓	
Day 180 Objects transition to Standard-IA	
↓	
Day 730 Objects transition to Glacier	

Define S3 Lifecycle configuration rules for objects that have a well-defined lifecycle. For example:

- If you upload periodic logs to a bucket, your application might need them for a week or a month. After that, you might want to delete them.
- Some documents are frequently accessed for a limited period of time. After that, they are infrequently accessed. At some point, you might not need real-time access to them, but your organization or regulations might require you to archive them for a specific period. After that, you can delete them.
- You might upload some types of data to Amazon S3 primarily for archival purposes. For example, you might archive digital media, financial and healthcare records, raw genomic sequence data, long-term database backups, and data that must be retained for regulatory compliance.

With S3 Lifecycle configuration rules, you can tell Amazon S3 to transition objects to less expensive storage classes, or archive or delete them. Amazon S3 Glacier Deep Archive is the lowest-cost storage class and supports long-term retention and digital preservation for data that may be accessed once or twice a year. With this storage class, you can retrieve your data within 12 hours.

You should use Amazon S3 Standard for the processed data since compliance requires them to be stored in a highly available environment with immediate access. With the S3 lifecycle policy, you can eventually move them to Amazon S3 Glacier to save costs. Additionally, since unprocessed data are rarely accessed, using Amazon S3 Glacier Archive to store them is the best option as it also supports retrieving data within the required timeframe.

AWS Kinesis Data Firehose

Amazon Kinesis Data Firehose buffers incoming streaming data to a certain size or for a certain period of time before delivering it to destinations. You can configure the buffer size and buffer interval while creating your delivery stream.

Kinesis Data Firehose - Create delivery stream

Step 1: Name and source
Step 2: Process records
Step 3: Choose a destination
Step 4: Configure settings
Step 5: Review

Configure settings

Configure buffer, compression, logging, and IAM role settings for your delivery stream. [Learn more](#)

S3 buffer conditions

Kinesis Data Firehose buffers incoming records before delivering them to your S3 bucket. Record delivery will be triggered once either of these conditions has been satisfied. [Learn more](#)

Buffer size
 MIB
Enter a buffer size between 1 - 128 MiB.

Buffer interval
 seconds
Enter a buffer interval between 60 - 900 seconds.

Buffer size is in MBs and ranges from 1MB to 128MB for Amazon S3 destination and 1MB to 100MB for Amazon OpenSearch Service destination. Buffer interval is in seconds and ranges from 60 seconds to 900 seconds. Please note that in circumstances where data delivery to destination is falling behind data writing to a delivery stream, Firehose raises buffer size dynamically to catch up and make sure that all data is delivered to the destination.

For Kinesis Data Streams, You can't set the Data Retention Period to 180 seconds because the minimum value for it is 24 hours

Kinesis Data Firehose Transformation

Uses lambda to do the transformations. By default the lambda timeout value is 3 seconds which will not be enough to do the transformations so it will fail writes to the destination.

All transformed records from Lambda must contain the following parameters, or Amazon Data Firehose rejects them and treats that as a data transformation failure.

For Kinesis Data Streams and Direct PUT:

recordId

The record ID is passed from Amazon Data Firehose to Lambda during the invocation. The transformed record must contain the same record ID. Any mismatch between the ID of the original record and the ID of the transformed record is treated as a data transformation failure.

result

The status of the data transformation of the record. The possible values are: Ok (the record was transformed successfully), Dropped (the record was dropped intentionally by your processing logic), and ProcessingFailed (the record could not be transformed). If a record has a status

of 0k or Dropped, Amazon Data Firehose considers it successfully processed. Otherwise, Amazon Data Firehose considers it unsuccessfully processed.

data

The transformed data payload, after base64-encoding.

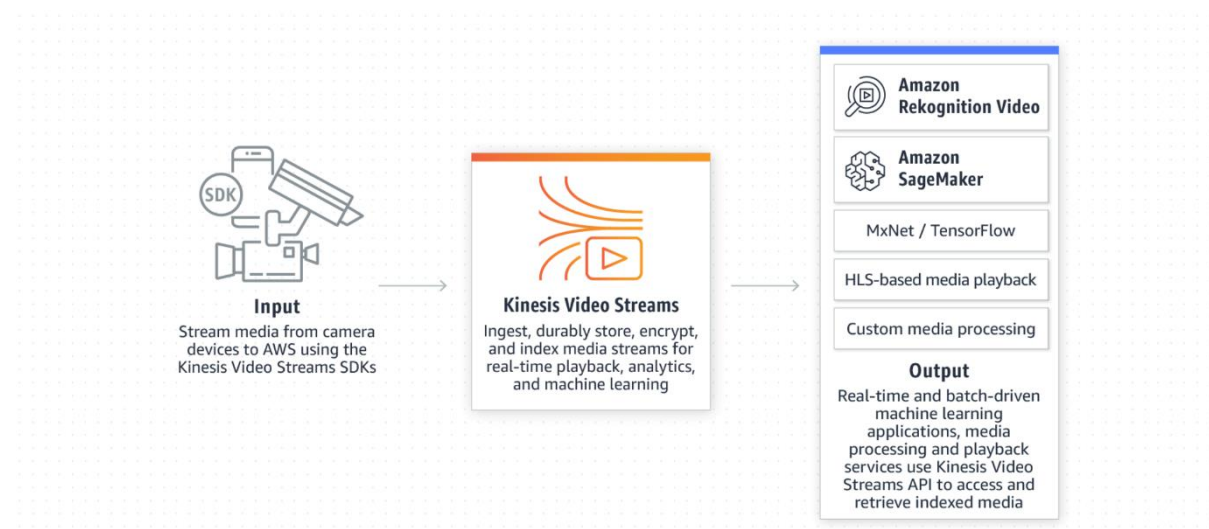
Following is a sample Lambda result output:

```
{
  "recordId": "<recordId from the Lambda input>",
  "result": "0k",
  "data": "<Base64 encoded Transformed data>"
}
```

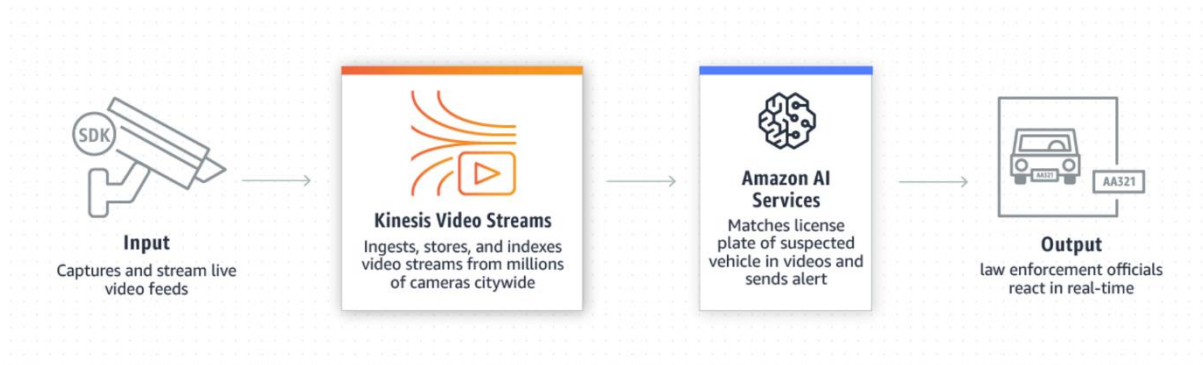
AWS Kinesis Video Streams

Amazon Kinesis Video Streams makes it easy to securely stream video from connected devices to AWS for analytics, machine learning (ML), playback, and other processing. Kinesis Video Streams automatically provisions and elastically scales all the infrastructure needed to ingest streaming video data from millions of devices. It durably stores, encrypts, and indexes video data in your streams, and allows you to access your data through easy-to-use APIs.

Kinesis Video Streams enables you to playback video for live and on-demand viewing, and quickly build applications that take advantage of computer vision and video analytics through integration with Amazon Rekognition Video, and libraries for ML frameworks such as Apache MxNet, TensorFlow, and OpenCV. Kinesis Video Streams also supports WebRTC, an open-source project that enables real-time media streaming and interaction between web browsers, mobile applications, and connected devices via simple APIs. Typical uses include video chat and peer-to-peer media streaming.



You can use Amazon Kinesis Video Streams to securely and cost-effectively ingest, store, and analyze this massive volume of video data to help solve traffic problems mentioned in the given use-case.



Streaming Data Transformation

AWS Kinesis

Here are key concepts from Amazon Kinesis Data Streams:

- **Kinesis Data Stream:** Comprises shards, each with a sequence of data records.
- **Data Record:** Contains a sequence number, partition key, and data blob.
- **Capacity Modes:** On-demand (auto scales) and provisioned (fixed shards).
- **Retention Period:** Default 24 hours, extendable up to 365 days.
- **Producers and Consumers:** Producers push data; consumers process data.
- **Shard:** A unit of capacity within a stream.
- **Partition Key:** Groups data by shard.
- **Sequence Number:** Unique per partition key within a shard.

Here are key points on service sizes and limits for Amazon Kinesis Data Streams:

- **Stream Limits:** No upper limit on the number of streams; 50 streams by default in on-demand mode.
- **Shard Limits:** No upper limit; default quota is 500 shards in key regions, 200 elsewhere.
- **Throughput:** On-demand mode scales up to 200 MB/s write and 400 MB/s read.
- **Data Payload:** Maximum size of 1 MB per record before encoding.
- **API Limits:** Vary per API, e.g., GetRecords retrieves up to 10 MB or 10,000 records per call.

To secure Amazon Kinesis Data Streams:

- **Shared Responsibility:** AWS handles infrastructure security; users manage data security and access controls.
- **Data Protection:** Encrypt data in transit and at rest.
- **IAM Policies:** Use AWS Identity and Access Management (IAM) to control access.
- **Monitoring and Auditing:** Use AWS CloudTrail and Amazon CloudWatch for logging and monitoring.
- **Compliance:** Follow AWS compliance programs and best practices.



Kinesis Data Streams

Data streaming service that supports massive scale and durability of data

Requires manual shard configuration and scaling

Use cases include:

- Data storage of streaming data is needed
- Replay of streaming data is a need
- Real-time processing is a requirement



Kinesis Data Firehose

Fully-managed data delivery solution integrated with multiple AWS services

Does not require manual configuration to scale

Use cases include:

- Data will be delivered into S3, Redshift, or Elasticsearch
- Being near real-time is sufficient
- Amount of streaming data is somewhat unpredictable



Kinesis Video Streams

Fully-managed video data delivery solution integrated with AWS services

Does not require manual configuration to scale

Use cases include:

- Delivery of video streams from home cameras for analysis in SageMaker
- Integrating video data with facial recognition in Amazon Rekognition



Kinesis Data Analytics

Fully-managed streaming data analytics service

Enables queries to be run against input data



Amazon EMR

Big data platform for processing large-scale data in the cloud

Supported frameworks include:

- Spark
- Hive
- HBase
- Flink
- Hudi
- Presto



**AWS
Glue**

Fully-managed ETL service

Includes Glue Crawler that discovers data and metadata within data sources

Crawled data is indexed in the AWS Glue Catalog

Supported data sources include:

- S3
- DynamoDB
- RDS
- Redshift



**Kinesis Data Firehose
Data Transform**

Provides the ability to invoke a Lambda function on incoming data

Does not require configuration for scaling

Supports an invocation time of 5 minutes and payload limit of 6 MB



Amazon Rekognition

Provides automated analysis of image and video data

Enables video analysis in batch or real-time

Includes text extraction, object recognition, and face/emotion detection

Use cases include:

- Enabling search across a large library of digital content
- Implementing a facial authentication system

Calculate shards needed for AWS Kinesis data streams

How many shards would a Kinesis Data Streams application need if the average record size is 500KB and 2 records per second are being written into this application that has 7 consumers?

$\text{number_of_shards} = \max(\text{incoming_write_bandwidth_in_KB}/1000, \text{outgoing_read_bandwidth_in_KB}/2000)$

where

$\text{incoming_write_bandwidth_in_KB} = \text{average_data_size_in_KB} \text{ multiplied by the number_of_records_per_seconds.} = 500 * 2 = 1000$

$\text{outgoing_read_bandwidth_in_KB} = \text{incoming_write_bandwidth_in_KB} \text{ multiplied by the number_of_consumers} = 1000 * 7 = 7000$

So, $\text{number_of_shards} = \max(1000/1000, 7000/2000) = \max(1, 3.5) = 4$

So, 4 shards are needed to address this use case.

AWS Glue ML Transforms

You can create machine learning transforms to cleanse your data using AWS Glue ML Transforms. You can call these transforms from your ETL script. The FindMatches transform enables you to identify duplicate or matching records in your dataset, even when the records do not have a common unique identifier and no fields match exactly.

Matching Records with AWS Lake Formation FindMatches

[PDF](#) | [Kindle](#) | [RSS](#)

AWS Lake Formation provides machine learning capabilities to create custom transforms to cleanse your data. There is currently one available transform named FindMatches. The FindMatches transform enables you to identify duplicate or matching records in your dataset, even when the records do not have a common unique identifier and no fields match exactly. This will not require writing any code or knowing how machine learning works. FindMatches can be useful in many different problems, such as:

- **Matching Customers:** Linking customer records across different customer databases, even when many customer fields do not match exactly across the databases (e.g. different name spelling, address differences, missing or inaccurate data, etc).
- **Matching Products:** Matching products in your catalog against other product sources, such as product catalog against a competitor's catalog, where entries are structured differently.
- **Improving Fraud Detection:** Identifying duplicate customer accounts, determining when a newly created account is (or might be) a match for a previously known fraudulent user.
- **Other Matching Problems:** Match addresses, movies, parts lists, etc etc. In general, if a human being could look at your database rows and determine that they were a match, there is a really good chance that the FindMatches transform can help you.

You can create these transforms when you create a job. The transform that you create is based on a source data store schema and example data that you label (we call this process "teaching" a transform). In this process we generate a file which you label and then upload back which the transform would learn from. After you teach your transform, you can call it from your Spark-based AWS Glue job (PySpark or Scala Spark) and use it in other scripts with a compatible source data store.

After the transform is created, it is stored in AWS Glue. On the AWS Glue console, you can manage the transforms that you create. On the AWS Glue **ML transforms** tab, you can edit and continue to teach your machine learning transform. For more information about managing transforms on the console, see [Working with Machine Learning Transforms on the AWS Glue Console](#).

Note

AWS Glue version 2.0 jobs do not support machine learning transforms. For more information, see [Running Spark ETL Jobs with Reduced Startup Times](#).

Types of Machine Learning Transforms

You can create machine learning transforms to cleanse your data. You can call these transforms from your ETL script. Your data passes from transform to transform in a data structure called a *DynamicFrame*, which is an extension to an Apache Spark SQL *DataFrame*. The *DynamicFrame* contains your data, and you reference its schema to process your data.

The following types of machine learning transforms are available:

Find matches

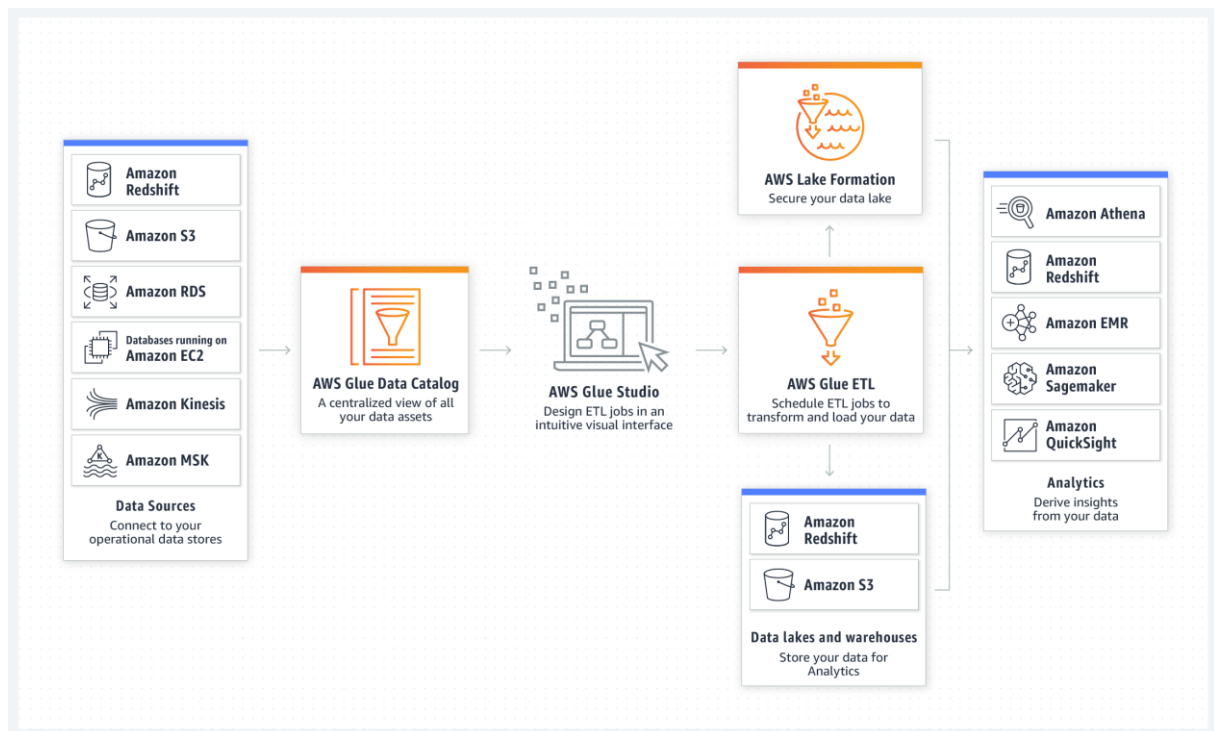
Finds duplicate records in the source data. You teach this machine learning transform by labeling example datasets to indicate which rows match. The machine learning transform learns which rows should be matches the more you teach it with example labeled data. Depending on how you configure the transform, the output is one of the following:

- A copy of the input table plus a `match_id` column filled in with values that indicate matching sets of records. The `match_id` column is an arbitrary identifier. Any records which have the same `match_id` have been identified as matching to each other. Records with different `match_id`'s do not match.
- A copy of the input table with duplicate rows removed. If multiple duplicates are found, then the record with the lowest primary key is kept.

AWS Glue ML Transforms job can perform deduplication in a serverless fashion and that's the correct choice for this use-case.

ETL Source input types not supported by AWS Glue

AWS Glue can run your ETL jobs as new data arrives. For example, you can use an AWS Lambda function to trigger your ETL jobs to run as soon as new data becomes available in Amazon S3. You can also register this new dataset in the AWS Glue Data Catalog as part of your ETL jobs.



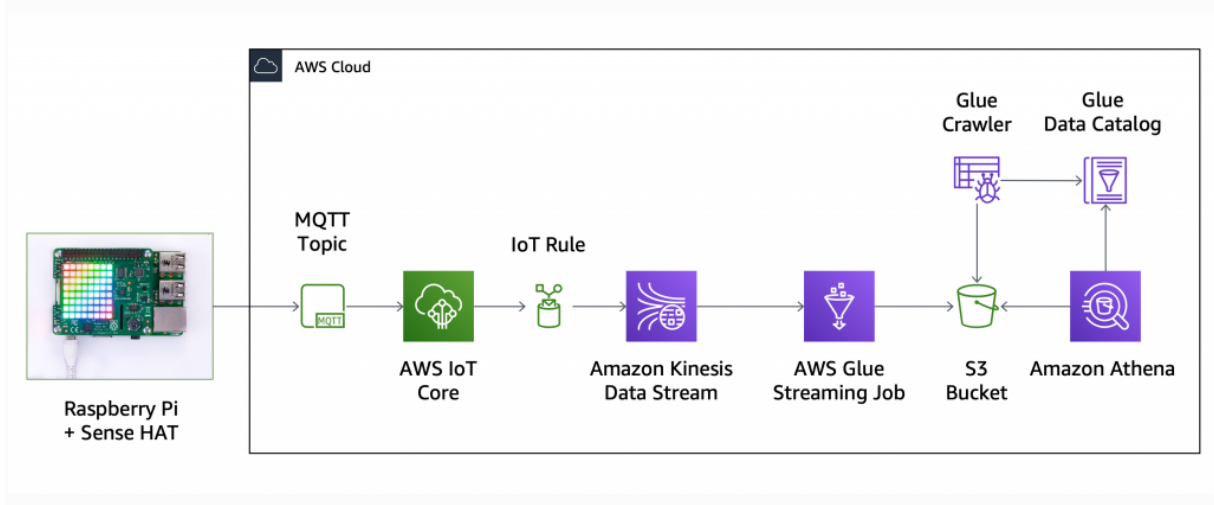
In AWS Glue, various PySpark and Scala methods and transforms specify the connection type using a `connectionType` parameter. AWS Glue does NOT support Timestream as the source input type

Glue Crawlers IAM

Create an IAM role named `AWSServiceRole-rolename` and attach the AWS managed policy, `AWSGlueServiceRole`, plus an inline policy that allows read access to: `s3://yourbucketname'`

Consume data using Kinesis Data streams and ETL using Glue

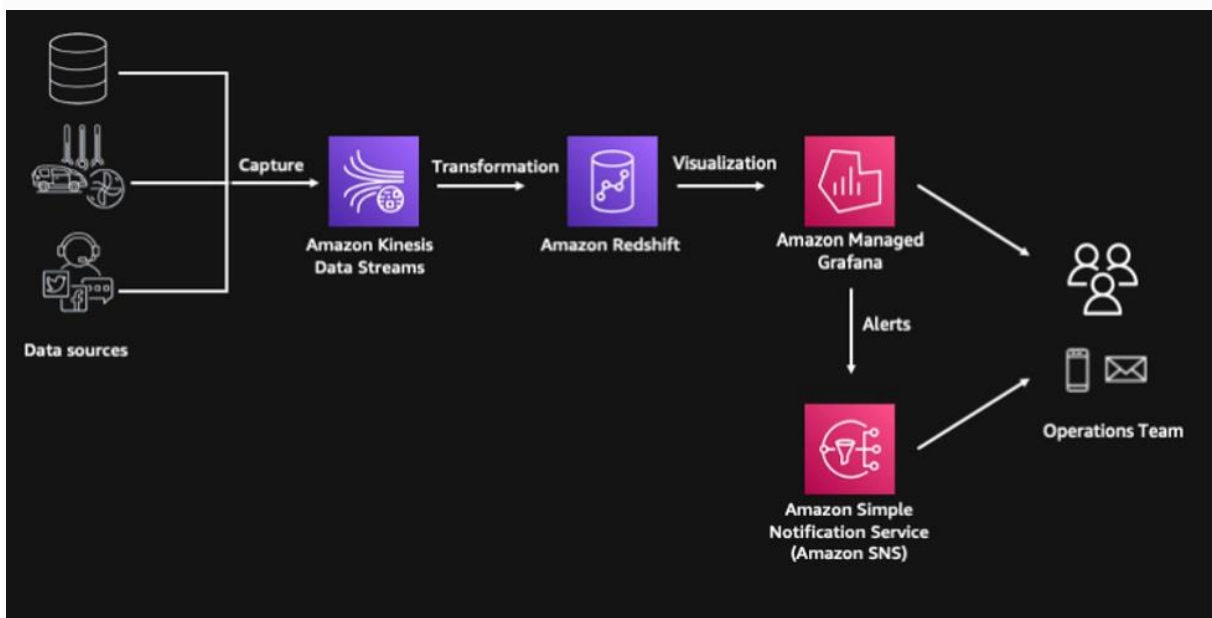
You can create streaming extract, transform, and load (ETL) jobs that run continuously, consume data from streaming sources like Amazon Kinesis Data Streams, Apache Kafka, and Amazon Managed Streaming for Apache Kafka (Amazon MSK). The jobs cleanse and transform the data, and then load the results into Amazon S3 data lakes or JDBC data stores.



As you process streaming data in a Glue job, you have access to the full capabilities of Spark Structured Streaming to implement data transformations, such as aggregating, partitioning, and formatting as well as joining with other data sets to enrich or cleanse the data for easier analysis. For example, you can access an external system to identify fraud in real-time or use machine learning algorithms to classify data or detect anomalies and outliers.

Better use Kinesis Data Streams with Redshift streaming ingestion instead of Kinesis Data Firehose

The Amazon Redshift Streaming ingestion feature makes it easier to access and analyze data coming from real-time data sources. It simplifies the streaming architecture by providing native integration between Amazon Redshift and the streaming engines in AWS, which are Amazon Kinesis Data Streams and Amazon Managed Streaming for Apache Kafka (Amazon MSK). Streaming data sources like system logs, social media feeds, and IoT streams can continue to push events to the streaming engines, and Amazon Redshift simply becomes just another consumer.



Before, loading data from a stream into Amazon Redshift included several steps. These included connecting the stream to Amazon Kinesis Data Firehose and waiting for Kinesis Data Firehose to stage the data in Amazon S3, using various-sized batches at varying-length buffer intervals. After this, Kinesis Data Firehose initiated a COPY command to load the data from Amazon S3 to a table in Redshift.

Amazon Redshift Streaming ingestion eliminates all of these extra steps, resulting in faster performance and improved latency.

AWS Batch

AWS Batch is a set of batch management capabilities that dynamically provisions the optimal quantity and type of compute resources (e.g., CPU or memory optimized compute resources) based on the volume and specific resource requirements of the batch jobs submitted.

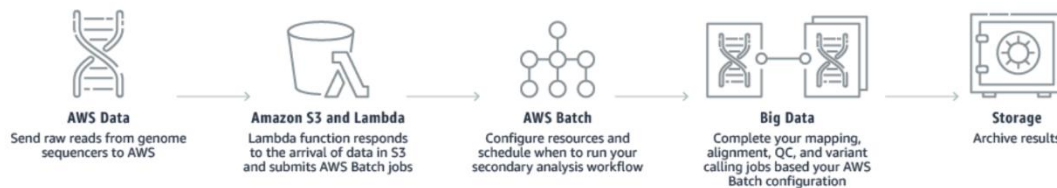
With AWS Batch, there is no need to install and manage batch computing software or server clusters, allowing you to instead focus on analyzing results and solving problems. AWS Batch plans, schedules,

and executes your batch computing workloads using Amazon EC2 (available with Spot Instances) and AWS compute resources with AWS Fargate or Fargate Spot.

AWS Batch can be used to configure and schedule resources for the given use case.

DNA sequencing

After bioinformaticians complete their primary analysis of a genomic sequence to produce the raw files, they can use AWS Batch to complete their secondary analysis. With AWS Batch, customers can simplify and automate the assembly of the raw DNA reads into a complete genomic sequence by comparing the multiple overlapping reads and the reference sequence, as well as potentially reduce data errors caused by incorrect alignment between the reference and the sample.



AWS EMR

AWS EMR (Elastic MapReduce) is a cloud service offered by Amazon Web Services that makes it easier for companies to process vast amounts of data quickly. Think of EMR like a powerful computer that can quickly solve large puzzles by breaking them down into smaller, manageable pieces, solving each piece, and then putting the puzzle back together.

How Does AWS EMR Work?

EMR uses a technology called Hadoop, which is popular for processing big data. Here's a simple way to understand how it operates:

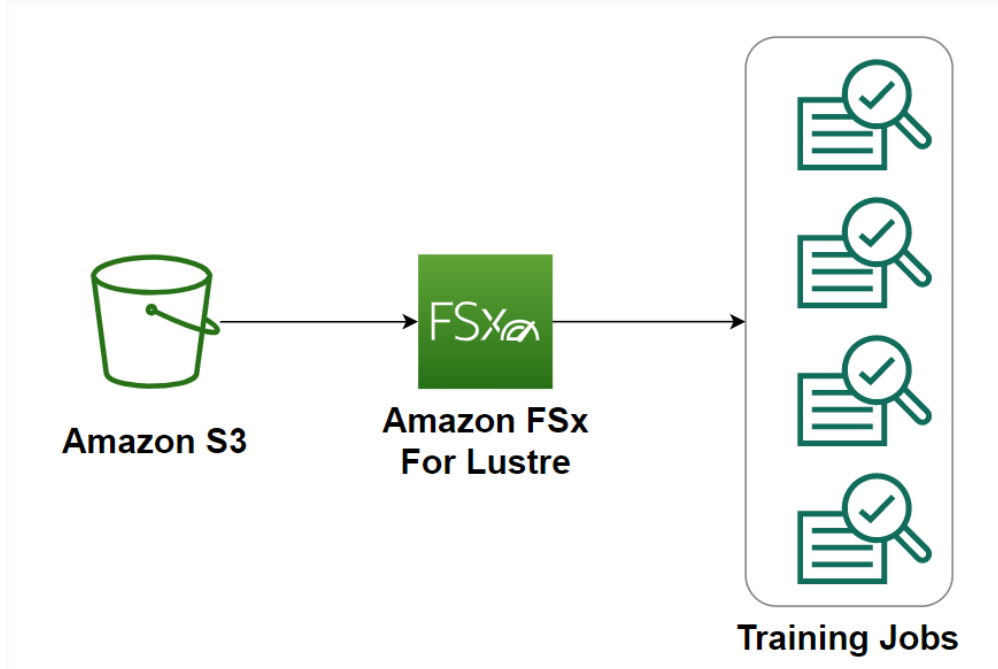
1. **Dividing the Task:** EMR takes a big data task and divides it into smaller pieces. This is similar to how you might break up a large project among several team members in an office.
2. **Processing in Parallel:** Each piece of the task is processed simultaneously on different servers. This parallel processing makes handling large datasets much more efficient than if one server were trying to do everything alone.
3. **Reassembling the Data:** Once all the pieces are processed, EMR combines the results to form a complete answer or dataset. This is like gathering all the individual contributions from team members and assembling them into a final report.

Key Features of AWS EMR

- **Scalable:** You can start with just a few servers and scale up to thousands, which helps handle increasing data or computing needs without any hassle.
- **Cost-Effective:** You pay only for the resources you use, and you can use spot instances (unused compute capacity in AWS at reduced prices) to save on costs.
- **Flexible:** EMR supports multiple big data frameworks besides Hadoop, such as Apache Spark, HBase, Presto, and Flink, allowing you to choose the best tool for your specific needs.

Amazon FSx for Lustre

Training machine learning models requires providing the training datasets to the training job. When using Amazon S3 as the training data source in File input mode, all training data have to be downloaded from Amazon S3 to the EBS volumes attached to the training instances at the start of the training job. A distributed file system such as Amazon FSx for Lustre or EFS can speed up machine learning training by eliminating the need for this download step.

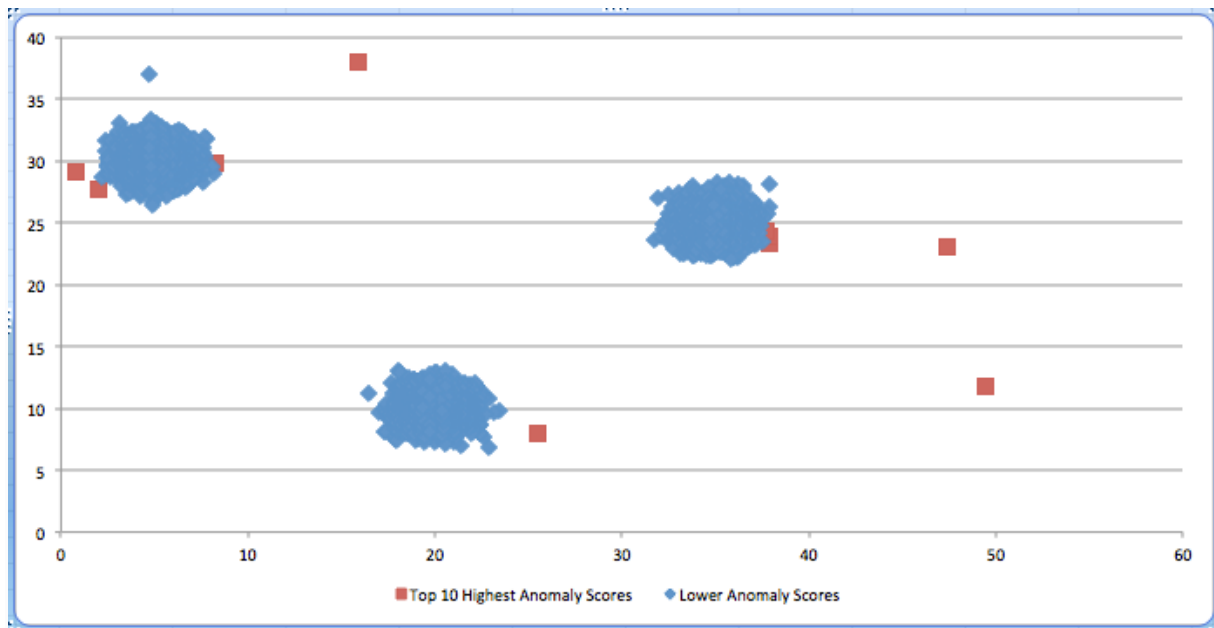


If your training data is already in Amazon S3, and your needs do not dictate faster training times for your training jobs, you can get started with Amazon SageMaker with no need for data movement. However, if you need faster startup and training times, we recommend that you take advantage of the Amazon FSx for Lustre file system, which is natively integrated with Amazon S3.

Amazon FSx for Lustre speeds up your training jobs by serving your Amazon S3 data to Amazon SageMaker at high speeds. The first time you run a training job, Amazon FSx for Lustre automatically copies data from Amazon S3 and makes it available to Amazon SageMaker. Additionally, the same Amazon FSx file system can be used for subsequent iterations of training jobs on Amazon SageMaker, preventing repeated downloads of common Amazon S3 objects. Because of this, Amazon FSx has the most benefit to training jobs that have training sets in Amazon S3 and in workflows where training jobs must be run several times using different training algorithms or parameters to see which gives the best result.

Amazon Managed Service For Apache Flink

Amazon Managed Service for Apache Flink allows you to process and analyze streaming data using standard SQL. The service enables you to quickly author and run powerful SQL code against streaming sources to perform time-series analytics, feed real-time dashboards, and create real-time metrics.



The **Amazon Managed Service for Apache Flink RANDOM CUT FOREST** function detects anomalies in your data stream. A record is an anomaly if it is distant from other records. The algorithm starts developing the machine learning model using current records in the stream when you start the application. The algorithm does not use older records in the stream for machine learning, nor does it use statistics from previous executions of the application.

Amazon Kendra

Amazon Kendra is a highly accurate and easy-to-use enterprise search service that's powered by machine learning (ML). It allows developers to add search capabilities to their applications so their end users can discover information stored within the vast amount of content spread across their company. This includes data from manuals, research reports, FAQs, human resources (HR) documentation, and customer service guides, which may be found across various systems such as Amazon Simple Storage Service (S3), Microsoft SharePoint, Salesforce, ServiceNow, RDS databases, or Microsoft OneDrive. When you type a question, the service uses ML algorithms to understand the context and return the most relevant results, whether that means a precise answer or an entire document. For example, you can ask a question such as "How much is the cash reward on the corporate credit card?" and Amazon Kendra will map to the relevant documents and return a specific answer (such as "2%"). Kendra provides sample code so you can get started quickly and easily integrate highly accurate search into your new or existing applications.

MLOPS

Early Stopping

To optimize your SageMaker training job, you can stop the training job early (that a hyperparameter tuning job launches) when they are not improving significantly as measured by the objective metric. Stopping training jobs early can help reduce compute time and helps you avoid overfitting your model.

When you enable early stopping for a hyperparameter tuning job, SageMaker uses the following criteria:

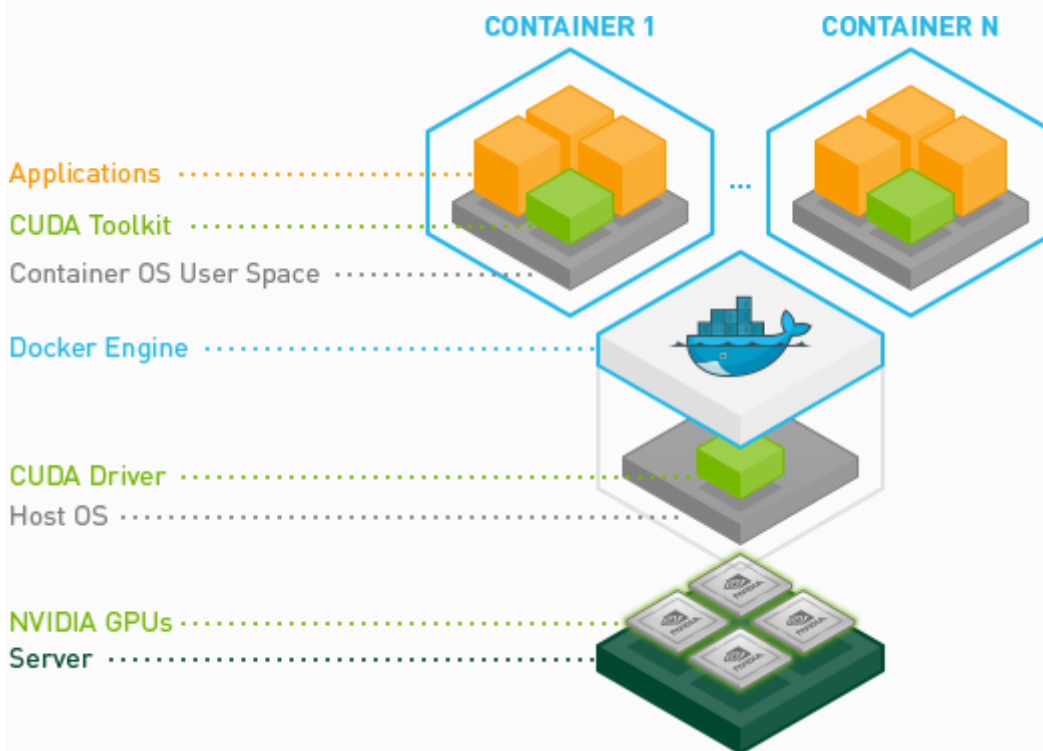
If the value of the objective metric for the current training job is worse (higher when minimizing or lower when maximizing the objective metric) than the median value of running averages of the objective metric for previous training jobs up to the same epoch, Amazon SageMaker stops the current training job.

Create a role to delegate access to your resources with the third-party AWS account

When third parties require access to your organization's AWS resources, you can use roles to delegate access to them. For example, a third party might provide a service for managing your AWS resources. With IAM roles, you can grant these third parties access to your AWS resources without sharing your AWS security credentials. Instead, the third party can access your AWS resources by assuming a role that you create in your AWS account.

Build GPU accelerated Docker Containers on Sagemaker

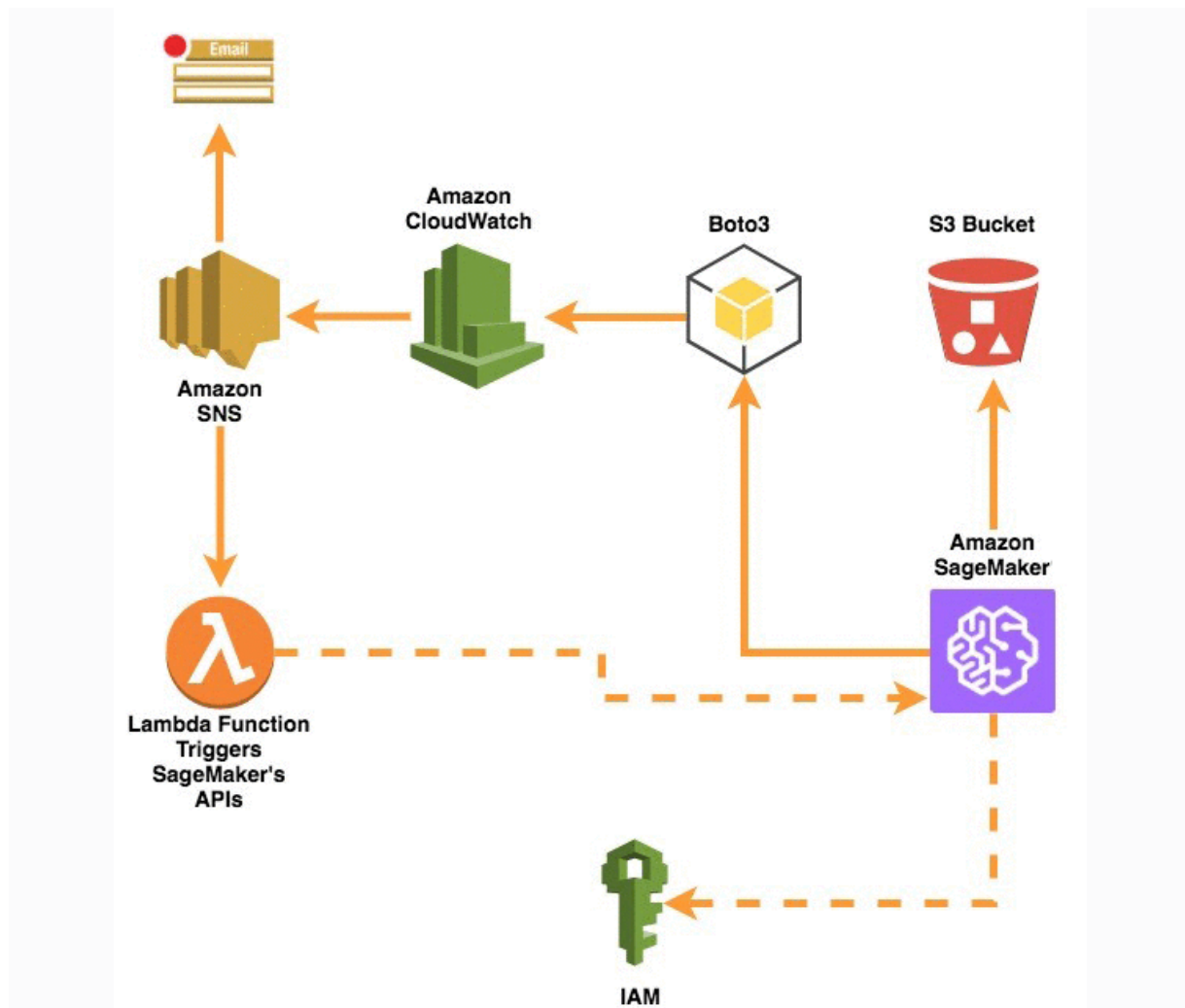
The NVIDIA Container Toolkit allows users to build and run GPU accelerated Docker containers. The toolkit includes a container runtime library and utilities to automatically configure containers to leverage NVIDIA GPUs.



In Amazon SageMaker, if you plan to use GPU devices for model training, **make sure that your containers are nvidia-docker compatible**. Only the CUDA toolkit should be included on containers; **don't bundle NVIDIA drivers with the image**.

Send Notifications when Model Overfit

You can use **Amazon CloudWatch API** operations to send the training metrics to CloudWatch, and create a dashboard of those metrics. Lastly, use Amazon Simple Notification Service (Amazon SNS) to send a notification when the model is overfitting.



You can optionally add a Lambda function to the architecture if you wish to run some mitigation plan (e.g., stop the training when overfitting occurs).

Encrypt Data before sending to Sagemaker Notebook Instance

To encrypt the machine learning (ML) storage volume that is attached to notebooks, processing jobs, training jobs, hyperparameter tuning jobs, batch transform jobs, and endpoints, you can pass an AWS KMS key to SageMaker. If you don't specify a KMS key, SageMaker encrypts storage volumes with a transient key and discards it immediately after encrypting the storage volume. For notebook instances, if you don't specify a KMS key, SageMaker encrypts both OS volumes and ML data volumes with a system-managed KMS key.

Notebook instance settings

Notebook instance name

tutorialsdojo

Maximum of 63 alphanumeric characters. Can include hyphens (-), but not spaces. Must be unique within your account in an AWS Region.

Notebook instance type

ml.t2.medium

► Additional configuration

Permissions and encryption

IAM role

Notebook instances require permissions to call other services including SageMaker and S3. Choose a role or let us create a role with the [AmazonSageMakerFullAccess](#) IAM policy attached.

Choose an IAM role

Root access - *optional*

- ☒ Enable - Give users root access to the notebook
- ☐ Disable - Don't give users root access to the notebook
- Lifecycle configurations always have root access

Encryption key - *optional*

Encrypt your notebook data. Choose an existing KMS key or enter a key's ARN.

arn:aws:kms:ap-southeast-1:123456789123:key/1395861a-a46d-46a3-b226-92...



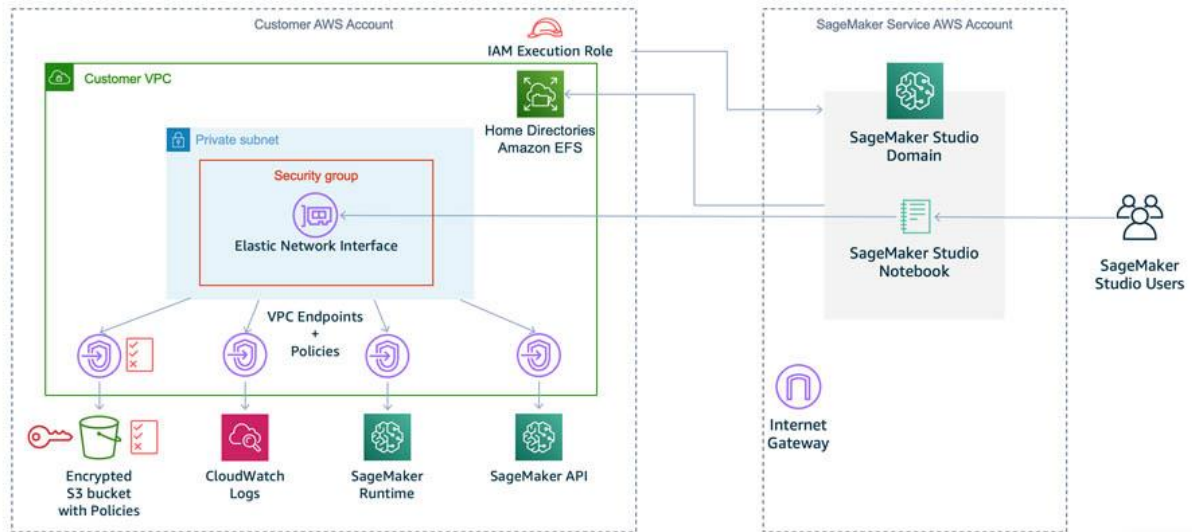
Sensitive data that needs to be encrypted with a KMS key for compliance reasons should be stored in the ML storage volume or in Amazon S3, both of which can be encrypted using a KMS key you specify. Additionally, you can use AWS Glue's sensitive data detection to redact Personal Identifiable Information (PII) and other sensitive data at both the column and cell levels.

VPC Endpoints without internet access

You can connect directly to the SageMaker API or to the SageMaker Runtime through an interface endpoint in your Virtual Private Cloud (VPC) instead of connecting over the internet. When you use a VPC interface endpoint, communication between your VPC and the SageMaker API or Runtime is conducted entirely and securely within the AWS network.

The SageMaker API and Runtime support Amazon Virtual Private Cloud (Amazon VPC) interface endpoints that are powered by AWS PrivateLink. Each VPC endpoint is represented by one or more Elastic Network Interfaces with private IP addresses in your VPC subnets.

Final deployment

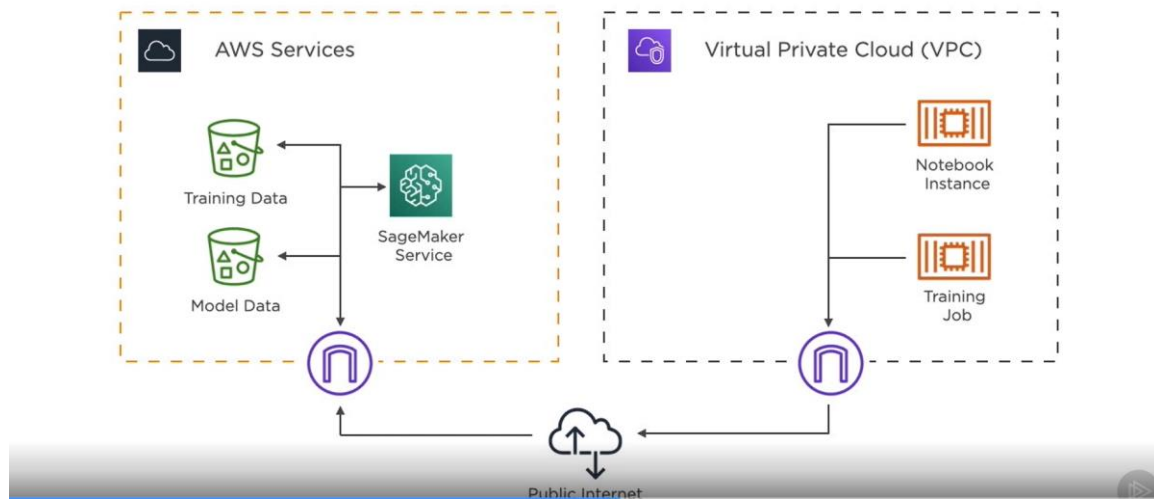


The VPC interface endpoint connects your VPC directly to the SageMaker API or Runtime without an internet gateway, NAT device, VPN connection, or AWS Direct Connect connection. The instances in your VPC don't need public IP addresses to communicate with the SageMaker API or Runtime.

VPC on Sagemaker

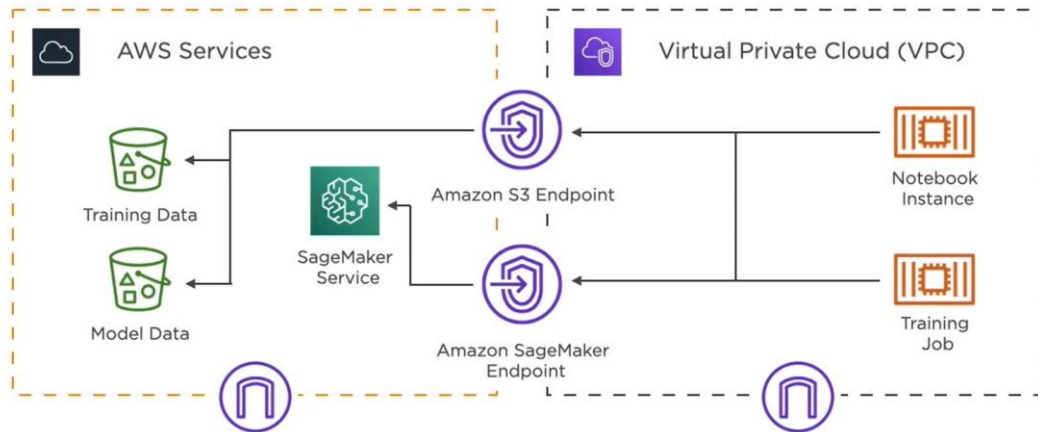
We generally send data over internet to access sagemaker services which is not secure

Using VPC Endpoints for Amazon S3



Instead by using VPC endpoints for S3 and Sagemaker, we can directly communicate with the sagemaker without sending anything over public internet

Using VPC Endpoints for Amazon S3



Securely remove PII data

There is a difference between encryption and tokenization. Encryption is the process of using an algorithm to transform plaintext into ciphertext. An algorithm and an encryption key are required to decrypt the original plaintext.

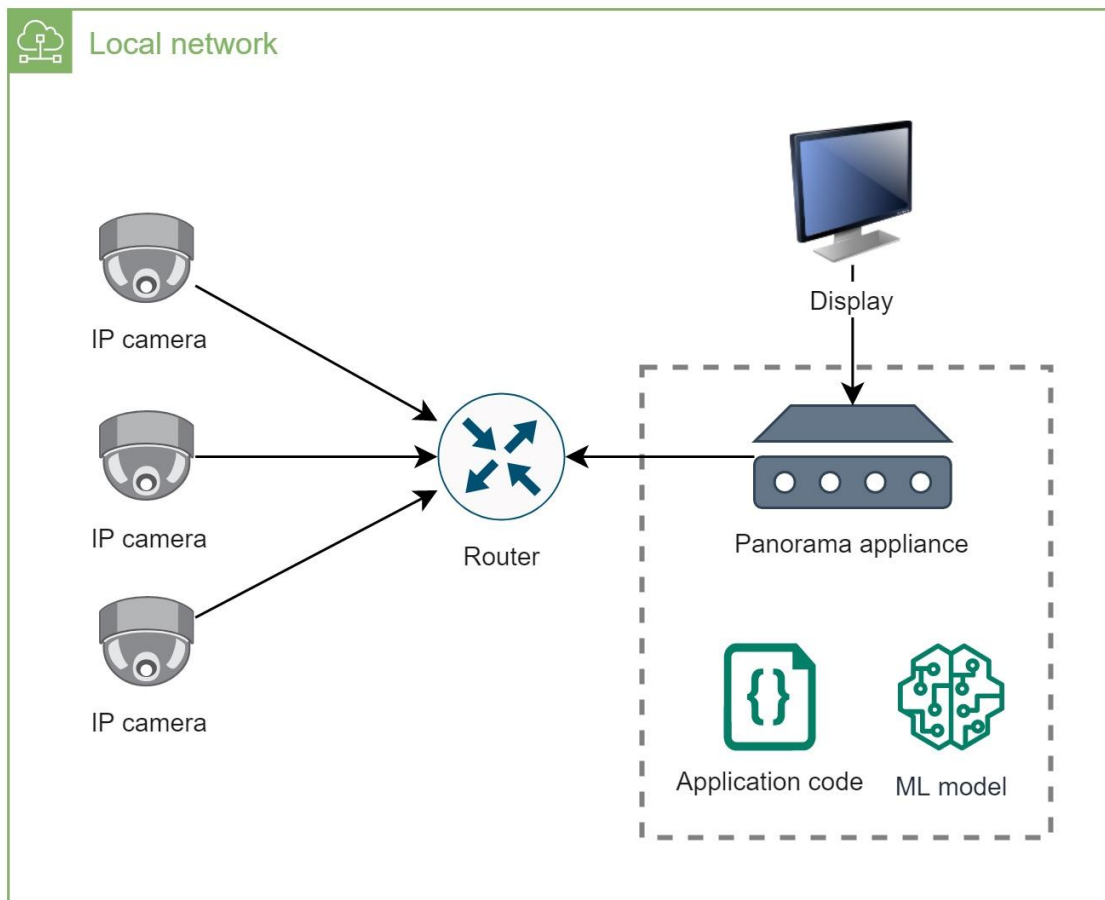
Tokenization is the process of transforming a piece of data into a random string of characters called a token. It does not have direct meaningful value in relation to the original data. Tokens serve as a reference to the original data, but cannot be used to derive that data.

Unlike encryption, tokenization does not use a mathematical process to transform the sensitive information into the token. Instead, tokenization uses a database, often called a token vault, which stores the relationship between the sensitive value and the token. The real data in the vault is then secured, often via encryption. The token value can be used in various applications as a substitute for the original data.

For example, for processing a recurring credit card payment, the token is submitted to the vault. The index is used to fetch the original data for use in the authorization process. Recently, tokens are also being used to secure other types of sensitive or personally identifiable information. This includes data like social security numbers (SSNs), telephone numbers, and email addresses.

AWS Panorama

AWS Panorama is a machine learning appliance and software development kit (SDK) that allows you to bring computer vision to on-premises cameras to make predictions locally with high accuracy and low latency. With the AWS Panorama Appliance, you can automate tasks that have traditionally required human inspection to improve visibility into potential issues. For example, you can use AWS Panorama Appliance to evaluate manufacturing quality, identify bottlenecks in industrial processes, and monitor workplace security even in environments with limited or no internet connectivity. The software development kit allows camera manufacturers to bring equivalent capabilities directly inside their IP cameras.



In the scenario, you can use AWS Panorama to leverage existing IP cameras instead of replacing them with those that have AI capabilities. We can run near-real-time inferences as well without worrying about bandwidth constraints since video feeds are sent and analyzed locally. Finally, we can use the SNS Publish API to send SMS alerts.

Detect Unauthorized Personnel using Amazon Kinesis Video Streams+ Rekognition Video

Amazon Kinesis Video Streams makes it easy to securely stream video from connected devices to AWS for analytics, machine learning (ML), playback, and other processing. Kinesis Video Streams automatically provisions and elastically scales all the infrastructure needed to ingest streaming video data from millions of devices.

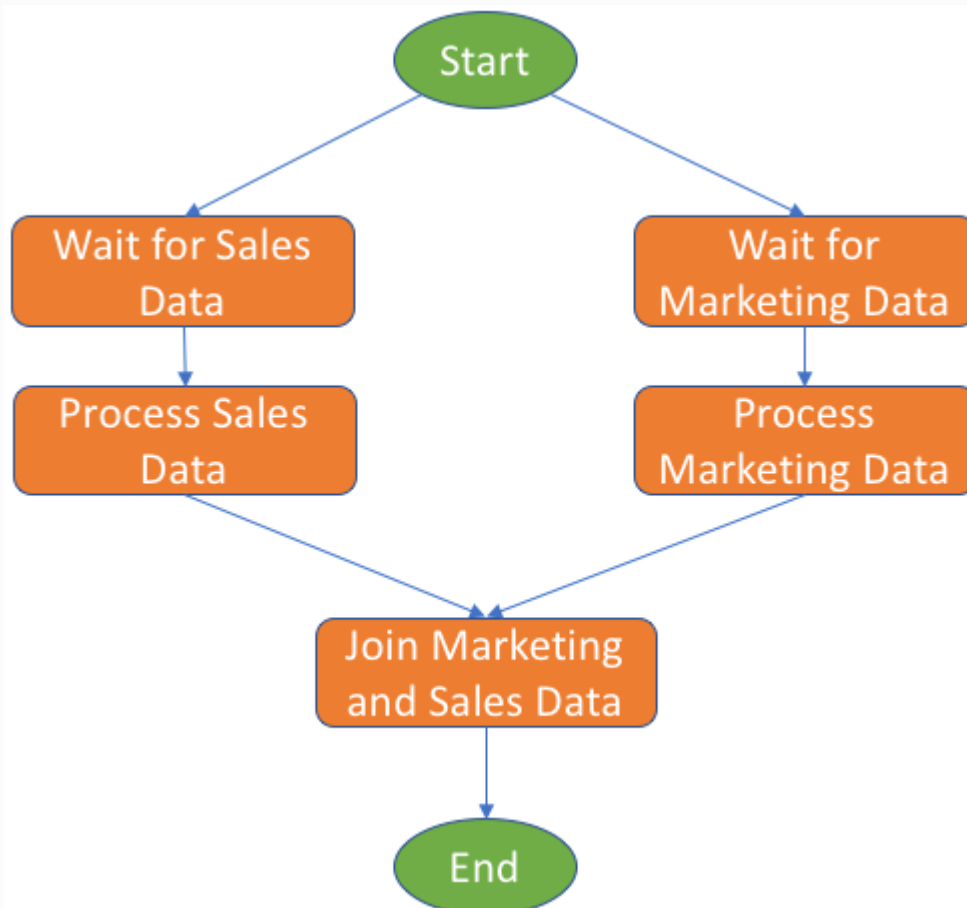
Amazon Rekognition Video can detect objects, scenes, faces, celebrities, text, and inappropriate content in videos. You can also search for faces appearing in a video using your own repository or collection of face images.



The example image above illustrates how we can combine these two services to create a face detection analysis.

AWS Step Functions

AWS Step Functions is a serverless function orchestrator that makes it easy to sequence AWS Lambda functions and multiple AWS services into business-critical applications.



Through its visual interface, you can create and run a series of checkpointed and event-driven workflows that maintain the application state. The output of one step acts as an input to the next. Each step in your application executes in order, as defined by your business logic.

In this scenario, you can efficiently orchestrate multiple ETL jobs using AWS Step Functions. Step Functions have different useful states that can be used in implementing the solution such as the Task State, Wait State, and Fail State. For example, we can define ETL jobs using the Task State, and wait

for other conditions to happen before proceeding on to the next state using Wait State. Handling errors and exemptions are also fairly easy to do using Fail State. Finally, we can use the combination of Amazon CloudWatch Alarms and Amazon SNS for notifications. Since AWS Lambda can be directly triggered through S3 Event Notifications, we can use it to start the AWS Step Functions workflow.

Sagemaker Data Wrangler

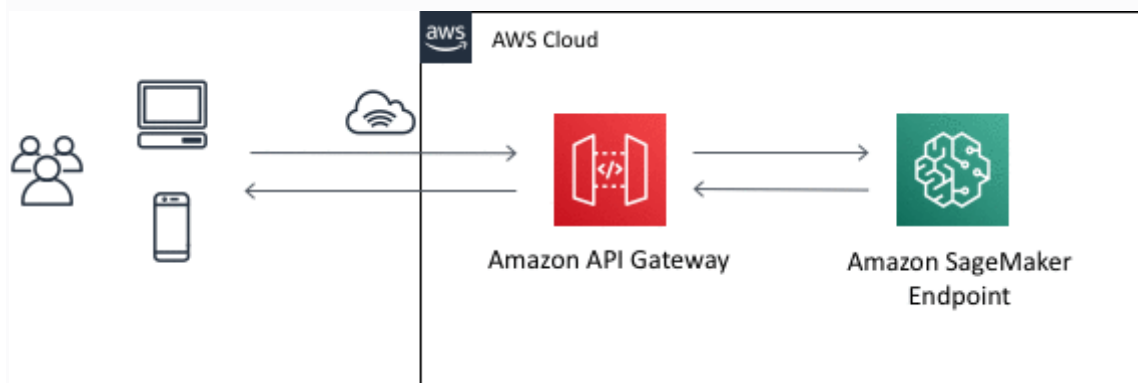
Amazon SageMaker Data Wrangler is a feature of Amazon SageMaker Studio that provides an end-to-end solution to import, prepare, transform, feature, and analyze data. You can integrate a Data Wrangler data preparation flow into your machine learning (ML) workflows to simplify and streamline data pre-processing and feature engineering using little to no coding. You can also add your own Python scripts and transformations to customize workflows. You can use Amazon SageMaker Data Wrangler to import data from the following *data sources*: Amazon Simple Storage Service (Amazon S3), Amazon Athena, Amazon Redshift, and Snowflake. The dataset that you import can include up to 1000 columns.

Additionally, Redshift credentials can be stored as ‘*secrets*’ in Secrets Manager. You can then leverage a library like the Data Wrangler SDK to retrieve the connection credentials at runtime. This eliminates the need to hardcode the credentials in your code and reduces the risk of accidental exposure.

Data Wrangler can be more cost-effective than using a Glue job because it eliminates the need for cluster management and scaling. Data Wrangler uses Redshift Data API to execute SQL statements and read data from Redshift, which does not require a persistent connection and thus reduces the load on your Redshift cluster. It also reduces the cost of running a Glue job, as they are charged per minute.

Deploying an endpoint – API Gateway

Amazon API Gateway is a fully managed service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs at any scale. API Gateway can be used to present an external-facing, single point of entry for Amazon SageMaker endpoints.

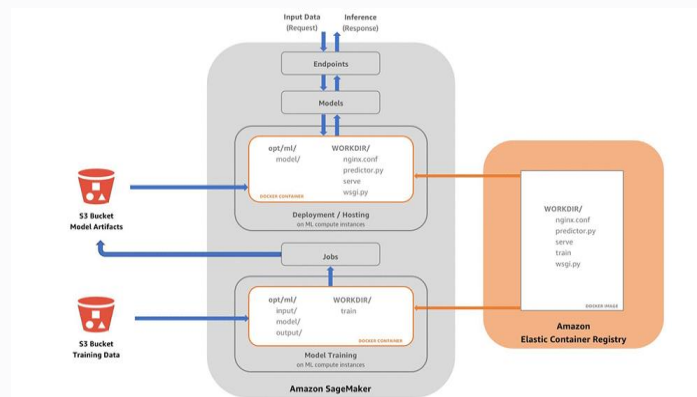


API Gateway can be used to front an Amazon SageMaker inference endpoint as (part of) a REST API, by making use of an API Gateway feature called mapping templates. This feature makes it possible for the REST API to be integrated directly with an Amazon SageMaker runtime endpoint, thereby avoiding the use of any intermediate compute resource (such as AWS Lambda or Amazon ECS containers) to invoke the endpoint. The result is a solution that is simpler, faster, and cheaper to run.

Sagemaker and Docker

Amazon SageMaker makes extensive use of Docker containers for build and runtime tasks. SageMaker provides prebuilt Docker images for its built-in algorithms and the supported deep

learning frameworks used for training and inference. Using containers, you can train machine learning algorithms and deploy models quickly and reliably at any scale.



Amazon SageMaker always uses Docker containers when running scripts, training algorithms, and deploying models. However, your level of engagement with containers depends on your use case:

Use a built-in SageMaker algorithm or framework

– You can use the built-in algorithms and frameworks without worrying about containers. You can train and deploy these algorithms from the SageMaker console, the AWS Command Line Interface (AWS CLI), a Python notebook, or the Amazon SageMaker Python SDK by specifying the algorithm or framework version when creating your Estimator.

Use prebuilt SageMaker container images.

– You can use the built-in algorithms and frameworks using Docker containers. SageMaker provides containers for its built-in algorithms and prebuilt Docker images for some of the most common machine learning frameworks, such as Apache MXNet, TensorFlow, PyTorch, and Chainer.

Extend a prebuilt SageMaker container image.

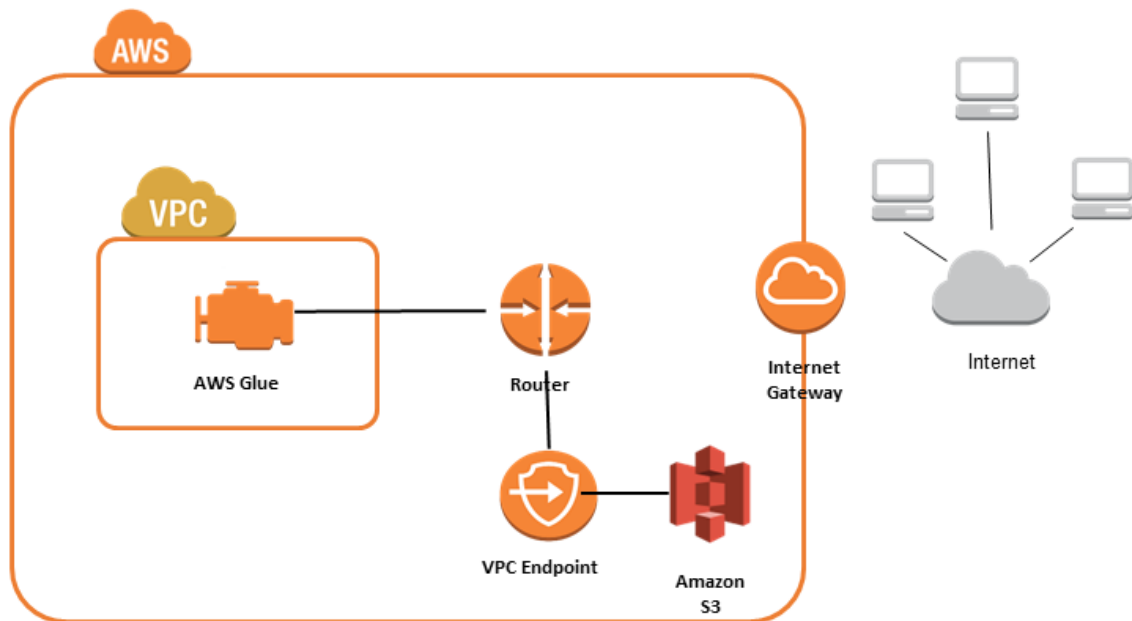
– If you would like to extend a prebuilt SageMaker algorithm or model Docker image, you can modify the SageMaker image to satisfy your needs.

Adapt an existing container image.

– If you would like to adapt a pre-existing container image to work with SageMaker, you need to modify the Docker container to enable either the SageMaker Training or Inference toolkit.

AWS Privatelink for VPC

A VPC endpoint enables private connections between your VPC and supported AWS services and VPC endpoint services powered by AWS PrivateLink. AWS PrivateLink is a technology that enables you to privately access services by using private IP addresses. Traffic between your VPC and the other service does not leave the Amazon network. A VPC endpoint does not require an Internet gateway, virtual private gateway, NAT device, VPN connection, or AWS Direct Connect connection. Instances in your VPC do not require public IP addresses to communicate with resources in the service.



VPC endpoints are virtual devices. They are horizontally scaled, redundant, and highly available VPC components. They allow communication between instances in your VPC and services without imposing availability risks.

Intercontainer Traffic Encryption

When performing distributed training, you can further protect data that is transmitted between instances. This can help you to comply with regulatory requirements.

Network

☒ **Enable network isolation**
Containers that run with network isolation can't make any outbound network calls. This is required for AWS ML Marketplace products.

VPC - optional
For better security, we recommend that you use a private VPC.

Default vpc-74508b1f (172.31.0.0/16) ▼

Subnet(s)
Choose a subnet in an availability zone supported by Amazon SageMaker.

subnet-0e8f4d65 (172.31.0.0/20) | us-east-2a ✕

Security group(s)
Configure your VPC to ensure that your training jobs have access to protected resources. [Allow Training Jobs to Access Resources in Your Private VPC](#)

sg-893b2cf2 (default) ✕

☒ **Enable inter-container traffic encryption**
Secure communication between instances for distributed training jobs. [Learn more.](#)

Enabling inter-container traffic encryption can increase training time, especially if you are using distributed deep learning algorithms. Enabling inter-container traffic encryption doesn't affect training jobs with a single compute instance. However, for training jobs with several compute instances, the effect on training time depends on the amount of communication between compute instances. For affected algorithms, adding this additional level of security also increases cost. The training time for most SageMaker built-in algorithms, such as XGBoost, DeepAR, and linear learner, typically isn't affected.

Monitoring Sagemaker

Monitoring is an important part of maintaining the reliability, availability, and performance of SageMaker. AWS provides the following monitoring tools to watch SageMaker, report when something is wrong, and take automatic actions when appropriate:

Amazon CloudWatch monitors your AWS resources and the applications that you run on AWS in real time. You can collect and track metrics, create customized dashboards, and set alarms that notify you or take actions when a specified metric reaches a threshold that you specify. You can further use Amazon CloudWatch Logs and CloudWatch Events to augment your monitoring processes.

CloudWatch Alarms just allows you to watch CloudWatch metrics and to receive notifications when the metrics fall outside of the levels that you configure

AWS CloudTrail captures API calls and related events made by or on behalf of your AWS account and delivers the log files to an Amazon S3 bucket that you specify. You can identify which users and accounts called AWS, the source IP address from which the calls were made, and when the calls occurred.

Amazon SageMaker is integrated with AWS CloudTrail, a service that provides a record of actions taken by a user, role, or an AWS service in SageMaker. *CloudTrail captures all API calls for SageMaker, with the exception of InvokeEndpoint, as events.* The calls captured include calls from the SageMaker console and code calls to the SageMaker API operations.



Amazon CloudWatch Metrics

Collects metrics from your SageMaker instances

- Training jobs
- Endpoints

Metrics are captured in a 1 minute frequency

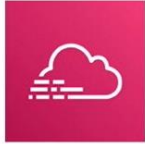
Stores metrics data for 15 months, with 2 weeks available in the console

Alarms can be configured on metrics

Invocation4XXErrors
Invocation5XXErrors
Invocations
InvocationsPerInstance
ModelLatency
OverheadLatency

CloudWatch SageMaker Endpoint Metrics

Dimensions provided on EndpointName, VariantName



Amazon CloudWatch Logs

Collects stdout and stderr for SageMaker instances

- Training jobs
- Endpoints
- Notebook instances
- Transform jobs

Log events are grouped into log streams and streams are grouped into log groups

Retention settings are customizable per log group



AWS CloudTrail

Captures actions from the SageMaker API from:

- Users
- Roles
- Other AWS Services

Once configured It collects all of this data into Amazon S3

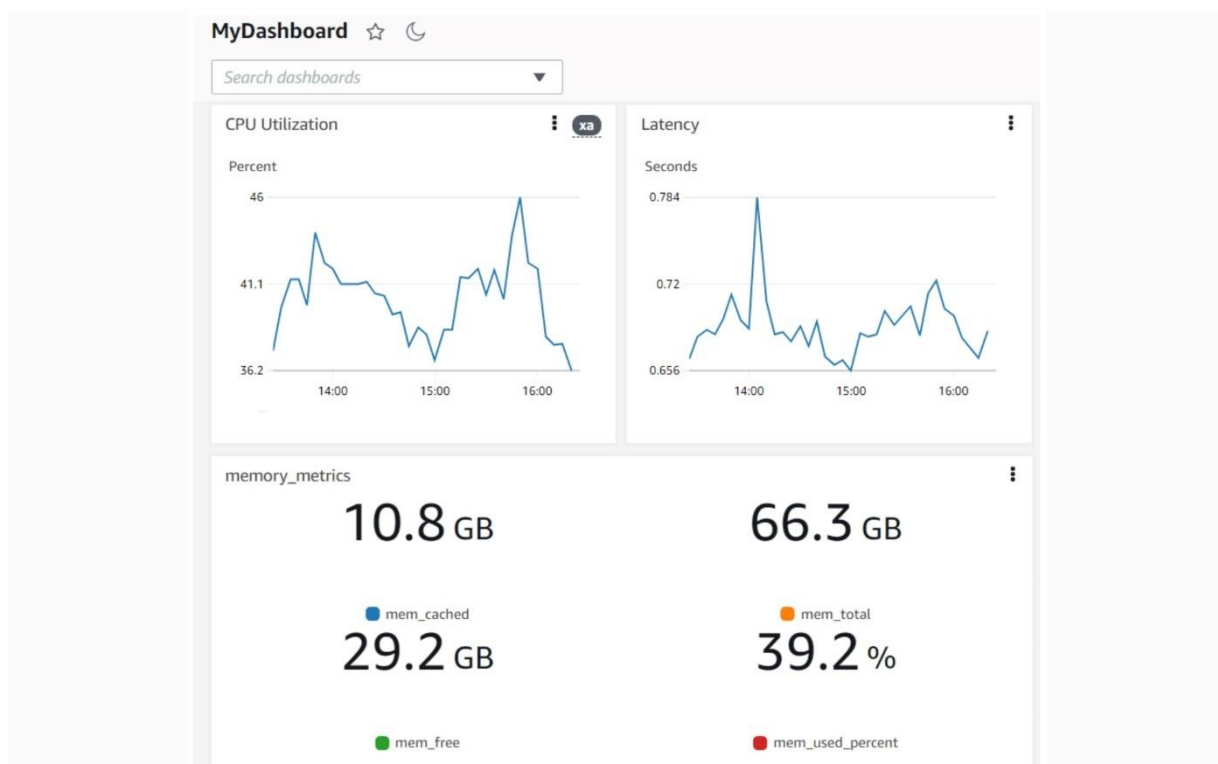
Since data is stored in Amazon S3 retention can be configured by the user

Provides an essential audit trail that should be in place for any AWS service

Data can be queried with AWS Athena

Dashboarding Using Cloudwatch

You can monitor Amazon SageMaker using Amazon CloudWatch, which collects raw data and processes it into readable, near-real-time metrics. These statistics are kept for 15 months so that you can access historical information and gain a better perspective on how your web application or service is performing.



You can then build customized dashboards for your CloudWatch metrics. Each dashboard can display multiple metrics and can be accessorized with text and images. You can build multiple dashboards if you'd like, each one focusing on providing a distinct view of your environment. You can even pull data from multiple regions into a single dashboard in order to create a global view.

Sagemaker MultiModel Endpoints Vs Inference Endpoints

Number of Models: Multimodel endpoints can host multiple models on a single endpoint, whereas inference endpoints are typically used to deploy a single model.

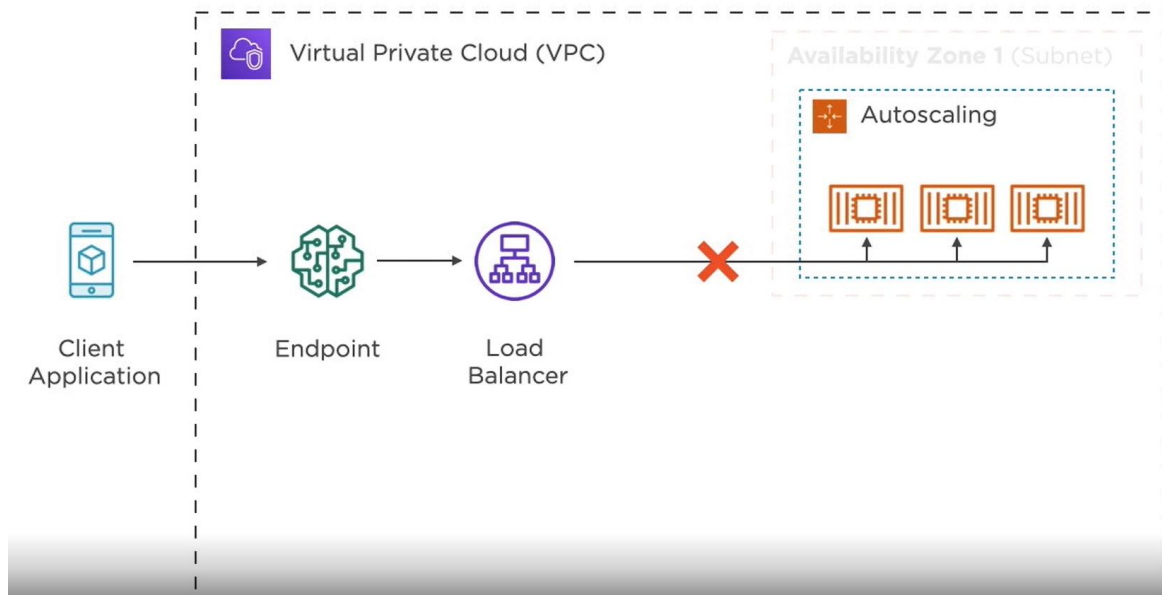
Model Loading: In multimodel endpoints, models are loaded and unloaded dynamically based on demand, which can introduce slight delays when a model is first called. In contrast, in inference endpoints, the model is loaded once and kept ready for immediate prediction at all times.

Use Case Suitability: Multimodel endpoints are ideal for scenarios with many models that have low or irregular prediction requests, making them cost-effective. Inference endpoints are better suited for high-demand, low-latency applications where a single model is used consistently.

Fault Tolerant Endpoint Configuration

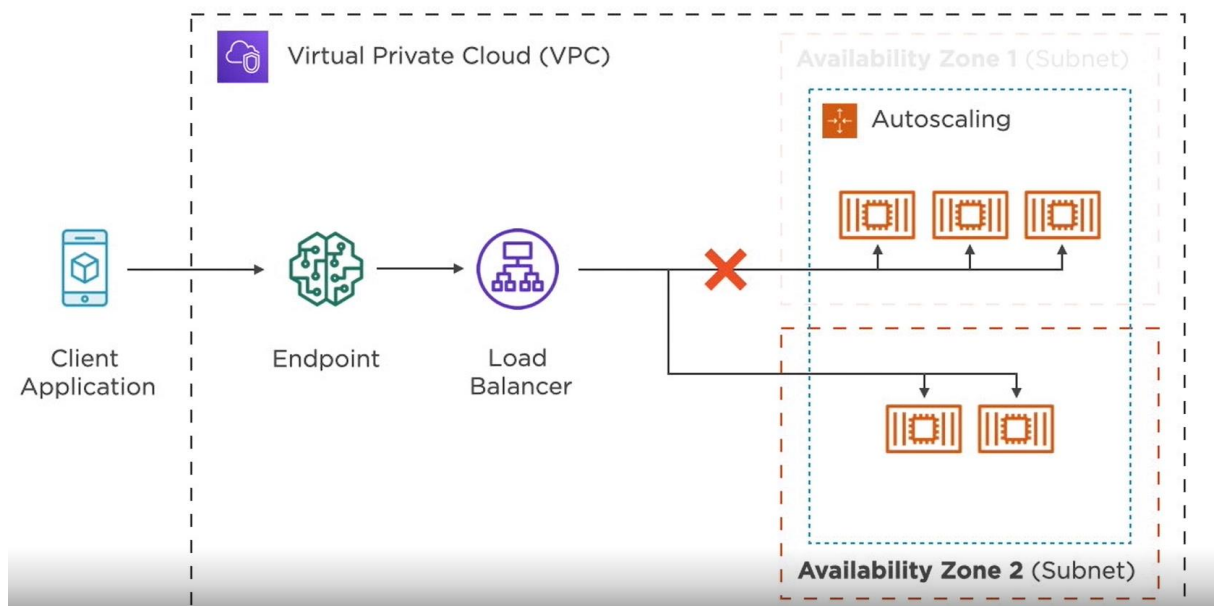
This is not fault tolerant as if we lose a particular availability zone then sagemaker can't route requests anywhere.

Fault Tolerant Endpoint Configuration



To tackle this issue, we can configure multiple subnets that belong to different availability zones and the load balancer will route requests to other AV if one is down

Fault Tolerant Endpoint Configuration



Fault Tolerant Endpoints

SageMaker is designed as a service for high-availability

To ensure security of data, configure SageMaker in a client-managed VPC

To support access during an instance outage, use more than 1 instance

To support access during an AZ outage, use more than 1 AZ

Vanishing Gradients

The vanishing gradient problem is a challenge that occurs during the training of deep neural networks, particularly those with many layers (deep networks). It primarily affects how the network learns and updates its parameters, which can lead to very slow training or even failure to effectively learn at all.

Here's a simple explanation:

- 1. What are gradients in neural networks?**
 - In neural networks, "gradients" are the derivatives (or slopes) of the loss function with respect to the network's weights. During training, these gradients tell us how to change the weights to reduce the error in predictions. The process of adjusting weights using gradients is known as "gradient descent."
- 2. How does the vanishing gradient problem occur?**
 - As the gradient descent algorithm updates the weights, it uses gradients calculated through a process called backpropagation, which moves from the output layer back to the input layer of the network. When neural networks are very deep, the gradients have to be propagated back through many layers by multiplying these gradients together repeatedly.
 - If these gradients are small numbers (less than 1), multiplying many of them together during backpropagation causes the resulting product to become very small, closer to zero. This is more pronounced the deeper the network is (i.e., the more layers it has).
- 3. What is the consequence of vanishing gradients?**
 - When the gradients approach zero, the weights in the earlier layers of the network barely change, meaning they don't learn effectively. Essentially, the network stops learning or learns extremely slowly, especially in the layers closer to the input. This makes it difficult to train deep networks effectively, as the necessary information and adjustments don't propagate back through all layers.
- 4. Typical Scenarios and Solutions:**
 - The vanishing gradient problem is particularly common with certain activation functions like the sigmoid or tanh functions, which squash the input values into a very small output range in a non-linear fashion. These functions can produce gradients that are very small.
 - To mitigate the vanishing gradient problem, alternative activation functions like ReLU (Rectified Linear Unit) or variants like Leaky ReLU and ELU (Exponential Linear

Unit) are used because they tend to produce larger gradients, reducing the risk of them diminishing as they are propagated back through the network. Additionally, techniques like careful initialization of weights, using batch normalization, or architectures designed to preserve gradient flow (like residual networks) can help manage or prevent this problem.

Understanding and addressing the vanishing gradient problem is crucial for the successful training of deep neural networks, ensuring that they can learn effectively across all layers.

Amazon SageMaker Studio

Amazon SageMaker Studio provides a comprehensive web-based IDE for machine learning workflows, including JupyterLab, Code Editor, and RStudio. It offers faster UI, integrated resource management, simplified model deployment, and enhanced JumpStart experience. Users can manage all SageMaker resources, access training jobs, and deploy models directly from Studio without needing the console. Studio Classic remains available for existing users, with a migration path to the updated experience.

Amazon SageMaker Studio offers shared spaces for real-time collaboration among users. These spaces support shared JupyterServer and JupyterLab applications, allowing users to access, edit, and run notebooks together. Shared spaces automatically tag resources for cost tracking and use the same VPC settings as the domain. Co-editing in notebooks is supported, with different cursors indicating users' activities. Note that some features like SageMaker Data Wrangler and EMR cross-account clusters are not supported.

Here are key points from the updated Amazon SageMaker Studio with JupyterLab:

- **Improved UI:** Enhanced user interface for better navigation and usability.
- **Integrated Resource Management:** Simplified management of compute resources.
- **Notebook Features:** Support for real-time collaboration and enhanced debugging.
- **Customization:** Ability to customize environments and kernels.
- **Deployment Integration:** Direct integration for model deployment and monitoring.

AWS SageMaker Studio is an integrated development environment (IDE) designed to provide data scientists and developers with all the tools needed to build, train, and deploy machine learning models quickly and efficiently on AWS. SageMaker Studio streamlines various machine learning tasks and enhances productivity through a unified user interface that supports numerous features and integrations. Here are some key features, including the use of SQL cells and connecting to EMR clusters:

1. SQL Cells

What It Is: SageMaker Studio supports SQL cells within Jupyter notebooks, which allows users to write and execute SQL queries directly within their notebooks.

Benefits:

- **Integrated Data Queries:** Users can interact directly with databases to pull, analyze, and visualize data without leaving the SageMaker environment. This is particularly useful for data exploration, cleaning, and preparation.
- **Seamless Integration:** This feature integrates well with AWS services such as Amazon Athena, Amazon Redshift, or any JDBC-compatible database service. It facilitates easy access

to data stored across different AWS data stores, enhancing data accessibility and manipulation.

2. Connecting to EMR Clusters

What It Is: SageMaker Studio can connect to Amazon EMR clusters, allowing data scientists to easily run big data jobs directly from the Studio.

Benefits:

- **Scalable Data Processing:** By connecting to EMR, users can handle large-scale data processing tasks using Apache Spark, Hadoop, and other big data frameworks hosted on EMR.
- **Unified Workflow:** This capability enables a seamless workflow where users can develop machine learning models in SageMaker, process big data with EMR, and move between these environments without switching tools.

3. Autopilot

- **Purpose:** Automates the process of model building. It automatically preprocesses your data, selects the best algorithms, and tunes your model.
- **Benefits:** Ideal for those who may not have deep expertise in machine learning but want to build robust models efficiently.

4. Experiments

- **Purpose:** Manages, tracks, and compares machine learning experiments. It organizes trials, tracks parameters, and logs results.
- **Benefits:** Great for iterative model tuning and experimentation, helping to determine the most effective model configurations.

5. Debugger

- **Purpose:** Provides detailed insights into the training process of machine learning models, helping to diagnose and fix issues quickly.
- **Benefits:** Helps optimize models by providing visibility into model performance and operational issues.

6. Model Monitor

- **Purpose:** Continuously monitors the machine learning model's performance in production to detect deviations in model quality.
- **Benefits:** Ensures model reliability and accuracy over time, alerting users to potential issues before they affect outcomes.

7. Data Wrangler

- **Purpose:** Simplifies the process of data preparation by providing an easy-to-use graphical interface to clean, transform, and visualize data.
- **Benefits:** Reduces the time and coding required to prepare data for machine learning, making the data scientist's job easier and more efficient.

Conclusion

AWS SageMaker Studio provides a rich set of features that streamline the entire machine learning lifecycle, from data preparation to model training, deployment, and monitoring. Its support for SQL cells and connectivity with AWS EMR enhances its functionality, making it a powerful tool for both machine learning and big data tasks. These features are designed to improve productivity, enhance collaboration, and enable more effective model building and deployment within the AWS ecosystem.

AWS Sagemaker Experiments

AWS SageMaker Experiments is a feature within the AWS SageMaker platform designed to help manage, track, and compare machine learning experiments. This tool simplifies the process of organizing the numerous trials and errors that often accompany machine learning model development, making it easier to optimize models effectively.

What Does SageMaker Experiments Do?

Here's how SageMaker Experiments helps streamline the machine learning workflow:

1. **Organization:** It allows you to organize all aspects of your machine learning experiment into a structured hierarchy, which includes:
 - **Experiments:** Top-level entities that are collections of related trials.
 - **Trials:** Components of an experiment that represent individual experiments. Each trial is an attempt to train a model using a particular set of parameters and data.
 - **Trial Components:** The most granular elements, which log the input configuration, metadata, and metrics of each trial execution step, such as data preprocessing, model training, or model evaluation.
2. **Tracking:** As you run experiments, SageMaker Experiments automatically tracks:
 - **Input Parameters:** Data sources, algorithm parameters, and configuration settings.
 - **Outputs:** Model artifacts and output parameters.
 - **Metrics:** Performance metrics like accuracy, precision, recall, etc.
3. **Comparison and Evaluation:** It provides tools to compare different trials visually. This comparison makes it easier to understand the impact of changes to algorithms, parameters, and data and helps identify the best-performing models based on objective metrics.

Use Cases for SageMaker Experiments

- **Model Optimization:** Data scientists can run multiple variations of machine learning models to find the most effective settings. SageMaker Experiments tracks these variations systematically, allowing for easy comparisons based on performance metrics.
- **A/B Testing:** When developing a new model, you might want to test how slight changes in parameters affect the outcome. SageMaker Experiments can manage and document these tests, helping to ensure that decisions about model adjustments are data-driven.
- **Regulatory Compliance and Reporting:** In industries like finance or healthcare, documenting the development process of machine learning models is crucial for regulatory compliance. SageMaker Experiments provides detailed logs and histories of model training, which can be crucial for audit trails.

How It Simplifies ML Workflows

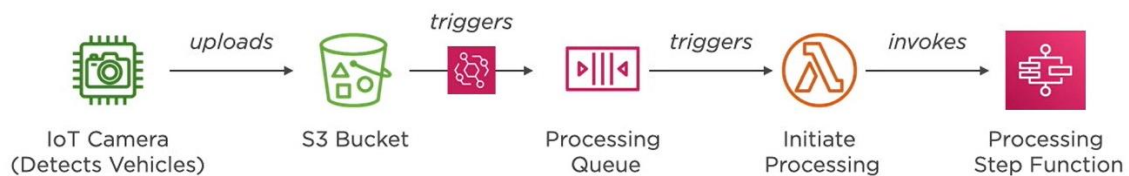
Imagine you're trying to improve a predictive model for customer churn. You might experiment with different types of machine learning algorithms, each with various hyperparameters, and use different subsets of features. AWS SageMaker Experiments allows you to:

- Set up each different approach as a separate "Trial" within an "Experiment."
- Automatically log all details about each trial, including the data used, the model parameters, the algorithm settings, and the resulting model performance.
- Easily compare these trials to see which model predicts customer churn most accurately.

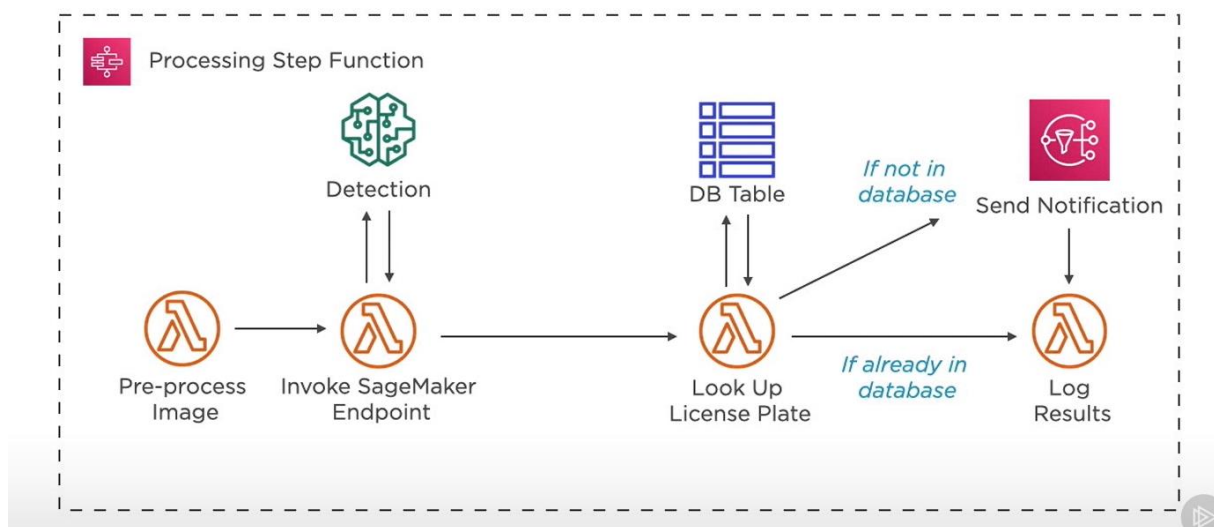
This structured approach helps you efficiently manage and evaluate numerous experiments, making it easier to derive insights and reach conclusions about which strategies work best.

Loosely coupled architecture

Loosely Coupled Architecture



Processing Step Function



Enabling High Availability

By using queues, any failure in launching the step function will be retried

EventBridge is used to leverage an event-driven workflow from S3

Step functions enable customizable retries if a specific step fails

SageMaker endpoint is already leveraging recommendations

Using IAM With Sagemaker

Amazon SageMaker supports authorization based on resource tags - You can attach tags to SageMaker resources or pass tags in a request to SageMaker. To control access based on tags, you provide tag information in the condition element of a policy using the sagemaker:ResourceTag/key-name, aws:RequestTag/key-name, or aws:TagKeys condition keys.

With IAM identity-based policies, you can specify allowed or denied actions and resources as well as the conditions under which actions are allowed or denied for Amazon.

SageMaker supports specific actions, resources, and condition keys. Administrators can use AWS JSON policies to specify who has access to what. That is, which principal can perform actions on what resources, and under what conditions. The Action element of a JSON policy describes the actions that you can use to allow or deny access in a policy.

SageMaker does not support resource-based policies

SageMaker doesn't support service-linked roles

Using IAM with SageMaker

You should define least privilege access for each user using IAM policies

Users should leverage multi-factor authentication (MFA)

Supports identity-based policies not resource-based policies

AWS provides sample policies depending on specific use cases

Using AWS IAM with Amazon SageMaker

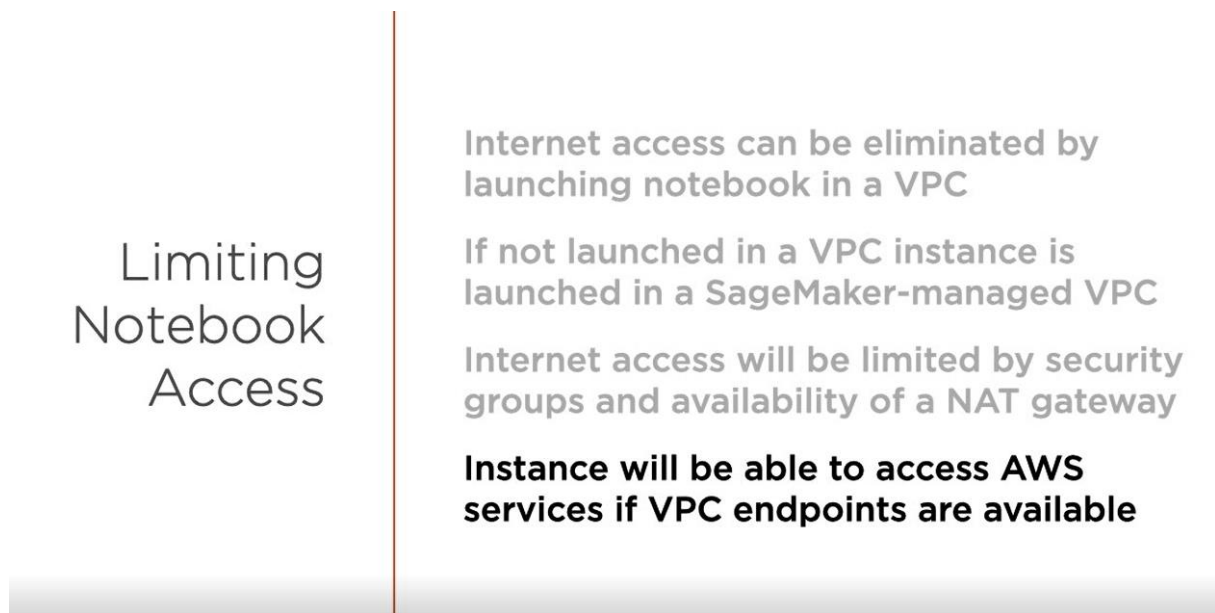
- **IAM Policies:** Control access to SageMaker resources using AWS Identity and Access Management (IAM) policies.
- **Roles:** Create IAM roles for SageMaker to access other AWS services.
- **Permissions:** Define granular permissions for users and roles to manage SageMaker resources.
- **Security Best Practices:** Follow IAM best practices to ensure secure access management.

IAM Policy Examples for SageMaker

- **Full Access:** Allows full access to all SageMaker resources.
- **Read-Only Access:** Grants read-only permissions to SageMaker resources.
- **Notebook Instances:** Policies for creating, stopping, and deleting notebook instances.
- **Training Jobs:** Permissions for starting, stopping, and managing training jobs.
- **Inference:** Policies for creating and managing endpoints and models.
- **Resource Tagging:** Manage tags for organizing and controlling access to resources.

Confused Deputy Prevention in SageMaker

- **Role ARNs:** Use specific AWS Identity and Access Management (IAM) role Amazon Resource Names (ARNs) to limit permissions.
- **Bucket Policies:** Apply bucket policies to restrict access to only the intended roles.
- **STS AssumeRole:** Use AWS Security Token Service (STS) AssumeRole to prevent confused deputy problems.
- **Least Privilege:** Follow the principle of least privilege to minimize access.



Evaluation metric vs Satisficing

Specificity

$$\text{Specificity} = (\text{True Negatives} / (\text{True Negatives} + \text{False Positives}))$$

If the model has a high specificity, it implies that all false positives (think of it as false alarms) have been weeded out. In other words, the *specificity* of a test refers to how well the test identifies those who have not indulged in substance abuse.

Here is an excellent reference article for a deep-dive on specificity:

<https://www.statisticshowto.datasciencecentral.com/sensitivity-vs-specificity-statistics/>

Incorrect options:

Accuracy - Accuracy measures the percentage of correct predictions.

Recall - Recall shows the percentage of actual positives among the total number of relevant instances (actual positives). In other words, how many positive items are selected?

F1-score - F1 score is a binary classification metric that considers both binary metrics precision and recall. It is the harmonic mean between precision and recall. The range is 0 to 1. A larger value indicates better predictive accuracy.

There are different metrics for assessing a classifier 's performance, they are called evaluation matrices. They can be categorized as matrices that satisfy and optimize. It is necessary to remember that these evaluation matrices must be evaluated on a training set, a production set or on the test set.

Say you've decided that you care about the classification accuracy of your cat's classifier, this could have been an F1 score or some other measure of accuracy, but let's say you also care about running time in addition to accuracy.

Classifier	Accuracy	Running time
A	90%	80 ms
B	92%	95 ms
C	95%	1 500 ms

You may want to select a classifier that maximizes accuracy, but subject to running time, that is the time it takes to classify an image that must be less than 100 milliseconds or equal to it. So in this case, we would say that accuracy is a metric that optimizes, because you want to maximize accuracy. In terms of accuracy you want to do as well as possible so that run time is what we call a satisfying metric. That is, it just needs to be nice enough, it just needs to be less than 100 milliseconds and beyond that you just don't care about, or at least you don't care that much. That will therefore be a fairly reasonable way to trade off or put off Both accuracy and running time together.

And by defining matrix optimization and matrix satisfying, This allows you to choose the best classifier, Which will be classifier B in this situation, because of all those with It has better running time than 100 milliseconds and the best accuracy.

In this case, accuracy and running time are the evaluation matrices. Accuracy is the optimizing metric, because you want the classifier to correctly detect a cat image as accurately as possible. The running time which is set to be under 100 ms in this example,is the satisficing metricwhich mean that the metric has to meet expectation set.

AWS Model Registry

With the Amazon SageMaker Model Registry you can do the following:

- Catalog models for production.
- Manage model versions.
- Associate metadata, such as training metrics, with a model.
- View information from Amazon SageMaker Model Cards in your registered models.
- Manage the approval status of a model.
- Deploy models to production.
- Automate model deployment with CI/CD.
- Share models with other users.

Catalog models by creating SageMaker Model Registry Model (Package) Groups that contain different versions of a model. You can create a Model Group that tracks all of the models that you train to solve a particular problem. You can then register each model you train and the Model Registry adds it to the Model Group as a new model version. Lastly, you can create categories of Model Groups by further organizing them into SageMaker Model Registry Collections. A typical workflow might look like the following:

- Create a Model Group.
- Create an ML pipeline that trains a model. For information about SageMaker pipelines, see [Create and Manage SageMaker Pipelines](#).
- For each run of the ML pipeline, create a model version that you register in the Model Group you created in the first step.
- Add your Model Group into one or more Model Registry Collections.

Amazon SageMaker Inference Pipelines

- **End-to-End Workflows:** Combine preprocessing, model inference, and postprocessing in a single endpoint.
- **Multiple Containers:** Use multiple containers for different stages of the pipeline.
- **Chaining Models:** Chain together multiple models for sequential predictions.
- **Simplified Deployment:** Deploy complex ML workflows easily.
- **Flexible Configurations:** Supports various configurations for different pipeline stages.
- **Cost Efficiency:** Optimize costs by sharing resources across pipeline stages.

SageMaker Inference Pipelines with MLeap and Scikit-Learn

- **MLeap Serialization:** Enables serializing Scikit-Learn models for low-latency predictions.
- **Multi-Container Deployment:** Combines MLeap and Scikit-Learn models in a single inference pipeline.
- **Data Transformation:** Performs feature transformations using MLeap before model inference.
- **Integration:** Seamlessly integrates with SageMaker endpoints for production deployment.
- **Use Case:** Ideal for complex preprocessing and model inference workflows.

Creating an Inference Pipeline Using the Console

- **Step-by-Step Guide:** Provides detailed instructions for creating an inference pipeline in the SageMaker console.
- **Model Registration:** Register your models and create model entities.

- **Pipeline Configuration:** Set up the pipeline with pre-processing, inference, and post-processing steps.
- **Endpoint Deployment:** Deploy the pipeline to a SageMaker endpoint.
- **Testing:** Validate the pipeline by testing the endpoint with sample data.

Real-Time Inference Pipeline

- **Real-Time Processing:** Enables real-time data preprocessing, model inference, and postprocessing.
- **Multi-Model Deployment:** Supports multiple models within a single endpoint.
- **Step Configuration:** Configure preprocessing, inference, and postprocessing steps.
- **SageMaker Integration:** Fully integrates with SageMaker endpoints for seamless deployment.
- **Scalability:** Automatically scales based on real-time traffic.

Batch Inference Pipeline

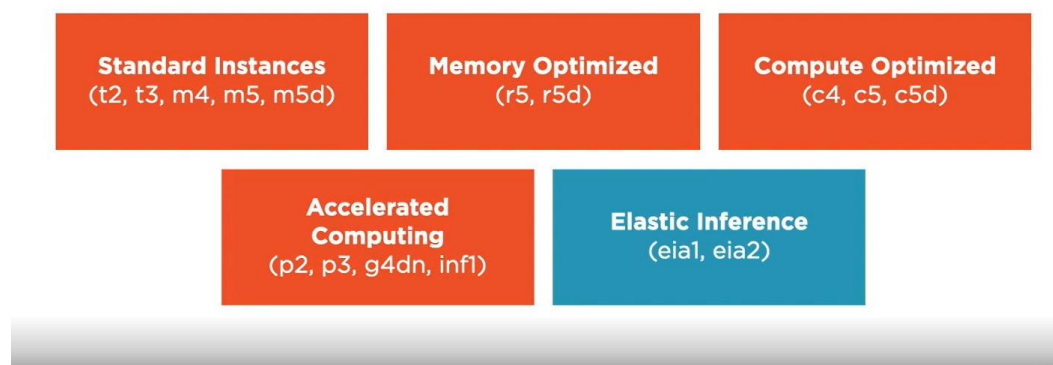
- **Batch Processing:** Enables batch data preprocessing, inference, and postprocessing.
- **Multi-Model Support:** Allows multiple models in one pipeline for batch jobs.
- **Pipeline Configuration:** Set up with preprocessing, inference, and postprocessing steps.
- **S3 Integration:** Uses Amazon S3 for input and output data storage.
- **Scalability:** Automatically scales to handle large datasets.

Monitoring and Logging for Inference Pipelines

- **CloudWatch Integration:** Logs and metrics are automatically captured in Amazon CloudWatch.
- **Logs:** Capture detailed logs for preprocessing, inference, and postprocessing steps.
- **Metrics:** Track performance metrics such as latency, error rates, and throughput.
- **Custom Metrics:** Enable custom metrics to monitor specific aspects of the inference pipeline.
- **Troubleshooting:** Use logs and metrics for diagnosing and resolving issues.

Scaling Sagemaker Endpoints

SageMaker Instance Type Families



SageMaker Autoscaling

Can be configured in the console,
through the CLI, or with an API

Utilizes CloudWatch metrics to define
the scaling policy

An initial metric to utilize for scaling is
SageMakerVariantInvocationsPerInstance

- The average times per minute that each instance is invoked for a variant

Additional metrics could be utilized

- CPUUtilization

Deep Learning Containers

Deep Learning Containers

Docker containers that are pre-configured for common machine learning frameworks that can be leveraged with Amazon Elastic Compute Cloud (EC2), Amazon Elastic Container Service (ECS), or Amazon Elastic Kubernetes Service (EKS).

Deep Learning Containers

Versions of each framework for Elastic Inference are also included

Deep learning containers can support both training and inference

Fault tolerant architecture following service best practices (EC2, ECS, EKS)